Unit-1

Need of Network Security:

The network needs security against attackers and hackers.

Network Security includes two basic securities. The first is the security of data information i.e., to protect the information from unauthorized access and loss. And

The second is computer security i.e., to protect data from hackers.

On internet or any network of an organization, thousands of important information is exchanged daily. This information can be misused by attackers. The information security is needed for the following given reasons.

Network Security:

1. To protect the secret information users on the net only. No other person should see or access it.
2. To protect the information from unwanted editing, accidentally or intentionally by unauthorized users.
3. To protect the information from loss and make it to be delivered to its destination properly.
4. To manage for acknowledgement of message received by any node in order to protect from denial by sender in specific situations. For example, let a customer order to purchase a few shares XYZ to the broader and denies for the order after two days as the rates go down.
5. To restrict a user to send some message to another user with name of a third one. For example, a user X for his own interest makes a message containing some favorable instructions and sends it to user Y in such a manner that Y accepts the message as coming from Z, the manager of the organization.
6. To protect the message from unwanted delay in the transmission lines/route in order to deliver it to required destination in time, in case of urgency.
7. To protect the data from wandering the data packets or information packets in the network for infinitely long time and thus increasing

congestion in the line in case destination machine fails to capture it because of some internal faults.

Computer Security:

1.It should be protected from replicating and capturing viruses from infected files.

2. It needs a proper protection from worms and bombs.

3. There is a need of protection from Trojan Horses as they are enough dangerous for your computer.

Security Approaches:

1.Trusted System.

2. Security Model

3.Security-Management Practices

1.Trusted System:

Is a Computer system that can be trusted to specified extent to enforce a special field security policy.

It used term reference monitor which logical heart of computer. It has following expectation.

1. It should be Tamper-proof.
2. It should always invoke.
3. It should be small enough so that it can be tested independently.

2.Security model:

- No Security: No security at all implemented.
- Security through Obscurity: This system is secure because nobody knows about its existence and contents. This approach cannot work for too long as there are many ways an attacker can come to know about it.
- In this security for each host is enforced individually .this is very safe approach, but the trouble is that it cannot scale well. The complexity and diversity of modern sites/organization makes the task even harder.
- Network Security: host security is tough to achieve as organization to grow and become more and become more diverse .In this technique focus is to control network access to various host their services ,rather than the individual host security. This is very efficient and scalable model.

3.Good Security-Management:

The practices always talk of a security policy. It has following aspects.

- Affordability
- Functionality
- Cultural issues
- Legality

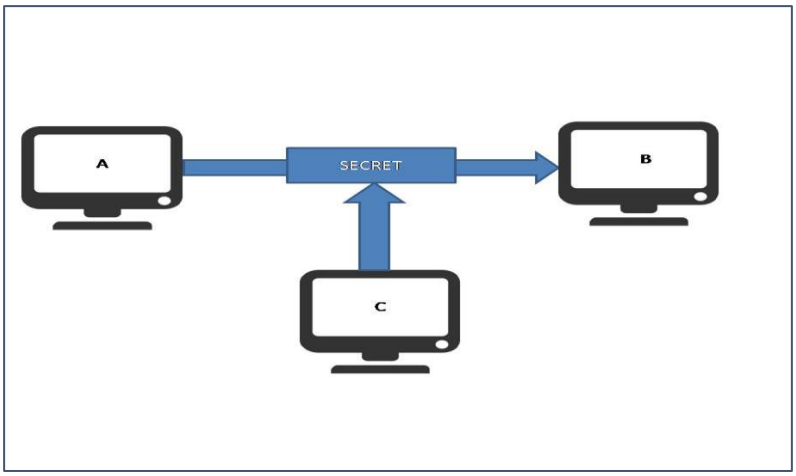
Once a security policy is in place the following point should be ensured.

1. Explanation of the to all concerned.
2. Outline everybody's responsibilities.
3. Use simple language in al communication.
4. Provide for exceptions and periodic reviews
5. Accountability should be established

Principles of Security

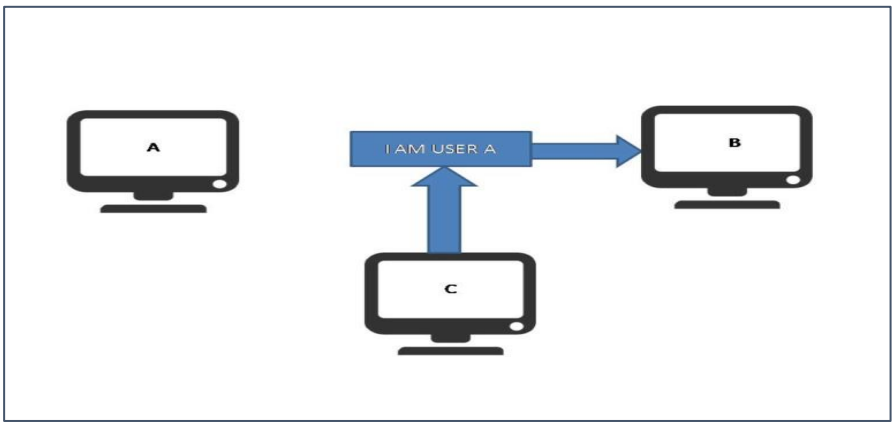
1.Confidentiality:

The degree of confidentiality determines the secrecy of the information. The principle specifies that only the sender and receiver will be able to access the information shared between them. Confidentiality compromises if an unauthorized person is able to access a message. For example, let us consider sender A wants to share some confidential information with receiver B and the information gets intercepted by the attacker C. Now the confidential information is in the hands of an intruder C.



2.Authentication:

Authentication is the mechanism to identify the user or system or the entity. It ensures the identity of the person trying to access the information. The authentication is mostly secured by using username and password. The authorized person whose identity is pre-registered can prove his/her identity and can access the sensitive information.

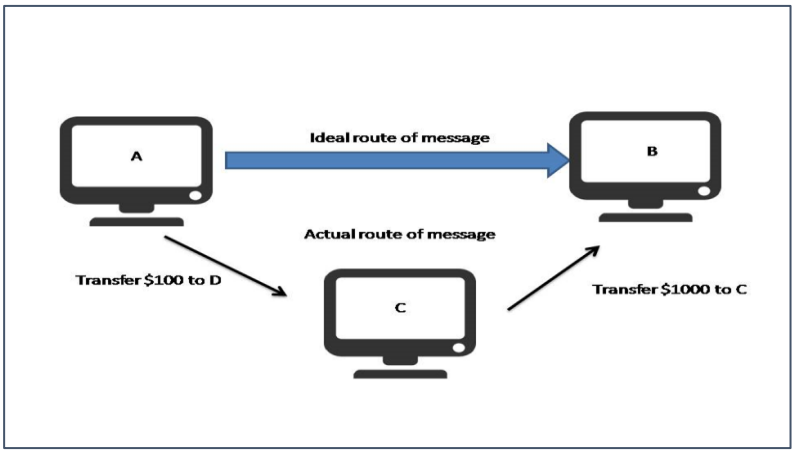


3.Integrity:

Integrity gives the assurance that the information received is exact and accurate. If the content of the message is changed after the sender sends it but before reaching the intended receiver, then it is said that the integrity of the message is lost.

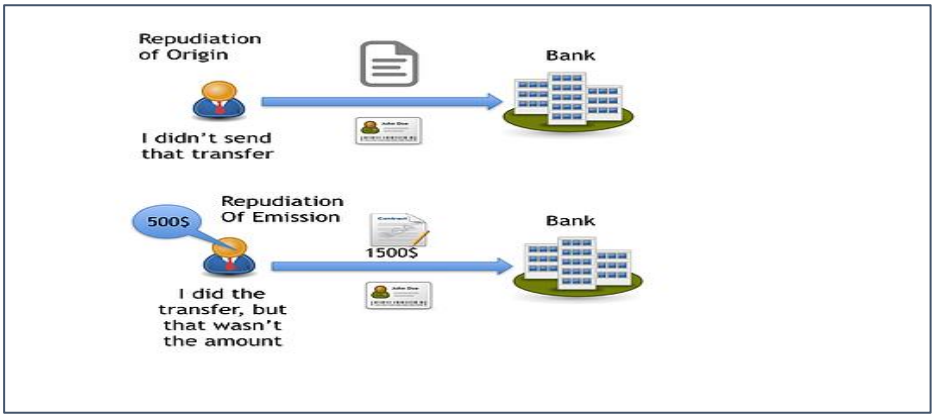
System Integrity: System Integrity assures that a system performs its intended function in an unimpaired manner, free from deliberate or inadvertent unauthorized manipulation of the system.

Data Integrity: Data Integrity assures that information (both stored and in transmitted packets) and programs are changed only in a specified and authorized manner.



4.Non-Repudiation:

Non-repudiation is a mechanism that prevents the denial of the message content sent through a network. In some cases, the sender sends the message and later denies it. But the non-repudiation does not allow the sender to refuse the receiver.



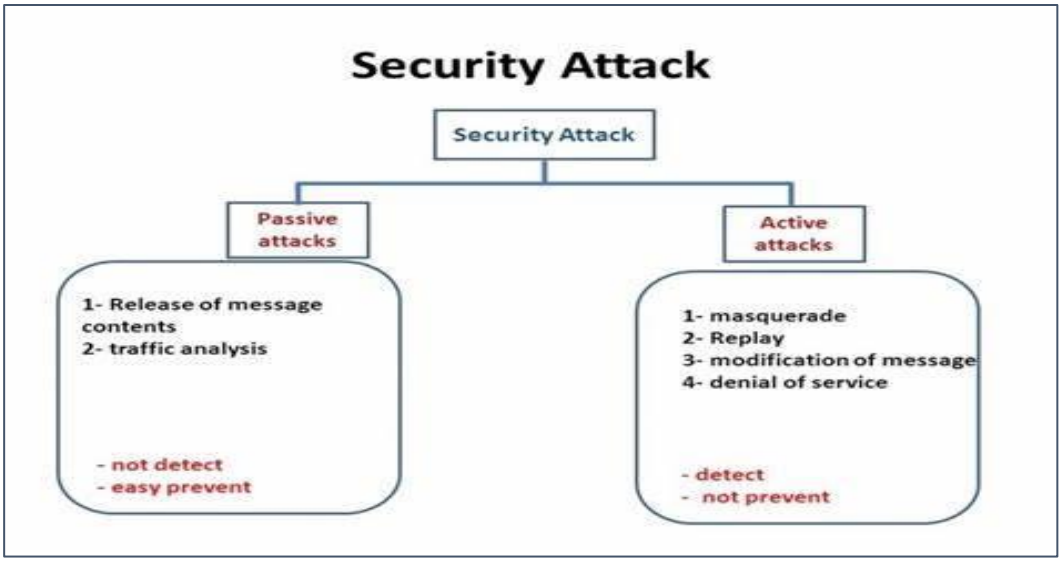
5.Availability:

The principle of availability states that the resources will be available to authorize party at all times. Information will not be useful if it is not available to be accessed. Systems should have sufficient availability of information to satisfy the user request.

6.Access control:

The principle of access control is determined by role management and rule management. Role management determines who should access the data while rule management determines up to what extent one can access the data. The information displayed is dependent on the person who is accessing it.

Types of security Attacks:



1. Active Attacks:

An Active attack attempts to alter system resources or affect their operations. Active attacks involve some modification of the data stream or the creation of false statements. Active attacks involve an attacker intentionally altering or destroying data, or disrupting the normal operation of a system. Examples of active attacks include denial of service (DoS), where an attacker floods a system with traffic in an attempt to make it unavailable to legitimate users, and malware, where an attacker installs malicious software on a system to steal or destroy data.

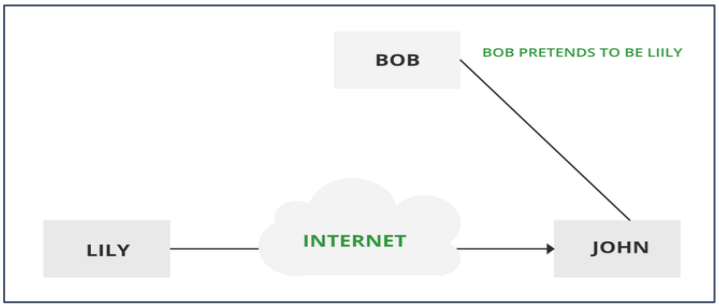
Types of active attacks are as follows:

- Masquerade
- Modification of messages
- Repudiation
- Replay
- Denial of Service

1. Masquerade:

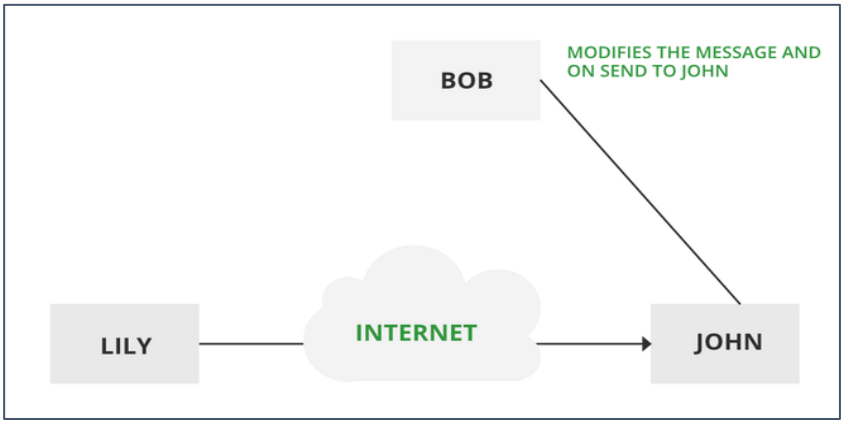
A masquerade attack takes place when one entity pretends to be a different entity. A Masquerade attack involves one of the other forms of active attacks. If an authorization procedure is not always absolutely protected, it is able to grow to be extraordinarily liable to a masquerade assault. Masquerade assaults may be performed using the

stolen passwords and logins, with the aid of using finding gaps in programs, or with the aid of using locating a manner across the authentication process.



2. Modification of messages:

It means that some portion of a message is altered or that message is delayed or reordered to produce an unauthorized effect. Modification is an attack on the integrity of the original data.
It basically means that unauthorized parties not only gain access to data but also spoof the data by triggering denial-of-service attacks, such as altering transmitted data packets or flooding the network with fake data. Manufacturing is an attack on authentication. For example, a message meaning “Allow JOHN to read confidential file X” is modified as “Allow Smith to read confidential file X.”



3. Repudiation:

This attack occurs when the network is not completely secured or the login control has been tampered with. With this attack, the author’s information can be changed by actions of a malicious user in order to save false data in log files, up to the general manipulation of data on behalf of others, similar to the spoofing of e-mail messages.

4. Replay:

It involves the passive capture of a message and its subsequent transmission to produce an authorized effect. In this attack, the basic aim of the attacker is to save a copy of the data originally present on that particular network and later on use this data for personal uses. Once the data is corrupted or leaked it is insecure and unsafe for the users.

5. Denial of Service:

It prevents the normal use of communication facilities. This attack may have a specific target. For example, an entity may suppress all messages directed to a particular destination. Another form of service denial is the disruption of an entire network either by disabling the network or by overloading it with messages so as to degrade performance.

2. Passive Attacks:

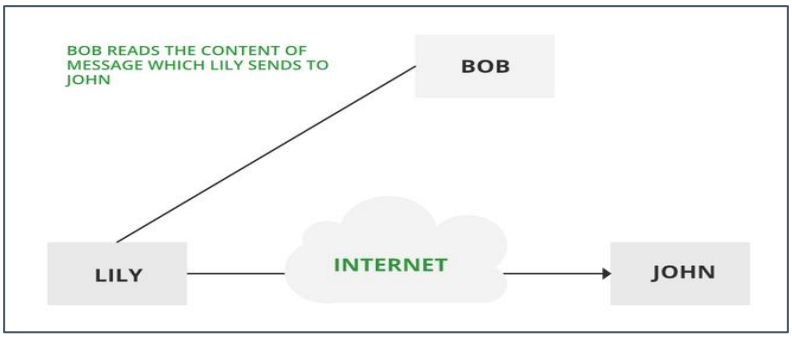
A Passive attack attempts to learn or make use of information from the system but does not affect system resources. Passive Attacks are in the nature of eavesdropping on or monitoring transmission. The goal of the opponent is to obtain information that is being transmitted. Passive attacks involve an attacker passively monitoring or collecting data without altering or destroying it. Examples of passive attacks include eavesdropping, where an attacker listens in on network traffic to collect sensitive information, and sniffing, where an attacker captures and analyzes data packets to steal sensitive information.

Types of Passive attacks are as follows:

- The release of message content
- Traffic analysis

1. **The release of message content:**

Telephonic conversation, an electronic mail message, or a transferred file may contain sensitive or confidential information. We would like to prevent an opponent from learning the contents of these transmissions.



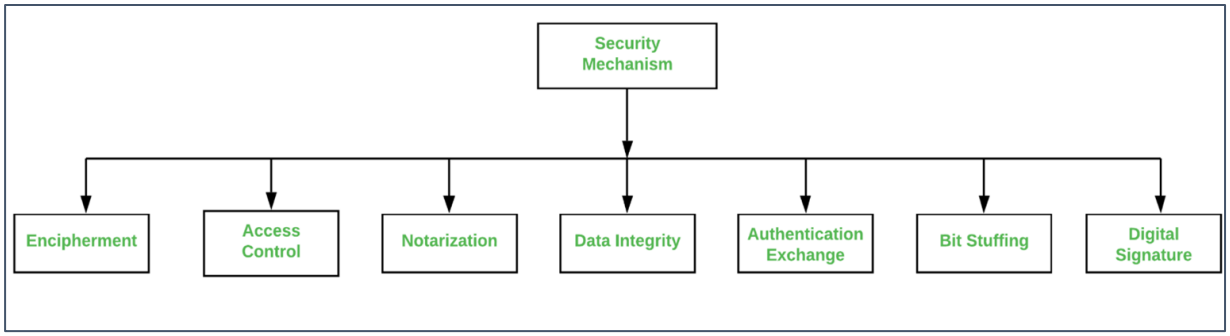
2. **Traffic analysis:**

Suppose that we had a way of masking (encryption) information, so that the attacker even if captured the message could not extract any information from the message.

The opponent could determine the location and identity of communicating host and could observe the frequency and length of messages being exchanged. This information might be useful in guessing the nature of the communication that was taking place.

The most useful protection against traffic analysis is encryption of SIP traffic. To do this, an attacker would have to access the SIP proxy (or its call log) to determine who made the call.

Security Mechanism:



Network Security is field in computer technology that deals with ensuring security of computer network infrastructure. As the network is very necessary for sharing of information whether it is at hardware level such as printer, scanner, or at software level. Therefore, security mechanism can also be termed as is set of processes that deal with recovery from security attack. Various mechanisms are designed to recover from these specific attacks at various protocol layers.

1.Encipherment :

This security mechanism deals with hiding and covering of data which helps data to become confidential. It is achieved by applying mathematical calculations or algorithms which reconstruct information into not readable form. It is achieved by two famous techniques named Cryptography and Encipherment. Level of data encryption is dependent on the algorithm used for encipherment.

2.Access Control :

This mechanism is used to stop unattended access to data which you are sending. It can be achieved by various techniques such as applying passwords, using firewall, or just by adding PIN to data.

3.Notarization :

This security mechanism involves use of trusted third party in communication. It acts as mediator between sender and receiver so that if any chance of conflict is reduced. This mediator keeps record of requests made by sender to receiver for later denied.

4.Data Integrity :

This security mechanism is used by appending value to data to which is created by data itself. It is similar to sending packet of information known to both sending and receiving parties and checked before and after data is received. When this packet or data which is appended is checked and is the same while sending and receiving data integrity is maintained.

5.Authentication exchange :

This security mechanism deals with identity to be known in communication. This is achieved at the TCP/IP layer where two-way handshaking mechanism is used to ensure data is sent or not.

Cryptography

Cryptography is technique of securing information and communications through use of codes so that only those person for whom the information is intended can understand it and process it. Thus, preventing unauthorized access to information. The prefix “crypt” means “hidden” and suffix “graphy” means “writing.” In Cryptography the techniques which are used to protect information are obtained from mathematical concepts and a set of rule-based calculations known as algorithms to convert messages in ways that make it hard to decode it. These algorithms are used for cryptographic key generation, digital signing, verification to protect data privacy, web browsing on internet and to protect confidential transactions such as credit card and debit card transactions.

Techniques used For Cryptography:

In today’s age of computers cryptography is often associated with the process where an ordinary plain text is converted to cipher text which is the text made such that intended receiver of the text can only decode it and hence this process is known as encryption. The process of conversion of cipher text to plain text this is known as decryption.

Features Of Cryptography are as follows:

1. **Confidentiality:**
   Information can only be accessed by the person for whom it is intended and no other person except him can access it.

2. **Integrity:**
   Information cannot be modified in storage or transition between sender and intended receiver without any addition to information being detected.

3. **Non-repudiation:**
   The creator/sender of information cannot deny his intention to send information at later stage.

4. **Authentication:**
   The identities of sender and receiver are confirmed. As well as destination/origin of information is confirmed.

Types Of Cryptography:

In general, there are three types of cryptography:

1. **Symmetric Key Cryptography:**

   It is an encryption system where the sender and receiver of message use a single common key to encrypt and decrypt messages. Symmetric Key Systems are faster and simpler but the problem is that sender and receiver have to somehow exchange key in a secure manner. The most popular symmetric key cryptography system are Data Encryption System(DES) and Advanced Encryption System(AES).

2. **Hash Functions:**

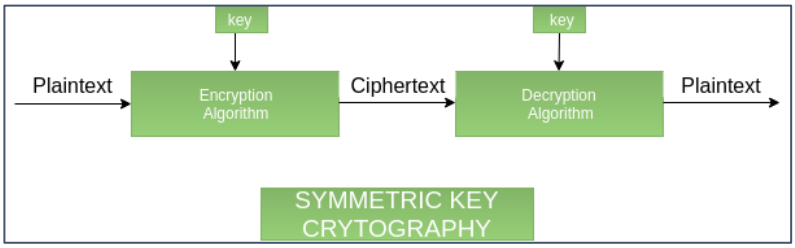
   There is no usage of any key in this algorithm. A hash value with fixed length is calculated as per the plain text which makes it impossible for contents of plain text to be recovered. Many operating systems use hash functions to encrypt passwords.

3. **Asymmetric Key Cryptography:**

   Under this system a pair of keys is used to encrypt and decrypt information. A receiver’s public key is used for encryption and a receiver’s private key is used for decryption. Public key and Private Key are different. Even if the public key is known by everyone the intended receiver can only decode it because he alone know his private key. The most popular asymmetric key cryptography algorithm is RSA algorithm.

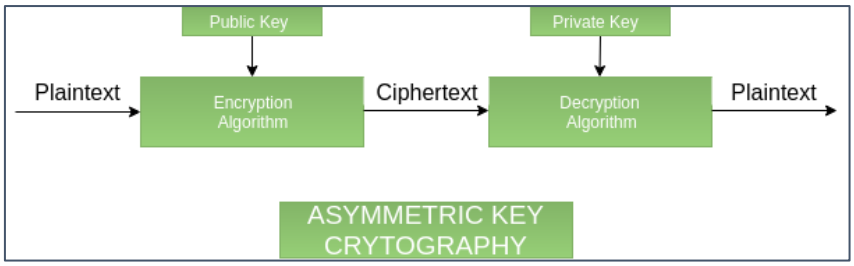
1. Symmetric Key Cryptography :

It involves the usage of one secret key along with encryption and decryption algorithms which help in securing the contents of the message. The strength of symmetric key cryptography depends upon the number of key bits. It is relatively faster than asymmetric key cryptography. There arises a key distribution problem as the key has to be transferred from the sender to the receiver through a secure channel.



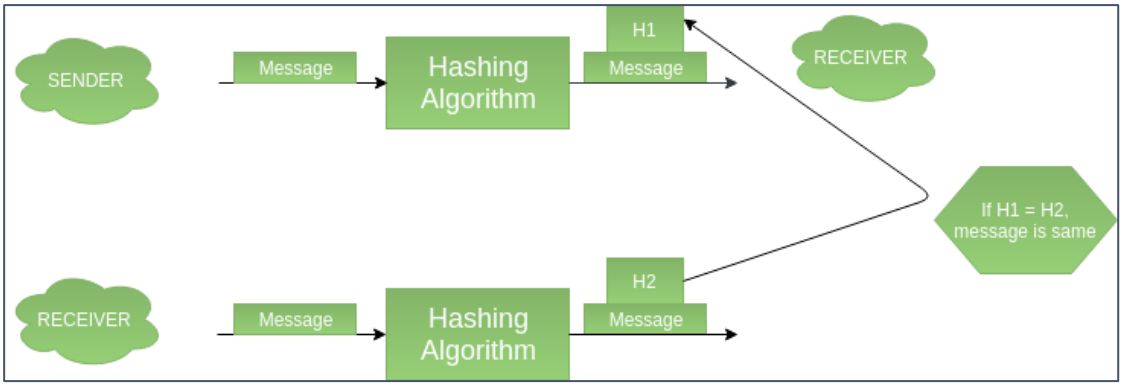
2. Asymmetric key cryptography:

It is also known as public-key cryptography because it involves the usage of a public key along with the secret key. It solves the problem of key distribution as both parties use different keys for encryption/decryption. It is not feasible to use for decrypting bulk messages as it is very slow compared to symmetric key cryptography.



3. Hashing:

It involves taking the plain text and converting it to a hash value of fixed size by a hash function. This process ensures the integrity of the message as the hash value on both, the sender’s and receiver’s sides should match if the message is unaltered.


SENDER
Message
Hashing Algorithm
H1
Message
RECEIVER
If H1 = H2, message is same
RECEIVER
Message
Hashing Algorithm
H2
Message

Substitution Technique:

What is Substitution Technique?

The ***substitution technique*** involves replacing letters with other letters and symbols. In simple terms, the plaintext characters are substituted, and additional substitute letters, numerals, and symbols are implemented in their place. The Caesar cipher employs the substitution technique. In this technique, the alphabet is substituted with the alphabet three positions forward of the line. The substitution cipher technique was invented by Julius Caesar and named after him as the Caesar Cipher.

Let's take an easy example to understand this technique. The plaintext ***"JUMP"*** will be turned into ***"MXPS"*** using ***Caesar Cipher***. Following the Caesar cipher, various substitution techniques were developed, including the Mono-alphabetic cipher, Polyalphabetic substitution cipher, Polygram substitution cipher, Playfair cipher, Homophobic substitution cipher, and Hill cipher.

The ***Caesar cipher*** was the weakest technique, but as the techniques evolved from time to time, the new version became stronger. The substitution technique's weakness is that it is highly predictable, and if the translation table is known, the substitution may be disrupted.

Features of Substitution Technique:

There are various features of the **substitution technique**. Some features of substitution techniques are as follows:

1. In the substitution cipher technique, the letters in plain text are substituted by other letters, numbers, or symbols.
2. A character's identity is changed, but its place remains constant in the substitution technique.
3. Some algorithms that use the substitution technique are monoalphabetic substitution cipher, Playfair cipher, and polyalphabetic substitution cipher.
4. The substitution cipher approach allows for the detection of plain text by low-frequency letters.
5. Caesar Cipher is an example of the substitution cipher technique.

Types of Substitution Technique:

1. **Caesar Cipher**

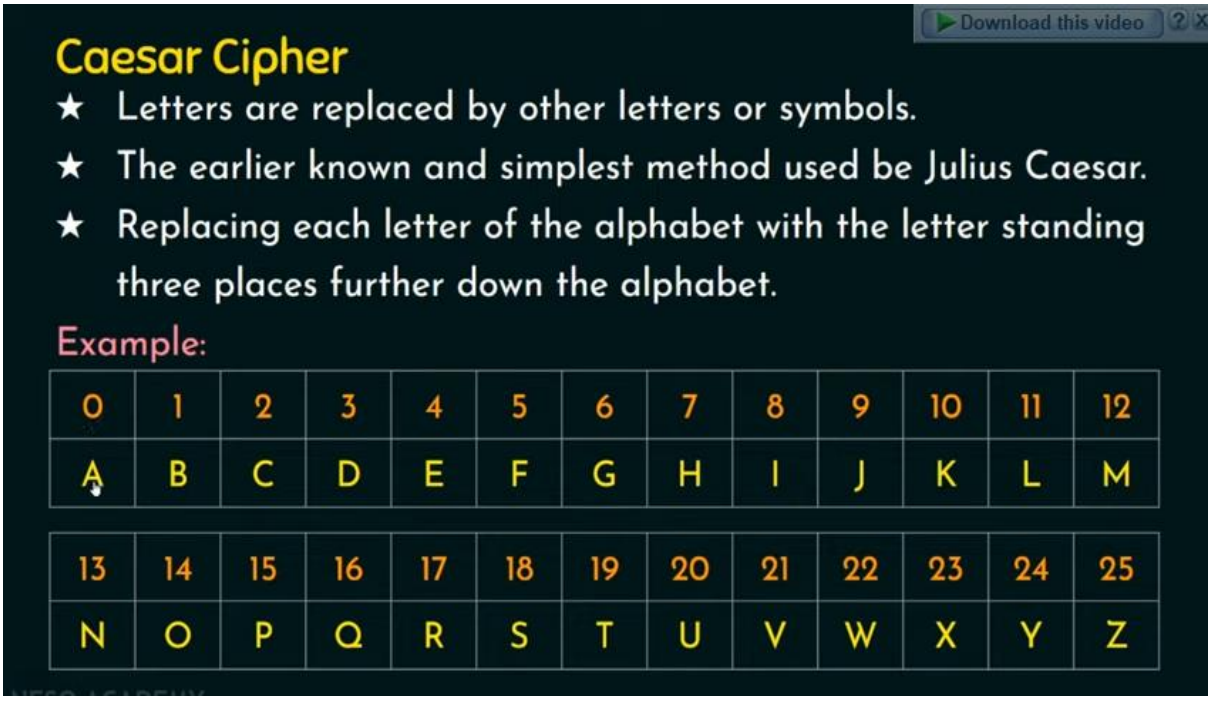
The earliest known use of a substitution cipher and the simplest was by JuliusCaesar. The Caesar cipher involves replacing each letter of the alphabet with the letter standing 3 places further down the alphabet. e.g., plain text : pay more money Cipher text: SDB PRUH PRQHB

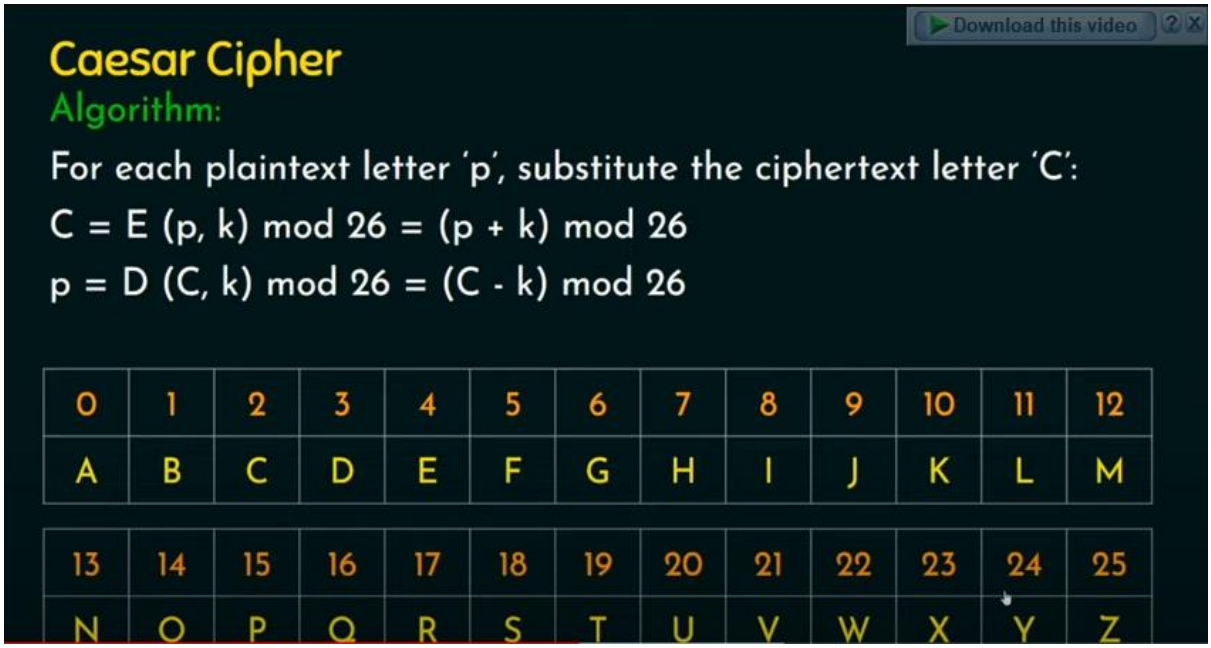
Note that the alphabet is wrapped around, so that letter following „z‟ is „a‟.

For each plaintext letter p, substitute the cipher text letter c such that C = E(p) =(p+3) mod 26

A shift may be any amount, so that general Caesar algorithm is C = E (p) = (p+k) mod26 Where k takes on a value in the range 1 to 25.

The decryption algorithm is simply P = D(C) = (C-k) mod 26





2. Monoalphabetic Cipher

Here, Plaintext characters are substituted by a different alphabet stream of characters shifted to the right or left by n positions. When compared to the Caesar ciphers, these monoalphabetic ciphers are more secure as each letter of the ciphertext can be any permutation of the 26 alphabetic characters leading to 26! or greater than 4 x 10^{26} possible keys. But it is still vulnerable to cryptanalysis, when a cryptanalyst is aware of the nature of the plaintext, he can find the regularities of the language. To overcome these attacks, multiple substitutions for a single letter are used. For example, a letter can be substituted by different numerical cipher symbols such as 17, 54, 69….. etc. Even this method is not completely secure as each letter in the plain text affects on letter in the ciphertext.

Or, using a common key which substitutes every letter of the plain text.

The key *ABCDEFGHIIJ*
KLMNOPQRSTUV
WXYZ
QWERTYUIIOPAS
DFGHJ KLZXCV
BNM

Would encrypt the message
II think therefore II am into

OZIIOFAZIITKTYGKTOQD

But any attacker would simply break the cipher by using frequency analysis by observing the number of times each letter occurs in the cipher text and then looking upon the English letter frequency table. So, substitution cipher is completely ruined by these attacks. Monoalphabetic ciphers are easy to break as they reflect the frequency of the original alphabet. A countermeasure is to provide substitutes, known as homophones for a single letter.

3. Play-Fair Cipher

It is the best-known multiple –letter encryption cipher which treats diagrams in the plaintext as single units and translates these units into ciphertext diagrams. The Playfair Cipher is a diagram substitution cipher offering a relatively weak method of encryption. It was used for tactical purposes by British forces in the Second Boer War and in World War I and for the same purpose by the Australians and Germans during World War II. This was because Playfair is reasonably fast to use and requires no special equipment. A typical scenario for Playfair use would be to protect important but non-critical secrets during actual combat. By the time the enemy cryptanalysts could break the message, the information was useless to them. It is based around a 5x5 matrix, a copy of which is held by both communicating parties, into which 25 of the 26 letters of the alphabet (normally either j and i are represented by the same letter or x is ignored) are placed in a random fashion. For example, the plain text is *ShiSherry loves*

***Heath Ledger* and the agreed key is *sherry*. The matrix will be built according to the following rules.**

- In pairs,
- Without punctuation,
- All Js are replaced with Is.

SH IS HE RR YL OV ES HE AT HL ED GE R

- Double letters which occur in a pair must be divided by an X or a Z.
- E.g., LI TE RA LL Y LI TE RA LX LY

SH IS HE RX RY LO VE SH EA TH LE DG ER The alphabet square is prepared using, a 5*5 matrix, no repetition letters, no Js and key is written first followed by the remaining alphabets with no i and j.

S H E R Y A B C D F G I K L M N O P Q T U V W X Z

For the generation of cipher text, there are three rules to be followed by each pair of letters.

Letters appear on the same row: Replace them with the letters to their immediate right respectively.

Letters appear on the same column: Replace them with the letters immediately below respectively.

Not on the same row or column: Replace them with the letters on the same row respectively but at the other pair of corners of the rectangle defined by the original pair. Based on the above three rules, the cipher text obtained for the given plain text is

HE GH ER DR YS IQ WH HE SC OY KR AL RY

Another example which is simpler than the above one can be given as:

Here, key word is *playfair*. Plaintext is *Hellothere hellothere* becomes ---- *he lx lo th er ex* .

Applying the rules again, for each pair, If they are in the same row, replace each with the letter to its right (mod 5)

he *KG*

If they are in the same column, replace each with the letter below it (mod 5)

lo *RV*

Otherwise, replace each with letter we'd get if we swapped their column indices

lx *YV*

So, the cipher text for the given plain text is **KG YV RV QM GI KU**

$$\begin{bmatrix} p & l & a & y & f \\ i & r & b & c & d \\ e & g & h & k & m \\ n & o & q & s & t \\ u & v & w & x & z \end{bmatrix}$$

To decrypt the message, just reverse the process. Shift up and left instead of down and right. Drop extra x's and locate any missing I's that should be j's. The message willbe back into the original readable form. no longer used by military forces because of the advent of digital encryption devices. Playfair is now regarded as insecure for any purpose because modern hand-held computers could easily break the cipher within seconds.

4. Hill Cipher

It is also a multi letter encryption cipher. It involves substitution of '*m*' ciphertext letters for '*m'* successive plaintext letters. For substitution purposes using '*m'* linear equations, each of the characters are assigned a numerical values i.e., a=0, b=1, c=2, d=3,…….z=25.

For example, if m=3, the system can be defined as:
c1 = (k11p1+ k12p2 + k13p3) mod 26 c2 = (k21p1 + k22p2 + k23p3) mod 26 c3 = (k31p1 + k32p2 + k33p3) mod 26

If we represent in matrix form, the above statements as matrices and column vectors:

$$\begin{pmatrix} c_1 \\ c_2 \\ c_3 \end{pmatrix} = \begin{pmatrix} k_{11} & k_{12} & k_{13} \\ k_{21} & k_{22} & k_{23} \\ k_{31} & k_{32} & k_{33} \end{pmatrix} \begin{pmatrix} p_1 \\ p_2 \\ p_3 \end{pmatrix} \text{ mod } 26$$

Thus, C = KP *mod26,*

Where;

- C= Column vectors of length 3
- P = Column vectors of length 3
- K= 3x3 encryption key matrix. For decryption process, inverse of matrix **K**

i.e., **K-1** is required which is defined by the equation KK-1 = K-1K = I,

Where;

- I is the identity matrix that contains only 0's and 1's as its elements.

Plaintext is recovered by applying K-1 to the cipher text.

It is expressed as

C = EK(P) = KP *mod26*

P = DK(C) = K-1

C *mod26.* = K-1

KP = IP =P

Example: The plain text is I can't do it and the size of m is 3 and key K is chosen as following

I can't do it

8 2 0 13 19 3 14 8 19

$$\begin{pmatrix} 9 & 18 & 10 \\ 16 & 21 & 1 \\ 5 & 12 & 23 \end{pmatrix}$$

The encryption process is carried out as follows

$$\begin{pmatrix} 4 \\ 14 \\ 12 \end{pmatrix} = \begin{pmatrix} 9 & 18 & 10 \\ 16 & 21 & 1 \\ 5 & 12 & 23 \end{pmatrix} \begin{pmatrix} 8 \\ 2 \\ 0 \end{pmatrix} \pmod{26}$$

$$\begin{pmatrix} 19 \\ 12 \\ 14 \end{pmatrix} = \begin{pmatrix} 9 & 18 & 10 \\ 16 & 21 & 1 \\ 5 & 12 & 23 \end{pmatrix} \begin{pmatrix} 13 \\ 19 \\ 3 \end{pmatrix} \pmod{26}$$

$$\begin{pmatrix} 18 \\ 21 \\ 9 \end{pmatrix} = \begin{pmatrix} 9 & 18 & 10 \\ 16 & 21 & 1 \\ 5 & 12 & 23 \end{pmatrix} \begin{pmatrix} 14 \\ 8 \\ 19 \end{pmatrix} \pmod{26}$$

So, the encrypted text will be given as → **EOM TMY SVJ**

The Main Advantages of hill cipher are given below:

- Perfectly hides single-letter frequencies.
- It Use of **3x3** Hill ciphers can perfectly hide both the single letter and two-letter frequency information.
- Strong enough against the attacks made only on the cipher text.
- But it still can be easily broken if the attack is through a known plaintext.

5. Polyalphabetic Cipher

In order to make substitution ciphers more secure, more than one alphabet can be used. Such ciphers are called **polyalphabetic**, which means that the same letter of a message can be represented by different letters when encoded. Such a one-to-many correspondence makes the use of frequency analysis much more difficult in order to crack the code. We describe one such cipher named for Blaise de Vigenere a 16-th century Frenchman. The **Vigenere cipher** is a polyalphabetic cipher based on using successively shifted alphabets, a different shifted alphabet for each of the 26 English letters. The procedure is based on the tableau shown below and the use of a keyword. The letters of the keyword determine the shifted alphabets used in the encoding process.

A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A
C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B
D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C
E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D
F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E
G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F
H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G
I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H
J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I
K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J
L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K
M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L
N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M
O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N
P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q
S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R
T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S
U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T
V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U
W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V
X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W
Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X
Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y

For the message COMPUTING GIVES INSIGHT and keyword LUCKY

We proceed by repeating the keyword as many times as needed above the message, as follows.

L	U	C	K	Y	L	U	C	K	Y	L	U	C	K	Y	L	U	C	K	Y	L
C	O	M	P	U	T	I	N	G	G	I	V	E	S	I	N	S	I	G	H	T

Encryption is simple:
Given a key letter x and a plaintext letter y, the ciphertext letter is at the intersection of the row labeled *x* and the column labeled *y*; so, for L, the ciphertext letter would be N. So, the ciphertext for the given plaintext would be given as:

L	U	C	K	Y	L	U	C	K	Y	L	U	C	K	Y	L	U	C	K	Y	L			
C	O	M	P	U	T	I	N	G	G	I	V	E	S	I	N	S	I	G	H	T			<==MESSAGE
N	I	O	Z	S	E	C	P	Q	E	T	P	G	C	G	Y	M	K	Q	F	E			<==Encoded Message

Decryption is equally simple:
The key letter again identifies the row and position of ciphertext letter in that row decides the column and the plaintext letter is at the top of that column. The strength of this cipher is that there are multiple ciphetext letters for each plaintext letter, one for each unique letter of the keyword and thereby making the letter frequency information is obscured. Still, breaking this cipher has been made possible because this reveals some mathematical principles that apply in cryptanalysis. To overcome the drawback of the periodic nature of the keyword, a new technique is proposed which is referred as an autokey system, in which a key word is concatenated with the plaintext itself to provide a running key.

For Example:
In the above example, the key would be *luckycomputinggivesin* Still, this scheme is vulnerable to cryptanalysis as both the key and plaintext share the same frequency distribution of letters allowing a statistical technique to be applied.

Thus, the ultimate defense against such a cryptanalysis is to choose a keyword that is as long as plaintext and has no statistical relationship to it. A new system which works on binary data rather than letters is given as:

$C_i = p_i \; k_i$ where, p_i = ith binary digit of plaintext k_i = ith binary digit of key C_i= ith

binary digit of ciphertext = = exclusive-or operation. Because of the properties of XOR, decryption is done by performing the same bitwise operation.

$p_i = C_i \; k_i$ A very long but, repeation key word is used making cryptanalysis difficult.

Transposition Technique:

What is Transposition Technique?

In the ***transposition technique***, the characters' identities are kept the same, but their positions are altered to produce the ciphertext. A transposition cipher in cryptography is a type of encryption that scrambles the locations of characters without altering the characters themselves. Transposition ciphers produce a ciphertext that is a permutation of the plaintext by rearranging the components of the plaintext in accordance with a regular method. It is distinct from substitution ciphers, which don't replace the unit's positions of plaintext but instead substitute the units themselves. A bijective function is utilized to the character locations to encrypt data, and an inverse function is employed to decode data. It is not a very secure technique.

Rail Fence encryption is a sort of transposition cipher that acquires its name from how it is encrypted the data. The plaintext is written down and diagonally on successive ***"rail"*** of an artificial fence in the rail fence and then pushed up when you get to the bottom. After that, the message is read aloud in a row-by-row fashion.

The ***Rail Fence Cipher*** is based on an old ***Greek mechanical device*** for building a transposition cipher that follows a ***fairytale-like pattern***. The mechanism consisted of a cylinder with a ribbon wrapped around it. The encrypted message was written on the coiled ribbon. The characters of the original message were rearranged when the ribbon was uncoiled from the cylinder. The message was decrypted when the ribbon was wrapped in a cylinder with a similar diameter to the encrypting cylinder.

Features of the Transposition Technique:

There are various features of the ***transposition technique***. Some main features of the transposition techniques are as follows:

1. The keys that are closer to the proper key in the transposition cipher technique can reveal plain text.
2. The transposition cipher approach does not exchange one sign for another but rather moves the symbol.
3. The two most common types of transposition cipher are keyless and keyed transpositional cipher.

4. The Rail Fence Cipher is an excellent instance of a transposition technique.
5. The position of the character is modified in the transposition cipher technique, but the character's identity remains unchanged.

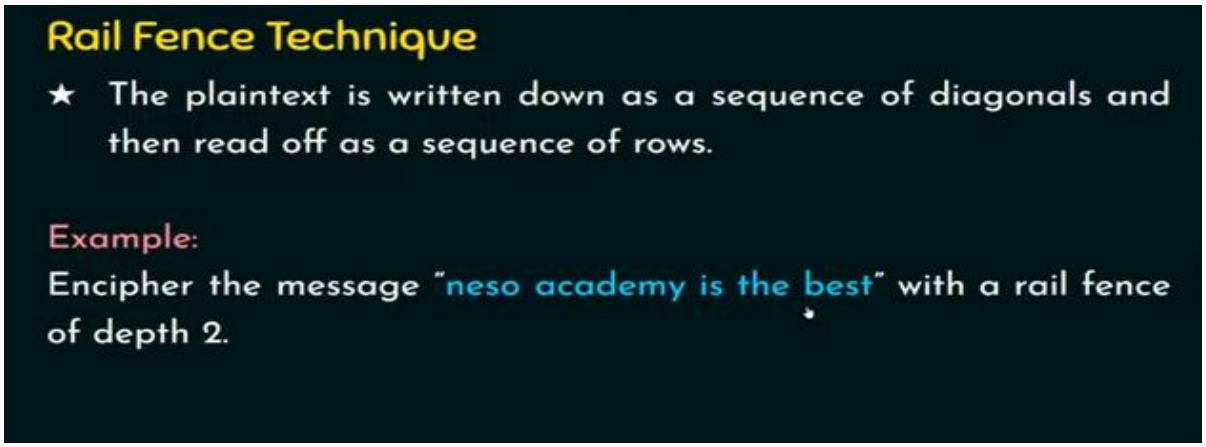
Types of Transposition Technique:

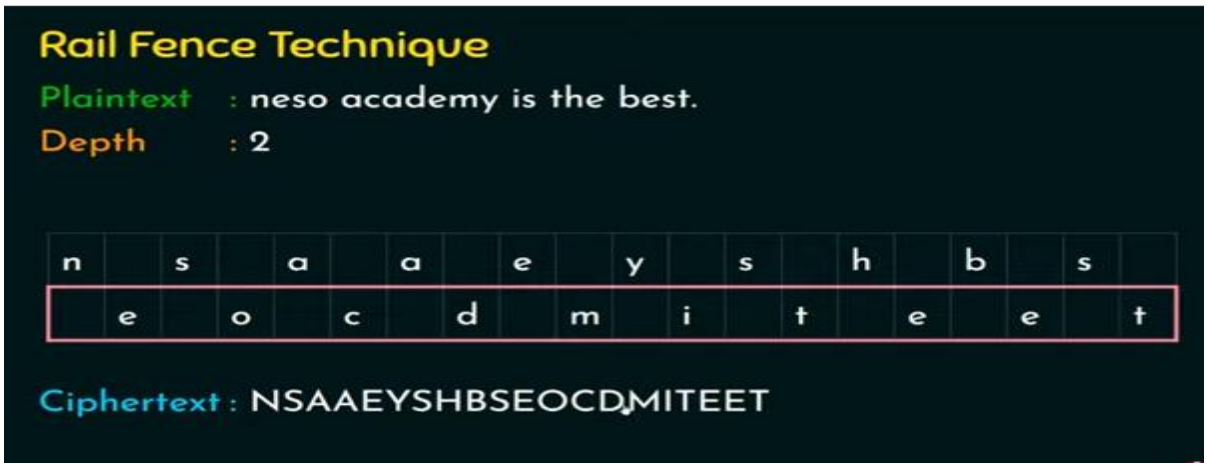
1. **Rail Fence:**
   It is simplest of such cipher, in which the plaintext is written down as a sequence of diagonals and then read off as a sequence of rows.

   Plaintext = meet at the school house

   To encipher this message with a rail fence of depth 2,
   We write the message as follows: m e a t e c o l o s e t t h s h o h u e

   The encrypted message is MEATECOLOSETTHSHOHUE





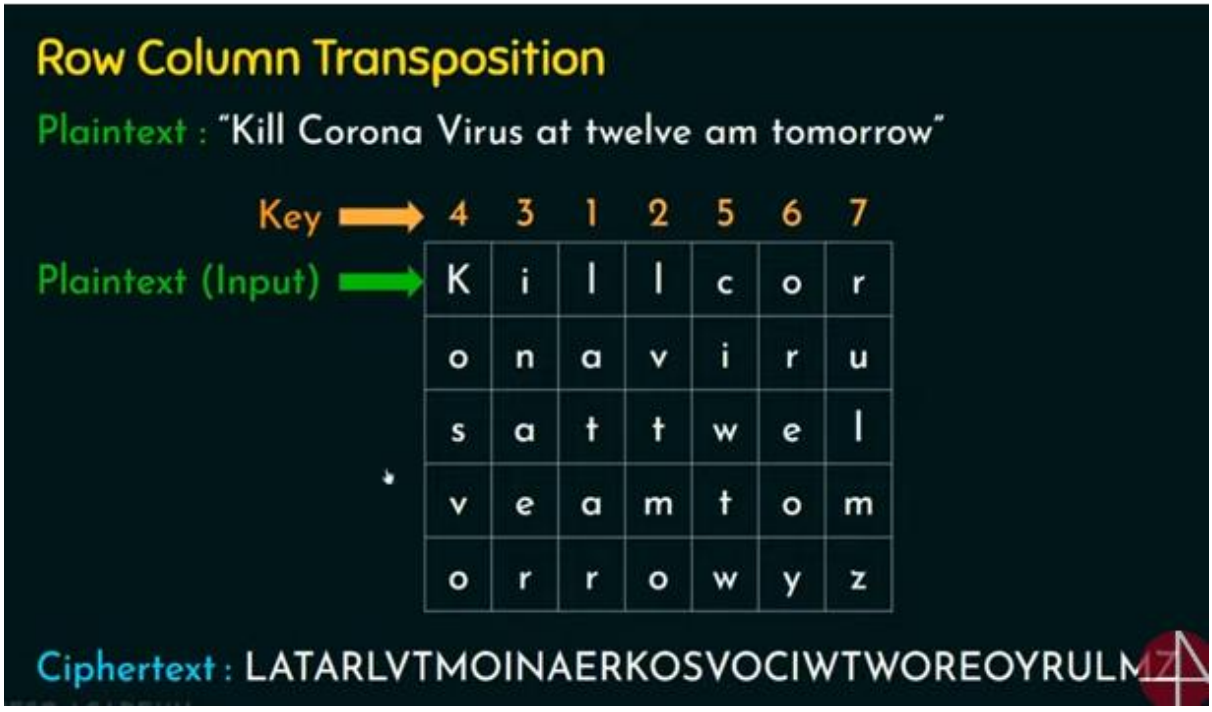
2. **Row Transposition Ciphers**-A more complex scheme is to write the message in a rectangle, row by row, and read the message off, column by column, but permute the order of the columns. The order of columns then becomes the key of the algorithm.

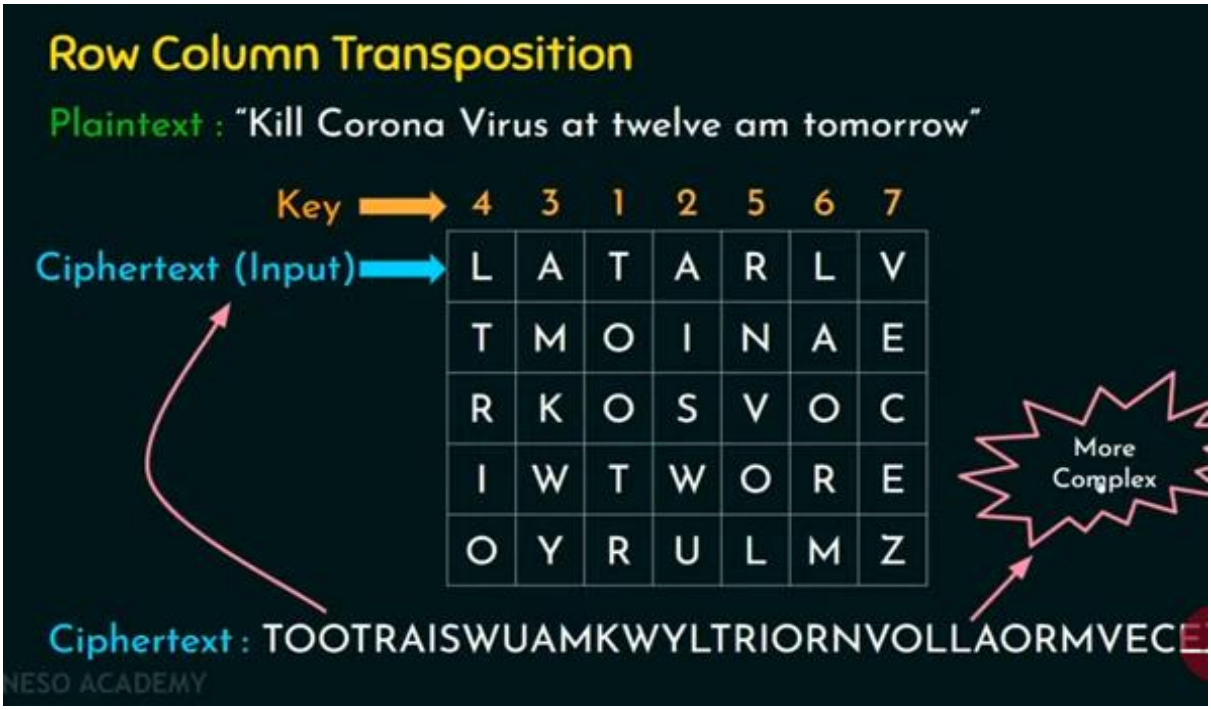
   Plain-Text = meet at the school house Key = 4 3 1 2 5 6 7

   PT = m e e t a t t h e s c h o o l h o u s e

   CT = ESOTCUEEHMHLAHSTOETO

   A pure transposition cipher is easily recognized because it has the same letter frequencies as the original plaintext. The transposition cipher can be made significantly more secure by performing more than one stage of transposition. The result is more complex permutation that is not easily reconstructed.





Features	Substitution Technique	Transposition Technique
Definition	It replaces the plaintext characters with other numbers, characters, and symbols.	It scrambles the character's position in the plaintext.
Alterations	The character's identity is changed, while its position does not change.	The character's identity is changed instead of its identity.
Forms	It utilizes the monoalphabetic, polyalphabetic substitution cipher, and Playfair cipher.	It utilizes the keyed and keyless transpositional ciphers.
Detection	The low-frequency letter may easily identify the plaintext.	The keys close to the right key lead to the discovery of the plaintext.
Examples	Caesar Cipher	Rail Fence Cipher

Steganography

A plaintext message may be hidden in any one of the two ways. The methods of steganography conceal the existence of the message, whereas the methods of cryptography render the message unintelligible to outsiders by various transformations of the text. A simple form of steganography, but one that is time consuming to construct is one in which an arrangement of words or letters within an apparently innocuous text spells out the real message. e.g., (i) the sequence of first letters of each word of the overall message spells out the real (hidden) message. (ii) Subset of the words of the overall message is used to convey the hidden message. Various other techniques have been used historically, some of them are

- **Character marking** – selected letters of printed or typewritten text are overwritten in pencil. The marks are ordinarily not visible unless the paper is held to an angle to bright light.
- **Invisible ink** – a number of substances can be used for writing but leave no visible trace until heat or some chemical is applied to the paper.
- **Pin punctures** – small pin punctures on selected letters are ordinarily not visible unless the paper is held in front of the light.
- **Typewritten correction ribbon** – used between the lines typed with a black ribbon, the results of typing with the correction tape are visible only under a strong light.

Drawbacks of Steganography:

- Requires a lot of overhead to hide a relatively few bits of information.
- Once the system is discovered, it becomes virtually worthless.

Unit-2

Block Cipher:

Block cipher is an encryption and decryption method which operates on the **blocks** of plain text, instead of operating on each bit of plain text separately. Each block is of equal size and has fixed no of bits. The generated ciphertext has blocks equal to the number of blocks in plaintext and also has the same number of bits in each block as of plain text. Block cipher uses the same key for encryption and decryption.

What is Block Cipher

Block cipher is an encryption method which divides the plain text into blocks of fixed size. Each block has an equal number of bits. At a time, block cipher operates only on one block of plain text and applies key on it to produce the corresponding block of ciphertext.

While decryption also only one block of ciphertext is operated to produce its corresponding plain text. Data Encryption Standard (DES) is the best example of it.

DES divides the plain text into the number of blocks, each of 64-bit. DES operates on one block of plain text at a time. Key of 56-bit is applied to each block of plain text to produce its corresponding ciphertext of 64-bit.

During decryption also only one block of ciphertext is operated at a time to produce its corresponding block plain text. In DES the decryption algorithm is the same as the encryption one.

On an all the block cipher operates on a block of bits at a time instead of one bit a time. Operating bit by bit is a very time-consuming process and as block cipher is a computer-based cryptographic algorithm it needs to be fast. That's why operating one block of bits at a time makes it faster as compared to a stream cipher.

But there was a limitation in the block cipher as it would generate the same ciphertext for the repeating text pattern in plain text. However, this limitation was resolved by implementing **chaining** in the block cipher.

Block Cipher Principles

A block cipher is designed by considering its three critical aspects which are listed as below:

1. Number of Rounds
2. Design of Function F
3. Key Schedule Algorithm

1. Number of Rounds

The number of rounds judges the strength of the block cipher algorithm. It is considered that more is the number of rounds, difficult is for cryptanalysis to break the algorithm.

It is considered that even if the function F is relatively weak, the number of rounds would make the algorithm tough to break.

2. Design of Function F

The function F of the block cipher must be designed such that it must be impossible for any cryptanalysis to unscramble the substitution. The criterion that strengthens the function F is it non-linearity.

More the function F is nonlinear, more it would be difficult to crack it. Well, while designing the function F it should be confirmed that it has a good avalanche property which states that a change in one-bit of input must reflect the change in many bits of output.

The Function F should be designed such that it possesses a bit independence criterion which states that the output bits must change independently if there is any change in the input bit.

3. Key Schedule Algorithm

It is suggested that the key schedule should confirm the strict avalanche effect and bit independence criterion.

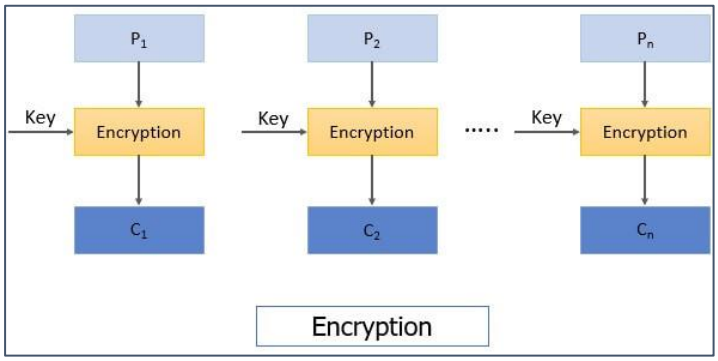
Block Cipher Modes of Operation

There are five important block cipher modes of operation defined by NIST. These five modes of operation enhance the algorithm so that it can be adapted by a wide range of applications which uses block cipher for encryption.

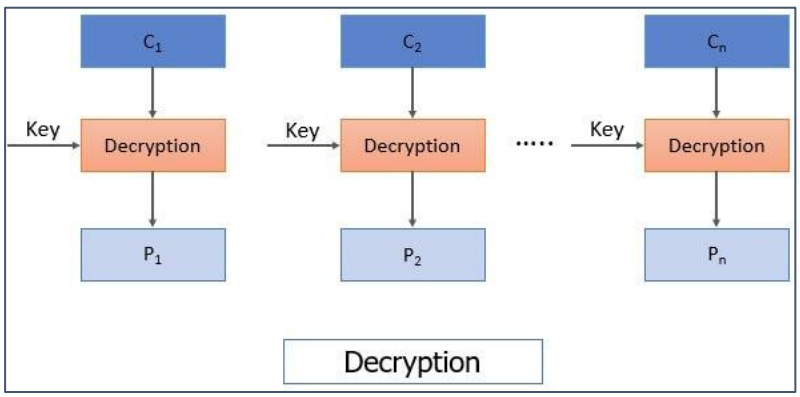
1. Electronic Code Book Mode
2. Cipher Block Chaining Mode
3. Cipher Feedback Mode
4. Output Feedback Mode
5. Counter Mode

1. Electronic Feedback Mode

This is considered to be the easiest block cipher mode of operation. In electronic codebook mode (ECB) the plain text is divided into the blocks, each of 64-bit. Each block is encrypted one at a time to produce the cipher block. The same key is used to encrypt each block.



When the receiver receives the message i.e., ciphertext. This ciphertext is again divided into blocks, each of 64-bit and each block is decrypted independently one at a time to obtain the corresponding plain text block. Here also the same key is used to decrypt each block which was used to encrypt each block.



As the same key used to encrypt each block of plain text there arises an issue that for a repeating plain text block it would generate the same cipher and will ease the cryptanalysis to crack the algorithm. Hence, ECB is considered for encrypting the small messages which have a rare possibility of repeating text.

2. Cipher Block Chaining Mode

To overcome the limitation of ECB i.e., the repeating block in plain text produces the same ciphertext, a new technique was required which is Cipher Block Chaining (CBC) Mode. CBC confirms that even if the plain text has repeating blocks its encryption won't produce same cipher block.

To achieve totally different cipher blocks for two same plain text blocks **chaining** has been added to the block cipher. For this, the result obtained from the encryption of the first plain text block is fed to the encryption of the next plaintext box.

In this way, each ciphertext block obtained is dependent on its corresponding current plain text block input and all the previous plain text blocks. But during the encryption of first plain text block, no previous plain text block is available so a random block of text is generated called **Initialization vector**.

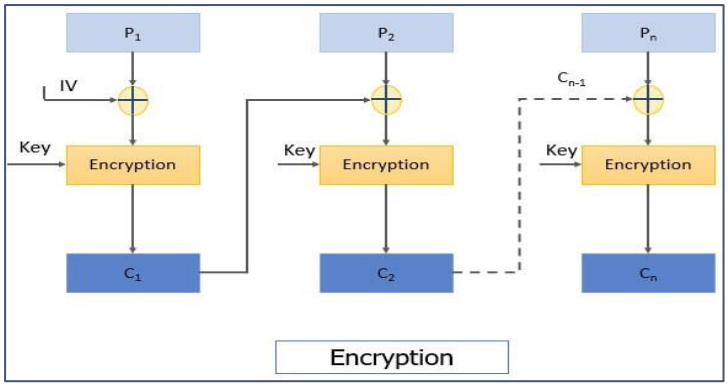
Now let's discuss the encryption steps of CBC

Step 1: The initialization vector and first plain text block are XORed and the result of XOR is then encrypted using the key to obtain the first ciphertext block.

Step 2: The first ciphertext block is fed to the encryption of the second plain text block. For the encryption of second plain text block, first ciphertext block and second plain text block is XORed and the result of XOR is encrypted using the same key in step 1 to obtain the second ciphertext block.

Similarly, the result of encryption of second plain text block i.e., the second ciphertext block is fed to the encryption of third plain text block to obtain third ciphertext block. And the process continues to obtain all the ciphertext blocks.

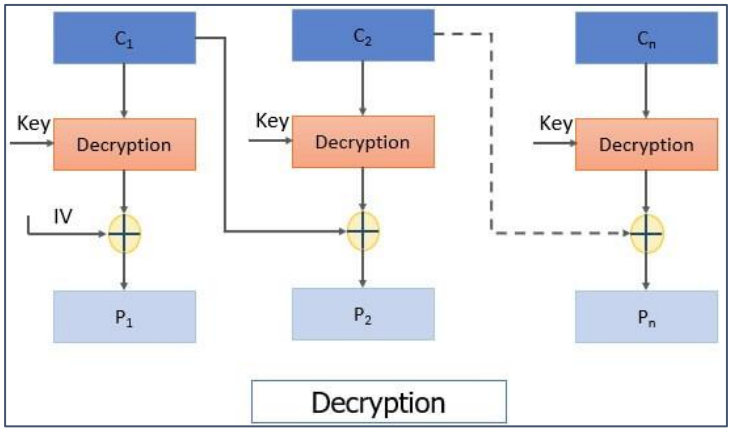
You can see the steps of CBC in the figure below:



Decryption steps of CBC:

Step 1: The first ciphertext block is decrypted using the same key that was used for encrypting all plain text blocks. The result of decryption is then XORed with the initialization vector (IV) to obtain the first plain text block.

Step 2: The second ciphertext block is decrypted and the result of decryption is XORed with the first ciphertext block to obtain the second plain text block. And the process continues till all plain text blocks are retrieved.



There was a limitation in CBC that if we have two identical messages and if we use the same IV for both the identical message it would generate the same ciphertext block.

3. Cipher Feedback Mode

All applications may not be designed to operate on the blocks of data, some may be **character or bit-oriented**. Cipher feedback mode is used to operate on smaller units than blocks.

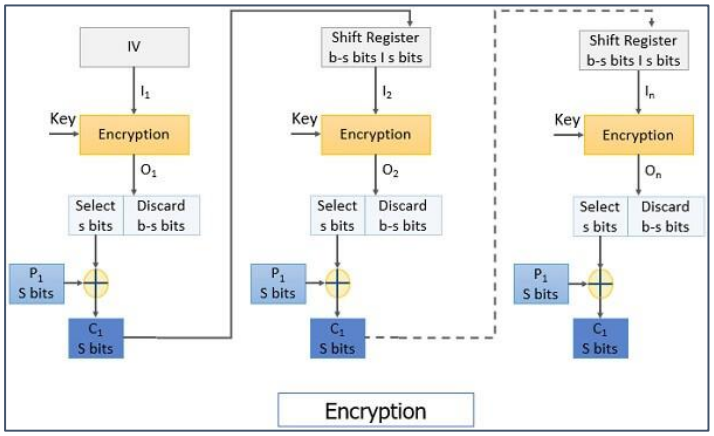
Let us discuss the encryption steps in cipher feedback mode:

Step 1: Here also we use initialization vector, IV is kept in the shift register and it is encrypted using the key.

Step 2: The left most s bits of the encrypted IV is then XORed with the first fragment of the plain text of s bits. It produces the first ciphertext C1 of s bits.

Step 3: Now the shift register containing initialization vector performs left shift by s bits and s bits C1 replaces the rightmost s bits of the initialization vector.

Then again, the encryption is performed on IV and the leftmost s bit of encrypted IV is XORed with the second fragment of plain text to obtain s bit ciphertext C2.



The process continues to obtain all ciphertext fragments.

Decryption Steps:

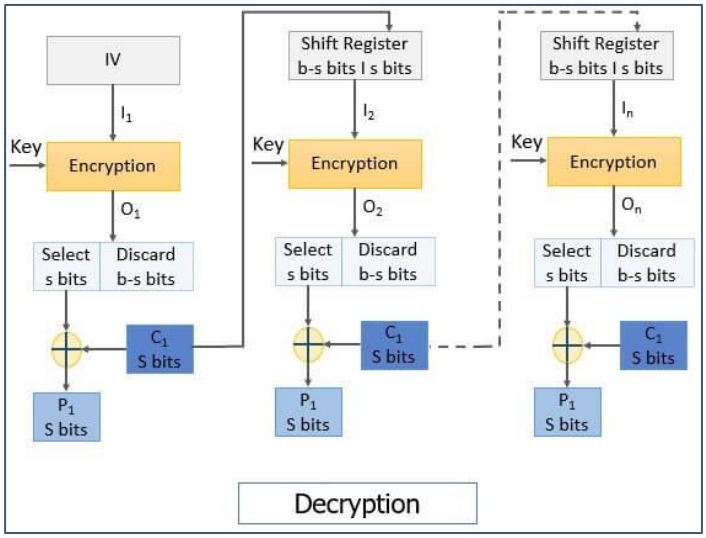
Step 1: The initialization vector is placed in the shift register. It is encrypted using the same key.

Keep a note that even in the **decryption process** the **encryption** algorithm is implemented instead of the decryption algorithm.

Then from the encrypted IV s bits are XORed with the s bits ciphertext C1 to retrieve s bits plain text P1.

Step 2: The IV in the shift register is left-shifted by s bits and the s bits C1 replaces the rightmost s bits of IV.

The process continues until all plain text fragments are retrieved.



It has a limitation that if there occur a bit error in any ciphertext C_i it would affect all the subsequent ciphertext units as C_i is fed to the encryption of next P_{i+1} to obtain C_{i+1}. In this way, bit error would propagate.

4. Output Feedback Mode

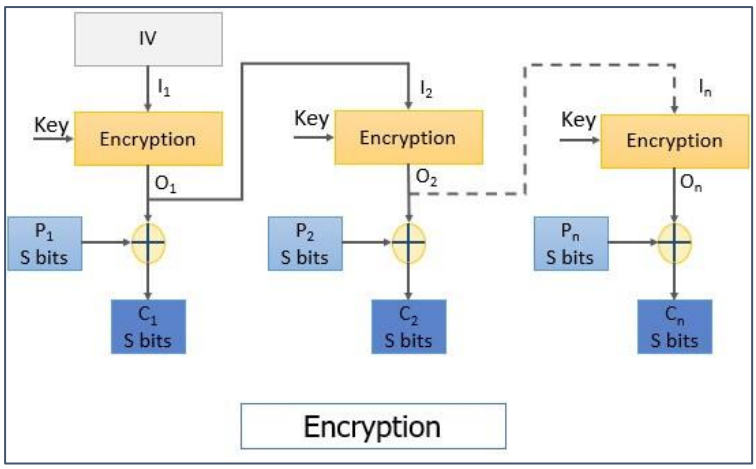
The output feedback (OFB) mode is almost similar to the CFB. The difference between CFB and OFB is that unlike CFB, in OFB the encrypted IV is fed to the encryption of next plain text block. The other difference is that CFB operates on a stream of bits whereas OFB operates on the block of bits.

Steps for encryption:

Step 1: The initialization vector is encrypted using the key.

Step 2: The encrypted IV is then XORed with the plain text block to obtain the ciphertext block.

The encrypted IV is fed to the encryption of next plain text block as you can see in the image below.



Steps for decryption:

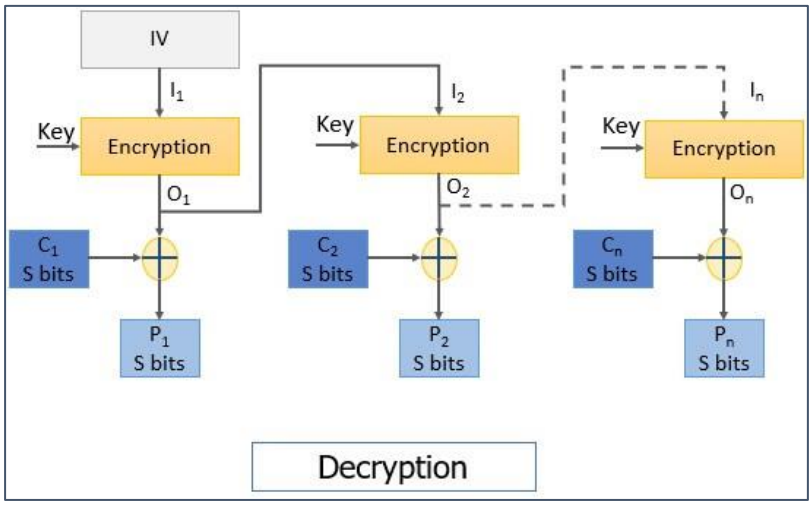
Step 1: The initialization vector is encrypted using the same key used for encrypting all plain text blocks.

Note: In the **decryption process** also the **encryption** function is implemented.

Step2: The encrypted IV is then XORed with the ciphertext block to retrieve the plain text block.

The encrypted IV is also fed to the decryption process of the next ciphertext block.

The process continues until all the plain text blocks are retrieved.



The advantage of OFB is that it protects the propagation of bit error means the if there occurs a bit error in C1 it will only affect the retrieval of P1 and won't affect the retrieval of subsequent plain text blocks.

5. Counter Mode

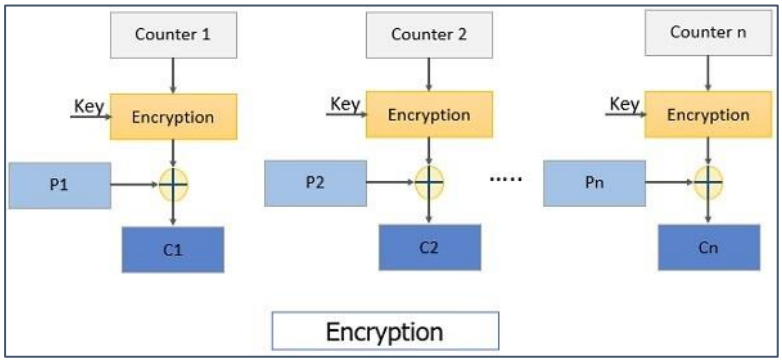
It is similar to OFB but there is no feedback mechanism in counter mode. Nothing is being fed from the previous step to the next step instead it uses a sequence of number which is termed as a **counter** which is input to the encryption function along with the key. After a plain text block is encrypted the counter value increments by 1.

Steps of encryption:

Step1: The counter value is encrypted using a key.

Step 2: The encrypted counter value is XORed with the plain text block to obtain a ciphertext block.

To encrypt the next subsequent plain text block the counter value is incremented by 1 and step 1 and 2 are repeated to obtain the corresponding ciphertext.



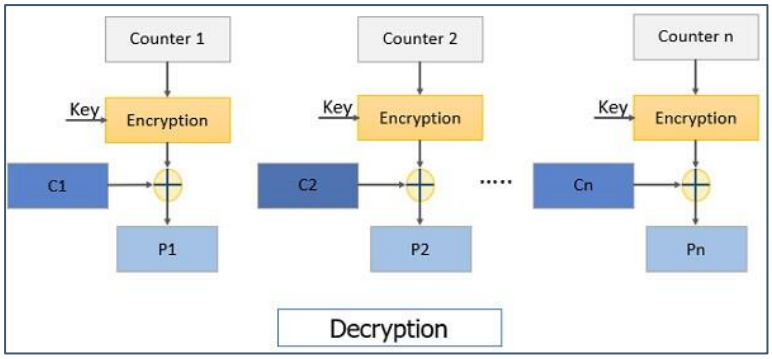
The process continues until all plain text block is encrypted.

Steps for decryption:

Step1: The counter value is encrypted using a key.

Note: Encryption function is used in the decryption process. The same counter values are used for decryption as used while encryption.

Step 2: The encrypted counter value is XORed with the ciphertext block to obtain a plain text block.



To decrypt the next subsequent ciphertext block the counter value is incremented by 1 and step 1 and 2 are repeated to obtain corresponding plain text.

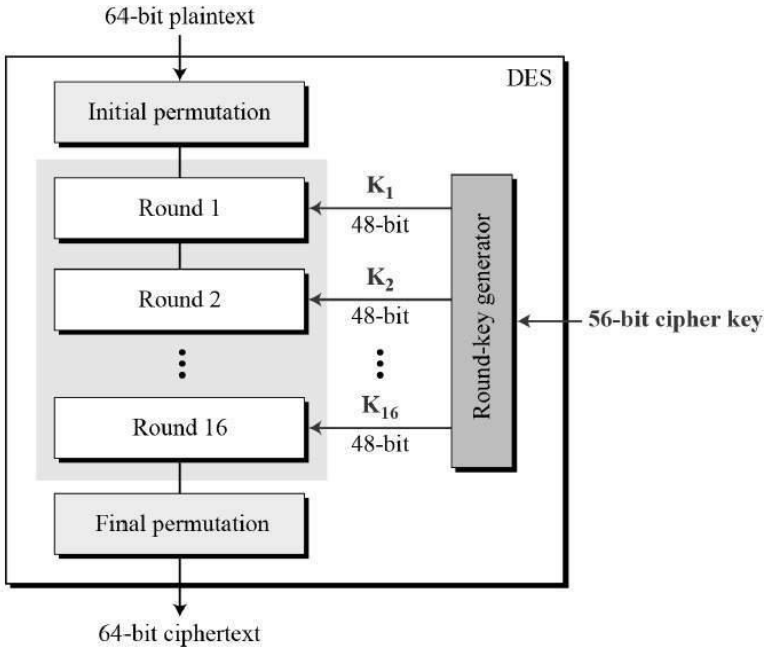
The process continues until all ciphertext block is decrypted.

Symmetric key Ciphers:

Data Encryption Standard

The Data Encryption Standard (DES) is a symmetric-key block cipher published by the National Institute of Standards and Technology (NIST).

DES is an implementation of a Feistel Cipher. It uses 16 round Feistel structure. The block size is 64-bit. Though, key length is 64-bit, DES has an effective key length of 56 bits, since 8 of the 64 bits of the key are not used by the encryption algorithm (function as check bits only). General Structure of DES is depicted in the following illustration −



Since DES is based on the Feistel Cipher, all that is required to specify DES is −

- Round function
- Key schedule
- Any additional processing − Initial and final permutation

How does DES works?

DES uses the same key to encrypt and decrypt a message, so both the sender and the receiver must know and use the same private key. DES was once the go-to, symmetric key algorithm for the encryption of electronic data, but it has been superseded by the more secure Advanced Encryption Standard (AES) algorithm.

Some key features affecting how DES works include the following:

- **Block Cipher:** The Data Encryption Standard is a block cipher, meaning a cryptographic key and algorithm are applied to a block of data simultaneously rather than one bit at a time. To encrypt a plaintext message, DES groups it into 64-bit blocks. Each block is enciphered using the secret key into a 64-bit ciphertext by means of permutation and substitution.

- **Several Rounds of Encryption:** The DES process involves encrypting 16 times. It can run in four different modes, encrypting blocks individually or making each cipher block dependent on all the previous blocks. Decryption is simply the inverse of encryption, following the same steps but reversing the order in which the keys are applied.

- **64-bit key:** DES uses a 64-bit key, but because eight of those bits are used for parity checks, the effective key length is only 56 bits. The encryption algorithm generates 16 different 48-bit subkeys, one for each of the 16 encryption rounds. Subkeys are generated by selecting and permuting parts of the key as defined by the DES algorithm.

- **Replacement and Permutation:** The algorithm defines sequences of replacement and permutation that the ciphertext undergoes during the encryption process.

- **Backward Compatibility:** DES also provides this capability in some instances.

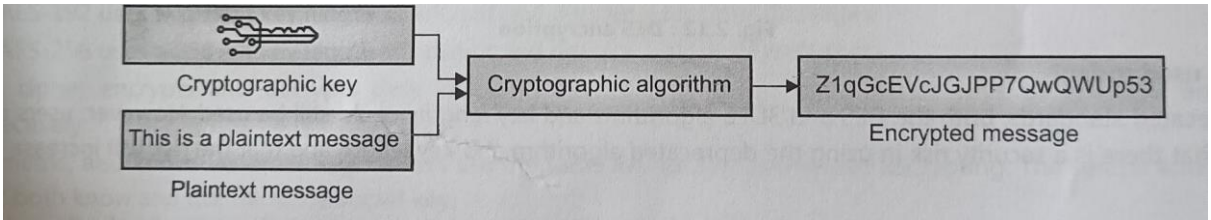


Fig. 2.11 : Encryption algorithm

Why DES is unsafe?

- For any cipher, the most basic method of attack is brute force, which involves trying each key until you find the right one. The length of the key determines the number of possible keys and hence the feasibility of this type of attack.
- The effective DES key length of 56 bits would require a maximum of 256, or about 72 quadrillion, attempts to find the correct key. This is not enough to protect data with DES against brute-force attempts with modern computers.
- Few messages encrypted using DES before it was replaced by AES were likely subjected to this kind of code-breaking effort. Nevertheless, many security experts felt the 56-bit key length was inadequate even before DES was adopted as a standard. There have always been suspicions that interference from the National Security Agency weakened the original algorithm.
- DES remained a trusted and widely used encryption algorithm through the mid-1990s. However, in 1998, a computer built by the Electronic Frontier Foundation (EFF) decrypted a DES-encoded message in 56 hours. By harnessing the power of thousands of networked computers, the following year, EFF cut the decryption time to 22 hours.
- Currently, a DES cracking service operated at the crack.sh website promises to crack DES keys, for a fee, in about 26 hours as of this writing. Crack.sh also offers free access to a rainbow table for known plaintexts of 1122334455667788 that can return a DES key in 25 seconds or less
- Today, reliance on DES for data confidentiality is a serious security design error in any computer system and should be avoided. There are much more secure algorithms available, such as AES. Much like a cheap suitcase lock, DES will keep the contents safe from honest people, but it will not stop a determined thief.

Advanced Encryption Standard

What is AES?

The Advanced Encryption Standard (AES) is a symmetric block cipher chosen by the U.S. government to protect classified information.

AES is implemented in software and hardware throughout the world to encrypt sensitive data. It is essential for government computer security, cybersecurity, and electronic data protection.

The National Institute of Standards and Technology (NIST) started development of AES in 1997 when it announced the need for an alternative to the Data Encryption Standard (DES), which was starting to become vulnerable to brute-force attacks.

NIST stated that the newer, advanced encryption algorithm would be unclassified and must be "capable of protecting sensitive government information well into the [21st] century." It was intended to be easy to implement in hardware and software, as well as in restricted environments -- such as a smart card -- and offer decent defences against various attack techniques.

AES was created for the U.S. government with additional voluntary, free use in public or private, commercial, or non-commercial programs that provide encryption services. However, nongovernmental organizations choosing to use AES are subject to limitations created by U.S. export control.

How AES encryption works ?

AES includes three block ciphers:

- AES-128 uses a 128-bit key length to encrypt and decrypt a block of messages.
- AES-192 uses a 192-bit key length to encrypt and decrypt a block of messages.

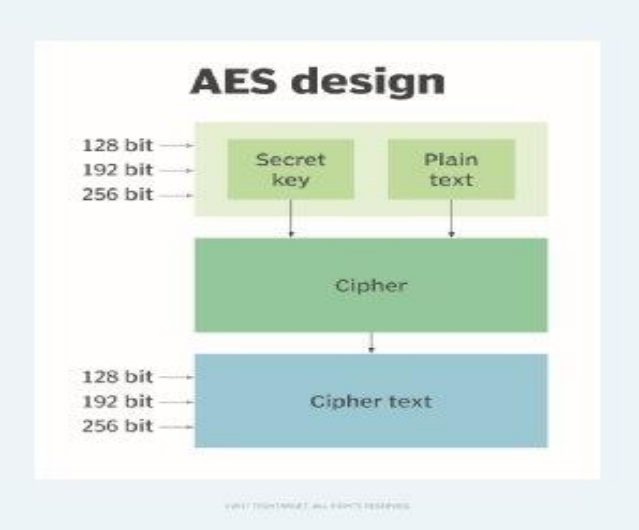
- AES-256 uses a 256-bit key length to encrypt and decrypt a block of messages.

Each cipher encrypts and decrypts data in blocks of 128 bits using cryptographic keys of 128, 192 and 256 bits, respectively.

Symmetric, also known as secret key, ciphers use the same key for encrypting and decrypting. The sender and the receiver must both know -- and use -- the same secret key.

The government classifies information in three categories: Confidential, Secret or Top Secret. All key lengths can be used to protect the Confidential and Secret level. Top Secret information requires either 192- or 256-bit key lengths.

There are 10 rounds for 128-bit keys, 12 rounds for 192-bit keys and 14 rounds for 256-bit keys. A round consists of several processing steps that include substitution, transposition and mixing of the input plaintext to transform it into the final output of ciphertext.



The AES encryption algorithm defines numerous transformations that are to be performed on data stored in an array. The first step of the cipher is to put the data into an array, after which the cipher transformations are repeated over multiple encryption rounds.

The first transformation in the AES encryption cipher is substitution of data using a substitution table. The second transformation shifts data rows. The third mixes columns. The last transformation is performed on each column using a different part of the encryption key. Longer keys need more rounds to complete.

What are the features of AES?

NIST specified the new AES algorithm must be a block cipher capable of handling 128-bit blocks, using keys sized at 128, 192 and 256 bits.

Other criteria for being chosen as the next AES algorithm included the following:

- **Security.** Competing algorithms were to be judged on their ability to resist attack as compared to other submitted ciphers. Security strength was to be considered the most important factor in the competition.
- **Cost.** Intended to be released on a global, nonexclusive and royalty-free basis, the candidate algorithms were to be evaluated on computational and memory efficiency.
- **Implementation.** Factors to be considered included the algorithm's flexibility, suitability for hardware or software implementation, and overall simplicity.

Blowfish Algorithm

What is Blowfish?

Blowfish is a variable-length, symmetric, 64-bit block cipher. Designed by Bruce Schneier in 1993 as a "general-purpose algorithm," it was intended to provide a fast, free, drop-in alternative to the aging Data Encryption Standard (DES) and International Data Encryption Algorithm (IDEA) encryption algorithms.

Blowfish is significantly faster than DES and IDEA and is unpatented and available free for all uses. However, it couldn't completely replace DES due to its small block size, which is considered insecure.

Twofish, its successor, addressed the security problem with a larger block size of 128 bits. Nonetheless, full Blowfish encryption has never been broken, and the algorithm is included in many cipher suites and encryption products available today.

Understanding Blowfish

Blowfish features a 64-bit block size and takes a variable-length key, from 32 bits to 448 bits. It consists of 16 Feistel-like iterations, where each iteration operates on a 64-bit block that's split into two 32-bit words. Blowfish uses a single encryption key to both encrypt and decrypt data.

The Blowfish algorithm consists of two major parts:

1. **Data encryption:** Data encryption happens through a 16-round Feistel network, with each round consisting of a key-dependent permutation and a key- and data-dependent substitution. Large, key-dependent S-boxes work with the substitution method and form an integral part of the data encryption system in Blowfish. All encryption operations are XORs -- a type of logic gate -- and additions on 32-bit words.

2. **Key expansion and subkeys:** In the key expansion process, maximum size 448-bit keys are converted into several subkey arrays totalling 4,168

bytes. Subkeys form an integral part of the Blowfish algorithm, which uses a large number of them. These subkeys are pre-computed before encryption or decryption can take place.

In Blowfish, the P-array consists of 18 32-bit subkeys and four 32-bit S-boxes with 256 entries each. The subkeys are calculated as follows:

1. The P-array and S-boxes are initialized with a fixed string of hexadecimal digits of pi.
2. The first element in the P-array (P1) is now XORed with the first 32 bits of the key, P2 is XORed with the second 32-bits and so on, until all the elements in the P-array are XORed with the key bits.
3. All-zero strings are encrypted by the algorithm as described in the above steps.
4. P1 and P2 arrays are replaced with the output from step 3 above.
5. This output is encrypted by Blowfish with modified subkeys.
6. The output of step 5 modifies P3 and P4 in the P-array.
7. This process continues until all the P-arrays and four S-boxes are modified.

In total, Blowfish runs 521 times to generate all the subkeys and processes – about 4 kilobytes (KB) of data.

Blowfish encryption/decryption process example:

Assume the message "Hi world" is to be encrypted with Blowfish. The following are the steps involved:

1. Initially, the input "Hi world" consists of seven characters plus one space, which is equal to 64 bits or 8 bytes.
2. The input is split into 32 bits. The left 32 bits -- "Hi w" -- are XORed with P1, which is generated by key expansion to create a value called

P1. (Note: *P denotes prime number, a number that is not divisible except by 1 and itself.*)

3. Then, P1 runs through a transformative F-function (F In) in which the 32 bits are split into 4 bytes each and passed to the four S-boxes.
4. The first two values from the first two S-boxes are added to each other and XORed with the third value from the third S-box.
5. This result is added to the output of the fourth S-box to produce 32 bits as output.
6. The output of F In is XORed with the right 32 bits of the input message -- "orld" -- to produce output F1'.
7. Then, F1' replaces the left half of the message, while P1' replaces the right half.
8. This same process is repeated for successive members of P-array for 16 rounds in total.
9. Finally, after 16 rounds, the outputs P16' and F16' are XORed with the last two entries of the P-array, i.e., P17 and P18. They are then recombined to produce the 64-bit ciphertext of the input message.

Advantages of Blowfish

One of the fastest and most compact block ciphers in public use, Blowfish uses a symmetric encryption key to turn data into ciphertext. Almost three decades after it was first developed, Blowfish is still widely used because it offers the following advantages:

- Much faster and more efficient than DES and IDEA algorithms;
- Unpatented and can be freely used by anyone even without a license;
- Despite the complex initialization phase before encryption, the data encryption process is efficient on large microprocessors;
- Provides extensive security for software and applications developed in Java;
- Provides secure access for backup tools; and

- Supports secure user authentication for remote access.

Disadvantages of Blowfish

There are some downsides to using Blowfish for encryption, including the following:

- Speed is affected when changing keys.
- The key schedule takes a long time.
- The small 64-bit block size makes the algorithm vulnerable to birthday attacks, a class of brute-force attacks.
- Each new key requires preprocessing equivalent to 4 KB of text, which affects its speed, making it unusable for some applications.

Applications of Blowfish

Blowfish is suitable for a wide range of applications, including the following:

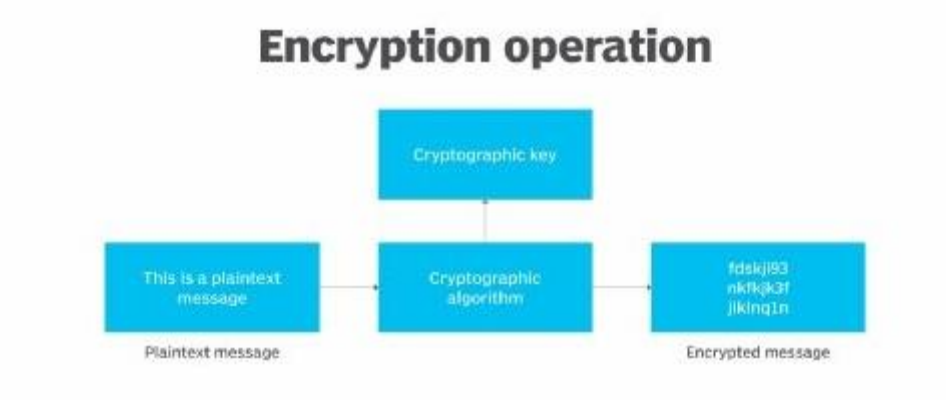
- Bulk encryption
- Random bit generation
- Packet encryption
- Password hashing and management
- Mobile processors
- Email, file or disk encryption
- Data backup
- Secure Shell

International Data Encryption Algorithm (IDEA)

What is the International Data Encryption Algorithm (IDEA)?

The International Data Encryption Algorithm (IDEA) is a symmetric key block cipher encryption algorithm designed to encrypt text to an unreadable format for transmission via the internet. It uses a typical block size of 128 bits and takes 64 bits as an input, i.e., 64-bit data. IDEA performs 8.5 identical encryption and decryption rounds using six different subkeys. It uses four keys for output transformation.

First published in 1991 to replace the Data Encryption Standard (DES), IDEA was originally called Proposed Encryption Standard. The name was changed to Improved Proposed Encryption Standard and eventually to IDEA.



Understanding IDEA

IDEA was developed at ETH, a research university in Zurich, Switzerland, and is generally considered to be secure. The IDEA cipher encrypts text with the assumption that security in IDEA is not predicated on keeping the algorithm a secret, but rather on ignorance of the secret key.

IDEA uses a 128-bit key and operates on 64-bit blocks. Essentially, it encrypts a 64-bit block of plaintext into a 64-bit block of ciphertext. This input plaintext block is divided into four subblocks of 16 bits each. It consists of a series of eight identical transformations, where each transformation is known as a round, as well as an output transformation, which is known as a half-round. Similar to the 16-bit plaintext block, the ciphertext block is also the exact same size.

A block cipher operates in round blocks, with part of the encryption key, known as round key, applied to each round, followed by other mathematical operations. After a certain number of rounds, the ciphertext for that block is generated.

Encryption in IDEA

IDEA derives most of its security from multiple interleaved mathematical operations:

- Modular addition
- Modular multiplication
- Bitwise exclusive-or (xor)

By using a 128-bit key, IDEA encrypts a 64-bit block of plaintext into a 64-bit block of ciphertext. One process partitions the plaintext block into four 16-bit subblocks for each of the eight complete rounds, namely X1, X2, X3 and X4.

Another process produces six 16-bit key subblocks for each of the encryption rounds, namely Z1, Z2, Z3, Z4, Z5 and Z6. For subsequent output transformation, a further four 16-bit key subblocks are required. Thus, from a 128-bit key, a total of 52 16-bit subblocks are generated.

In each complete round, three algebraic operations are performed: bitwise XOR, addition modulo 2^{16} and multiplication modulo $2^{16}+1$.

The 14 steps for a complete round are the following:

1. Multiply X1 with Z1.
2. Add X2 to Z2.
3. Add X3 to Z3.
4. Multiply X4 with Z4.
5. Bitwise XOR the results of steps 1 and 3.
6. Bitwise XOR the results of steps 2 and 4.

7. Multiply the result of step 5 with Z5.
8. Add the results of steps 6 and 7.
9. Multiply the result of step 8 with Z6.
10. Add the results of steps 7 and 9.
11. Bitwise XOR the results of steps 1 and 9.
12. Bitwise XOR the results of steps 3 and 9.
13. Bitwise XOR the results of steps 2 and 10.
14. Bitwise XOR the results of steps 4 and 10.

Six subkeys are used in each of the eight rounds, and the final 4 subkeys are used in the ninth half-round final transformation.

Swapping occurs for every round until the final complete round (round 8). After eight complete rounds, the final half-round transformation occurs. The steps involved are the following:

1. Multiply X1 with the first subkey.
2. Add X2 with the second subkey.
3. Add X3 with the third subkey.
4. Multiply X4 with the fourth subkey.

Decryption in IDEA

The decryption process uses the same steps as the encryption process. However, different 16-bit key subblocks are generated. Each of the 52 16-bit key subblocks used for decryption is the inverse of the key subblock used during encryption with respect to applied algebraic operations.

Also, these subblocks are used in reverse order during decryption. Decryption in IDEA works on the shoes and socks principle, i.e., the last encryption is the first to be removed.

Advantages and applications of IDEA

Although IDEA was originally meant to replace DES, it did not do so. Nonetheless, it was incorporated into Pretty Good Privacy, which indicates its reliability and security.

IDEA supports easy hardware and software implementation for quick execution. It can be easily embedded into encryption software to protect data transmission and storage in several real-world applications:

- Financial services
- Broadcasting
- Government
- Video conferencing
- Audio and video for cable TV
- Business TV
- Voice Over IP
- Email via public networks
- Smart cards

Stream Cipher

What is a stream cipher?

A stream cipher is a method of encrypting text (to produce ciphertext) in which a cryptographic key and algorithm are applied to each binary digit in a data stream, one bit at a time. The main alternative method to stream cipher is, in fact, the block cipher, where a key and algorithm are applied to blocks of data rather than individual bits in a stream.

How does a stream cipher work?

A stream cipher is an encryption algorithm that uses a symmetric key to encrypt and decrypt a given amount of data. A symmetric cipher key, as opposed to an asymmetric cipher key, is an encryption tool that is used in both encryption and decryption. Asymmetric keys will sometimes use one key to encrypt a message and another to decrypt the respective ciphertext.

What makes stream ciphers particularly unique is that they encrypt data one bit, or byte, at a time. This makes for a fast and relatively simple encryption process.

Basic encryption requires three main components:

1. A message, document or piece of data
2. A key
3. An encryption algorithm

The key typically used with a stream cipher is known as a one-time pad. Mathematically, a one-time pad is unbreakable because it's always at least the exact same size as the message it is encrypting.

Here is an example to illustrate the one-timed pad process of stream ciphering: Person A attempts to encrypt a 10-bit message using a stream cipher. The one-time pad, in this case, would also be at least 10 bits long.

This can become cumbersome depending on the size of the message or document they are attempting to encrypt, however.

Cryptographers also refer to the symmetric key used in a stream cipher as a keystream. This is because Person A could opt to create a pseudo-random cipher digit stream, or keystream, using a key that is smaller than the size of the plaintext file. Furthermore, to avoid having to create a larger keystream, users can use a cryptographic number generator to create a larger keystream from a smaller, pseudo-random key.

Here, Person A decides to use a 4-bit key to encrypt a 10-bit message. To do that, they must first use an initialization vector (IV) to generate a random seed value. Placing this seed value into a cryptographic number generator, Person A can create a pseudo-random keystream that matches the size of their desired plaintext file.

The quality of the number generator contributes to the randomness and security of the ciphertext, however. Lower-end cryptographic number generators can sometimes have patterns that malicious users, or hackers, can identify and use to decrypt the ciphertext.

After the user has created the keystream, the stream cipher combines the keystream with the corresponding digits of the plaintext using the exclusive-or (XOR) operator. The XOR operator creates new binary values, which make up the ciphertext. It generates these values by comparing bits in the plaintext and the keystream that share the same position.

For example, the first bit in Person A's 10-bit message will be XOR-ed with the first bit of the keystream. If the two digits are the same, the XOR operator will produce a zero. If the two are different -- i.e., a combination of 1 and 0 -- the XOR operator will produce a 1. This is part of what makes stream cipher encryption so fast.

Once each bit of data has been XOR-ed by the stream cipher, it will produce an unreadable ciphertext message.

Decryption of the ciphertext can happen in a manner similar to how the plaintext encryption occurs. This time, instead of the data and keystream being XOR-ed, the ciphertext and the keystream are XOR-ed.

Stream ciphers users should not use the same IV more than once, however, to maximize the security of this process.

The Advantages and Disadvantages of using a Stream Cipher

Speed of encryption tops the list of advantages for stream ciphers. Once a stream cipher makes a key, the encryption and decryption process is almost instantaneous. This is largely due to the simplicity of operation, a basic XOR function using two distinct data bits. Also, for this reason, the hardware complexity of a stream cipher is quite low -- meaning a wider range of technologies can facilitate this mode of operation.

Another advantage of using stream ciphers is the ability to decrypt selected sections of ciphertext. Since each bit of data in the ciphertext corresponds with plaintext data in the same position, users can decrypt ciphertext for part of a document rather than the entire file. If Person A, for example, wanted to retrieve the original plaintext data for just the first five bits, they would only need to decrypt the first five bits of the ciphertext.

The positional alignment among the plaintext, keystream and ciphertext is a significant security vulnerability of stream ciphers. If the encryption process does not use a hashing algorithm or IV during the encryption process, a hacker who obtains a segment of plaintext and its respective ciphertext will be able to deduce the keystream used by the process.

This is why cryptographers refer to stream ciphers as having low diffusion, meaning the plaintext and ciphertext are not vastly different from each other.

Confusion vs. Diffusion

Evaluate encryption methods using two main criteria: confusion and diffusion.

Confusion contributes to the ambiguity of the ciphertext -- i.e., the methods used to complicate the relationship between the plaintext and the ciphertext.

Diffusion, on the other hand, refers to how well the encryption hides these complex, or simple, relationships. For example, a standard for diffusion is that at least 50% of the ciphertext should change when a cipher alters a single bit of plaintext. Encryption operations with low diffusion are less secure than those with high diffusion.

Stream Cipher vs. Block Cipher

Key	Block Cipher	Stream Cipher
Definition	Block Cipher is the type of encryption where the conversion of plaintext is performed by taking its block at a time.	Stream Cipher is the type of encryption where the conversion of plaintext is performed by taking one byte of the plaintext at a time.
Conversion of Bits	Since Block Cipher converts blocks at a time, it converts a more significant number of bits than Stream Cipher, which can convert 64 bits or more.	In the case of Stream Cipher, however, only 8 bits can be transformed at a time.
Principle	Block Cipher uses both "confusion" and "diffusion" principle for the conversion required for encryption.	Stream Cipher uses only confusion principle for the conversion.
Algorithm	For encryption of plain text Block Cipher uses Electronic Code Book (ECB) and Cipher Block Chaining (CBC) algorithm.	Stream Cipher uses CFB (Cipher Feedback) and OFB (Output Feedback) algorithm.

RC4 Encryption Algorithm

RC4 is a stream cipher and variable-length key algorithm. This algorithm encrypts one byte at a time (or larger units at a time). A key input is a pseudorandom bit generator that produces a stream 8-bit number that is unpredictable without knowledge of input key, The output of the generator is called key-stream, is combined one byte at a time with the plaintext stream cipher using X-OR operation.

Example:

RC4 Encryption

10011000 ? 01010000 = 11001000

RC4 Decryption

11001000 ? 01010000 = 10011000

Key-Generation Algorithm – A variable-length key from 1 to 256 bytes is used to initialize a 256-byte state vector S, with elements S[0] to S[255]. For encryption and decryption, a byte k is generated from S by selecting one of the 255 entries in a systematic fashion, then the entries in S are permuted again.

Key-Scheduling Algorithm: Initialization: The entries of S are set equal to the values from 0 to 255 in ascending order, a temporary vector T, is created. If the length of the key k is 256 bytes, then k is assigned to T. Otherwise, for a key with length(k-len) bytes, the first k-len elements of T as copied from K, and then K is repeated as many times as necessary to fill T. The idea is illustrated as follow:

```
for
   i = 0 to 255 do S[i] = i;
T[i] = K[i mod k - len];
```

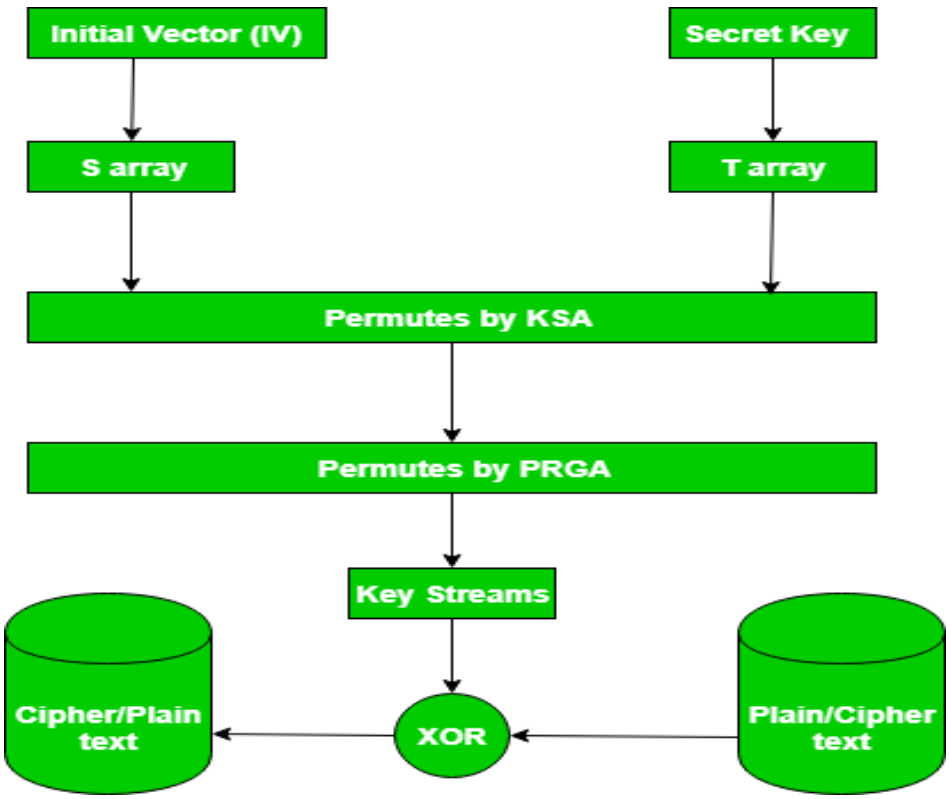
We use T to produce the initial permutation of S. Starting with S[0] to S[255], and for each S[i] algorithm swap it with another byte in S according to a scheme dictated by T[i], but S will still contain values from 0 to 255 :

```
int j = 0;
for (int i = 0; i <= 255; i++) {
    j = (j + S[i] + T[i]) % 256;
    swap(S[i], S[j]); // Swap S[i] and S[j]
}
```

Pseudo random generation algorithm (Stream Generation): Once the vector S is initialized, the input key will not be used. In this step, for each S[i] algorithm swap it with another byte in S according to a scheme dictated by the current configuration of S. After reaching S[255] the process continues, starting from S[0] again

```
i, j = 0;
while (true)
    i = (i + 1)mod 256;
j = (j + S[i])mod 256;
Swap(S[i], S[j]);
t = (S[i] + S[j])mod 256;
k = S[t];
```

Encrypt using X-Or():



Features of the RC4 encryption algorithm:

1. **Symmetric key algorithm:**
   RC4 is a symmetric key encryption algorithm, which means that the same key is used for encryption and decryption.

2. **Stream cipher algorithm:**
   RC4 is a stream cipher algorithm, which means that it encrypts and decrypts data one byte at a time. It generates a key stream of pseudorandom bits that are XORed with the plaintext to produce the ciphertext.

3. **Variable key size:**
   RC4 supports variable key sizes, from 40 bits to 2048 bits, making it flexible for different security requirements.

4. **Fast and efficient:**
   RC4 is a fast and efficient encryption algorithm that is suitable for low-power devices and applications that require high-speed data transmission.

5. **Widely used:**
   RC4 has been widely used in various applications, including wireless networks, secure sockets layer (SSL), virtual private networks (VPN), and file encryption.

6. **Vulnerabilities:**
   RC4 has several vulnerabilities, including a bias in the first few bytes of the keystream, which can be exploited to recover the key. As a result, RC4 is no longer recommended for use in new applications.

Advantages:

1. **Fast and efficient:**
   RC4 is a very fast and efficient encryption algorithm, which makes it suitable for use in applications where speed and efficiency are critical.

2. **Simple to implement:**
   RC4 is a relatively simple algorithm to implement, which means that it can be easily implemented in software or hardware.

3. **Variable key size:**
   RC4 supports variable key sizes, which makes it flexible and adaptable for different security requirements.

4. **Widely used:**
   RC4 has been widely used in various applications, including wireless networks, secure sockets layer (SSL), virtual private networks (VPN), and file encryption.

Disadvantages:

1. **Vulnerabilities:**
   RC4 has several known vulnerabilities that make it unsuitable for new applications. For example, there is a bias in the first few bytes of the keystream, which can be exploited to recover the key.

2. **Security weaknesses:**
   RC4 has some inherent weaknesses in its design, which make it less secure than other encryption algorithms, such as AES or ChaCha20.

3. **Limited key length:**
   The maximum key length for RC4 is 2048 bits, which may not be sufficient for some applications that require stronger encryption.

4. **Not recommended for new applications:**
   Due to its vulnerabilities and weaknesses, RC4 is no longer recommended for use in new applications. Other more secure stream cipher algorithms, such as AES-CTR or ChaCha20, should be used instead.

Asymmetric key Ciphers:

What are the principles of Public key Cryptosystem

Public key cryptography has become an essential means of providing confidentiality, especially through its need of key distribution, where users seeking private connection exchange encryption keys. It also features digital signatures which enable users to sign keys to check their identities.

The approach of public key cryptography derivative from an attempt to attack two of the most complex problems related to symmetric encryption. The first issue is that key distribution. Key distribution under symmetric encryption needed such as :

- That two communicants already shared a key, which somehow has been shared to them.
- The need of a key distribution centre.

Public key Cryptosystem:

Asymmetric algorithms depends on one key for encryption and a distinct but related key for decryption. These algorithms have the following characteristics which are as follows:

- It is computationally infeasible to decide the decryption key given only information of the cryptographic algorithm and the encryption key.
- There are two related keys such as one can be used for encryption, with the other used for decryption.

A public key encryption scheme has the following ingredients which are as follows:

- **Plaintext**: This is the readable message or information that is informer into the algorithm as input.
- **Encryption algorithm**: The encryption algorithm performs several conversion on the plaintext.

- **Public and Private keys** − This is a set of keys that have been selected so that if one can be used for encryption, and the other can be used for decryption.
- **Ciphertext** − This is scrambled message generated as output. It based on the plaintext and the key. For a given message, there are two specific keys will create two different ciphertexts.

- **Decryption Algorithm** − This algorithm get the ciphertext and the matching key and create the original plaintext.

The keys generated in public key cryptography are too large including 512, 1024, 2048 and so on bits. These keys are not simply to learn. Thus, they are maintained in the devices including USB tokens or hardware security modules.

The major issue in public key cryptosystems is that an attacker can masquerade as a legal user. It can substitute the public key with a fake key in the public directory. Moreover, it can intercept the connection or alters those keys.

Public key cryptography plays an essential role in online payment services and ecommerce etc. These online services are ensured only when the authenticity of public key and signature of the user are ensured.

The asymmetric cryptosystem should manage the security services including confidentiality, authentication, integrity, and non-repudiation. The public key should support the security services including non-repudiation and authentication. The security services of confidentiality and integrity considered as an element of encryption process completed by private key of the user.

RSA Algorithm:

The RSA algorithm (Rivest-Shamir-Adleman) is the basis of a cryptosystem -- a suite of cryptographic algorithms that are used for specific security services or purposes -- which enables public key encryption and is widely used to secure sensitive data, particularly when it is being sent over an insecure network such as the internet.

RSA was first publicly described in 1977 by Ron Rivest, Adi Shamir and Leonard Adleman of the Massachusetts Institute of Technology, though the 1973 creation of a public key algorithm by British mathematician Clifford Cocks was kept classified by the U.K.'s GCHQ until 1997.

Public key cryptography, also known as asymmetric cryptography, uses two different but mathematically linked keys -- one public and one private.
The public key can be shared with everyone, whereas the private key must be kept secret.

In RSA cryptography, both the public and the private keys can encrypt a message. The opposite key from the one used to encrypt a message is used to decrypt it. This attribute is one reason why RSA has become the most widely used asymmetric algorithm: It provides a method to assure the confidentiality, integrity, authenticity, and non-repudiation of electronic communications and data storage.

Many protocols, including Secure Shell (SSH), OpenPGP, S/MIME, and SSL/TLS, rely on RSA for encryption and digital signature functions. It is also used in software programs -- browsers are an obvious example, as they need to establish a secure connection over an insecure network, like the internet, or validate a digital signature. RSA signature verification is one of the most commonly performed operations in network-connected systems.

Why is the RSA algorithm used?

RSA derives its security from the difficulty of factoring large integers that are the product of two large prime numbers. Multiplying these two numbers is easy,

but determining the original prime numbers from the total -- or factoring -- is considered infeasible due to the time it would take using even today's supercomputers.

The public and private key generation algorithm is the most complex part of RSA cryptography. Two large prime numbers, p and q, are generated using the Rabin-Miller primality test algorithm. A modulus, n, is calculated by multiplying p and q. This number is used by both the public and private keys and provides the link between them. Its length, usually expressed in bits, is called the key length.

The public key consists of the modulus n and a public exponent, e, which is normally set at 65537, as it's a prime number that is not too large. The e figure doesn't have to be a secretly selected prime number, as the public key is shared with everyone.

The private key consists of the modulus n and the private exponent d, which is calculated using the Extended Euclidean algorithm to find the multiplicative inverse with respect to the totient of n.

How does the RSA algorithm work?

Alice generates her RSA keys by selecting two primes: p=11 and q=13. The modulus is $n=p\times q=143$. The totient is $n\ \phi(n)=(p-1)x(q-1)=120$. She chooses 7 for her RSA public key e and calculates her RSA private key using the Extended Euclidean algorithm, which gives her 103.

Bob wants to send Alice an encrypted message, M, so he obtains her RSA public key (n, e) which, in this example, is (143, 7). His plaintext message is just the number 9 and is encrypted into ciphertext, C, as follows:

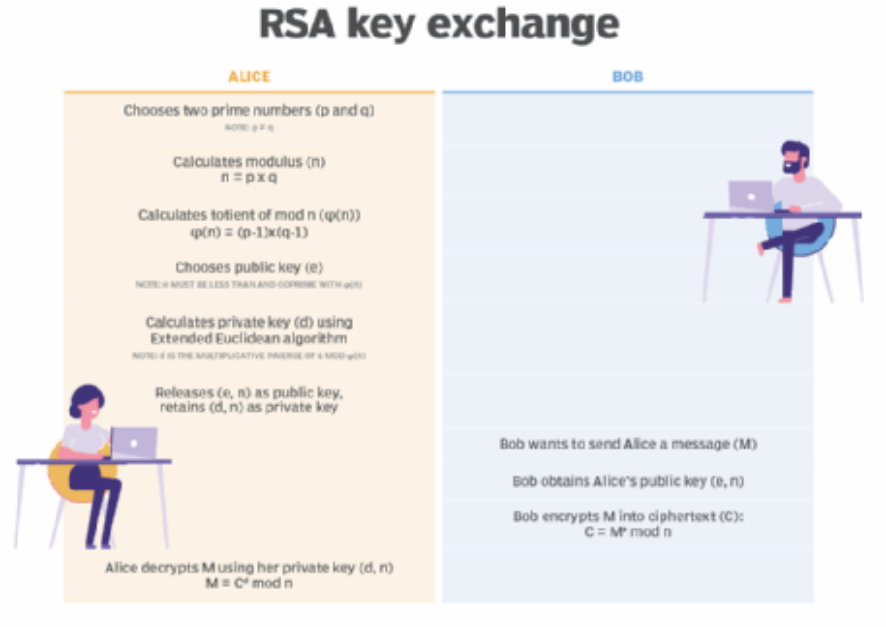
$M^e \bmod n = 9^7 \bmod 143 = 48 = C$

When Alice receives Bob's message, she decrypts it by using her RSA private key (d, n) as follows:

$C^d \bmod n = 48^{103} \bmod 143 = 9 = M$

To use RSA keys to digitally sign a message, Alice would need to create a hash -- a message digest of her message to Bob -- encrypt the hash value with her RSA private key, and add the key to the message. Bob can then verify that the message has been sent by Alice and has not been altered by decrypting the hash value with her public key. If this value matches the hash of the original message, then only Alice could have sent it -- authentication and non-repudiation -- and the message is exactly as she wrote it -- integrity.

Alice could, of course, encrypt her message with Bob's RSA public key -- confidentiality -- before sending it to Bob. A digital certificate contains information that identifies the certificate's owner and also contains the owner's public key. Certificates are signed by the certificate authority that issues them, and they can simplify the process of obtaining public keys and verifying the owner.



How is RSA secure?

RSA security relies on the computational difficulty of factoring large integers. As computing power increases and more efficient factoring algorithms are discovered, the ability to factor larger and larger numbers also increases.

Encryption strength is directly tied to key size. Doubling key length can deliver an exponential increase in strength, although it does impair performance. RSA keys are typically 1024- or 2048-bits long, but experts believe that 1024-bit keys are no longer fully secure against all attacks. This is why the government and some industries are moving to a minimum key length of 2048-bits.

Barring an unforeseen breakthrough in quantum computing, it will be many years before longer keys are required, but elliptic curve cryptography (ECC) is gaining favor with many security experts as an alternative to RSA to implement public key cryptography. It can create faster, smaller and more efficient cryptographic keys.

Modern hardware and software are ECC-ready, and its popularity is likely to grow. It can deliver equivalent security with lower computing power and battery resource usage, making it more suitable for mobile apps than RSA.

A team of researchers, which included Adi Shamir, a co-inventor of RSA, successfully created a 4096-bit RSA key using acoustic cryptanalysis. However, note that any encryption algorithm is vulnerable to attack.

Diffie-Hellman Key Exchange

Diffie-Hellman key exchange is a method of digital encryption that securely exchanges cryptographic keys between two parties over a public channel without their conversation being transmitted over the internet. The two parties use symmetric cryptography to encrypt and decrypt their messages. Published in 1976 by Whitfield Diffie and Martin Hellman, it was one of the first practical examples of public key cryptography.

Diffie-Hellman key exchange raises numbers to a selected power to produce decryption keys. The components of the keys are never directly transmitted, making the task of a would-be code breaker mathematically overwhelming. The method doesn't share information during the key exchange. The two parties have no prior knowledge of each other, but the two parties create a key together.

Where is Diffie-Hellman key exchange used?

Diffie-Hellman key exchange's goal is to securely establish a channel to create and share a key for symmetric key algorithms. Generally, it's used for encryption, password-authenticated key agreement and forward security. Password-authenticated key agreements are used to prevent man-in-the-middle (MitM) attacks. Forward secrecy-based protocols protect against the compromising of keys by generating new key pairs for each session.

Diffie-Hellman key exchange is commonly found in security protocols, such as Transport Layer Security (TLS), Secure Shell (SSH) and IP Security (IPsec). For example, in IPsec, the encryption method is used for key generation and key rotation.

Even though Diffie-Hellman key exchange can be used for establishing both public and private keys, the Rivest-Shamir-Adleman algorithm, or RSA algorithm, can also be used, since it's able to sign public key certificates.

How does Diffie-Hellman key exchange work?

To implement Diffie-Hellman, two end users, Alice and Bob, mutually agree on positive whole numbers p and q, such that p is a prime number and q is a

generator of p. The generator q is a number that, when raised to positive whole-number powers less than p, never produces the same result for any two such whole numbers. The value of p may be large, but the value of q is usually small.

Once Alice and Bob have agreed on p and q in private, they choose positive whole-number personal keys a and b. Both are less than the prime number modulus p. Neither user divulges their personal key to anyone; ideally, they memorize these numbers and don't write them down or store them anywhere. Next, Alice and Bob compute public keys a* and b* based on their personal keys according to the following formulas:

$a^* = q_a \bmod p$

$b^* = q_b \bmod p$

The two users can share their public keys a* and b* over a communications medium assumed to be insecure, such as the internet or a corporate wide area network. From these public keys, a number x can be generated by either user on the basis of their own personal keys. Alice computes x using the following formula:

$x = (b^*) \bmod p$

Bob computes x using the following formula:

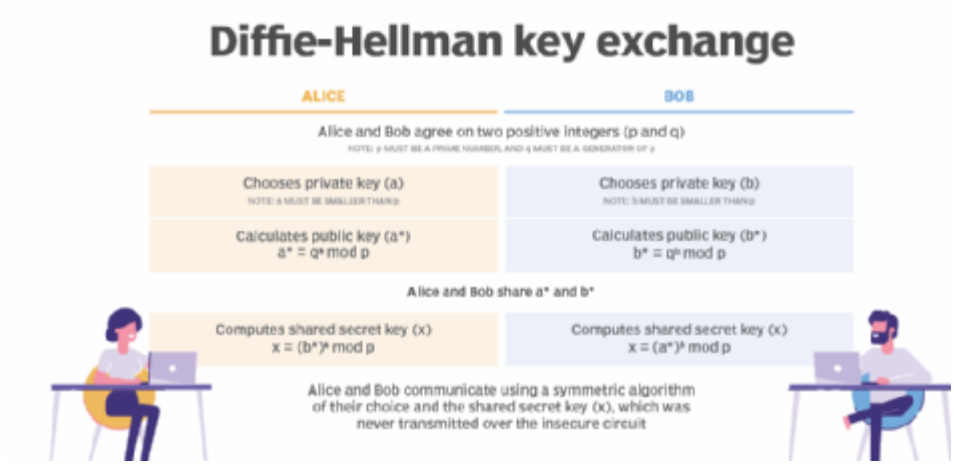
$x = (a^*) \bmod p$

The value of x turns out to be the same according to either of the above two formulas. However, the personal keys a and b, which are critical in the calculation of x, haven't been transmitted over a public medium. Because it's a large and apparently random number, a potential hacker has almost no chance of correctly guessing x, even with the help of a powerful computer to conduct millions of trials. The two users can, therefore, in theory, communicate privately over a public medium with an encryption method of their choice using the decryption key x.

Vulnerabilities of Diffie-Hellman key Exchange

The most serious limitation of Diffie-Hellman in its basic form is the lack of authentication. Communications using Diffie-Hellman by itself are vulnerable to MitM. Ideally, Diffie-Hellman should be used in conjunction with a recognized authentication method, such as digital signatures, to verify the identities of the users over the public communications medium.

Diffie-Hellman key exchange is also vulnerable to logjam attacks, specifically against the TLS protocol. Logjam attacks downgrade TLS connections to 512-bit cryptography, enabling an attacker to read and modify data that's passed through the connection. Diffie-Hellman key exchange can still be secure if implemented correctly. For example, logjam attacks won't work with a 2,048-bit key.



Examples of Diffie-Hellman key Exchange

If two people, say Alice and Bob, want to communicate sensitive data over an open public network but want to avoid hackers or eavesdroppers, they can use Diffie-Hellman key exchange method for encryption. This open public network could be at a cafe, for example.

Alice and Bob privately choose a secret key, and a function is run on these keys to create a public key. The results -- and not the function -- are shared. Even if a third party is listening in, that third party won't have all the involved numbers, making it difficult to derive the function the numbers came from.

From here, Alice and Bob each run a new function using the results they received from the opposite party, their own secret number and the original prime value. Alice and Bob then arrive at a common shared secret key that a third party can't deduce. Alice and Bob are now free to communicate without worrying about third parties.

Knapsack Encryption Algorithm

Knapsack Encryption Algorithm is the first general public key cryptography algorithm. It is developed by **Ralph Merkle** and **Mertin Hellman** in 1978.

As it is a Public key cryptography, it needs two different keys. One is Public key which is used for Encryption process and the other one is Private key which is used for Decryption process.

In this algorithm we will use two different knapsack problems in which one is easy and other one is hard. The easy knapsack is used as the private key and the hard knapsack is used as the public key. The easy knapsack is used to derived the hard knapsack.

For the easy knapsack, we will choose a **Super Increasing knapsack problem**. Super increasing knapsack is a sequence in which every next term is greater than the sum of all preceding terms.

Example –

{1, 2, 4, 10, 20, 40} is a super increasing as

1<2, 1+2<4, 1+2+4<10, 1+2+4+10<20 and 1+2+4+10+20<40.

Derive the Public key

- **Step-1:**

  Choose a super increasing knapsack {1, 2, 4, 10, 20, 40} as the private key.

- **Step-2:**

  Choose two numbers n and m. Multiply all the values of private key by the number n and then find modulo m. The value of m must be greater than the sum of all values in private key, for example 110. And the number n should have no common factor with m, for example 31.

- **Step-3:**
  Calculate the values of Public key using m and n.

1x31 mod(110) = 31
2x31 mod(110) = 62

4x31 mod(110) = 14

10x31 mod(110) = 90

20x31 mod(110) = 70

40x31 mod(110) = 30

- Thus, our public key is {31, 62, 14, 90, 70, 30}
  And Private key is {1, 2, 4, 10, 20, 40}.

Now take an example for understanding the process of encryption and decryption.

Example –

Let’s our plain text is 100100111100101110.

1. Encryption :
As our knapsacks contain six values, so we will split our plain text in a groups of six:
100100 111100 101110

Multiply each values of public key with the corresponding values of each group and take their sum.

100100 {31, 62, 14, 90, 70, 30}

1x31+0x62+0x14+1x90+0x70+0x30 = 121

111100 {31, 62, 14, 90, 70, 30}

1x31+1x62+1x14+1x90+0x70+0x30 = 197

101110 {31, 62, 14, 90, 70, 30}

1x31+0x62+1x14+1x90+1x70+0x30 = 205

So, our cipher text is 121 197 205.

2. Decryption :

The receiver receive the cipher text which has to be decrypt. The receiver also knows the values of m and n.

So, first we need to find the $\mathbf{n^{-1}}$, which is multiplicative inverse of n mod m i.e.,

n x $\mathbf{n^{-1}}$ mod(m) = 131 x $\mathbf{n^{-1}}$ mod(110) = 1 $\mathbf{n^{-1}}$ = 71

Now, we have to multiply 71 with each block of cipher text take modulo m.

121 x 71 mod(110) = 11

Then, we will have to make the sum of 11 from the values of private key {1, 2, 4, 10, 20, 40} i.e.,

1+10=11 so make that corresponding bits 1 and others 0 which is 100100.

Similarly,

197 x 71 mod(110) = 17

1+2+4+10=17 = 111100

And, 205 x 71 mod(110) = 35

1+4+10+20=35 = 101110

After combining them we get the decoded text.

100100111100101110 which is our plain text.

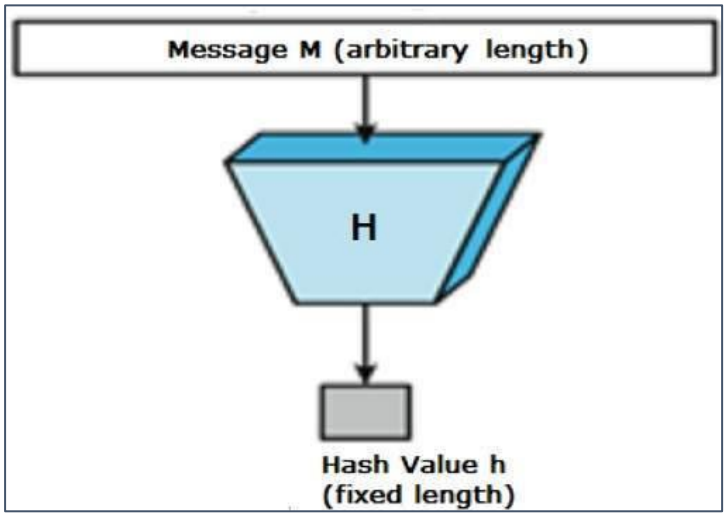
Unit-3

Cryptography Hash functions:

Hash functions are extremely useful and appear in almost all information security applications.

A hash function is a mathematical function that converts a numerical input value into another compressed numerical value. The input to the hash function is of arbitrary length but output is always of fixed length.

Values returned by a hash function are called **message digest** or simply **hash values**. The following picture illustrated hash function:



Features of Hash Functions:

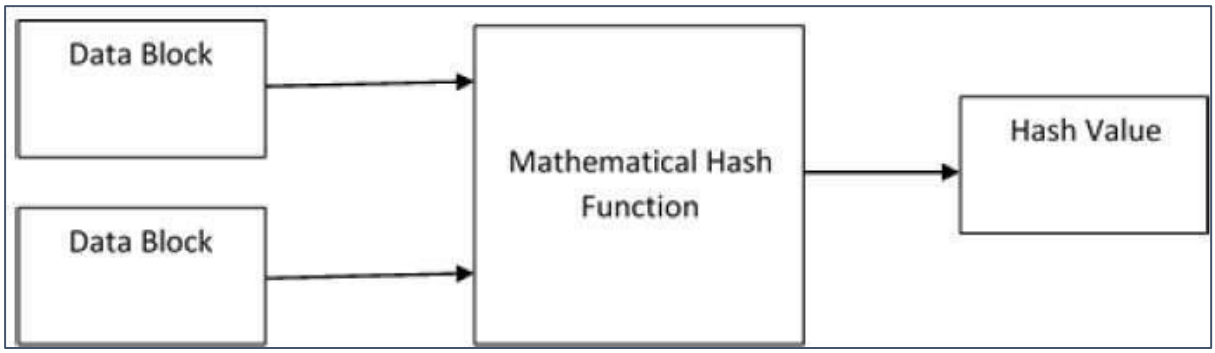
The typical features of hash functions are:

1. **Fixed Length Output (Hash Value):**
    - Hash function coverts data of arbitrary length to a fixed length. This process is often referred to as **hashing the data**.
    - In general, the hash is much smaller than the input data, hence hash functions are sometimes called **compression functions**.
    - Since a hash is a smaller representation of a larger data, it is also referred to as a **digest**.
    - Hash function with n bit output is referred to as an **n-bit hash function**. Popular hash functions generate values between 160 and 512 bits.
2. **Efficiency of Operation**
    - Generally, for any hash function h with input x, computation of h(x) is a fast operation.
    - Computationally hash functions are much faster than a symmetric encryption.

Design of Hashing Algorithms:

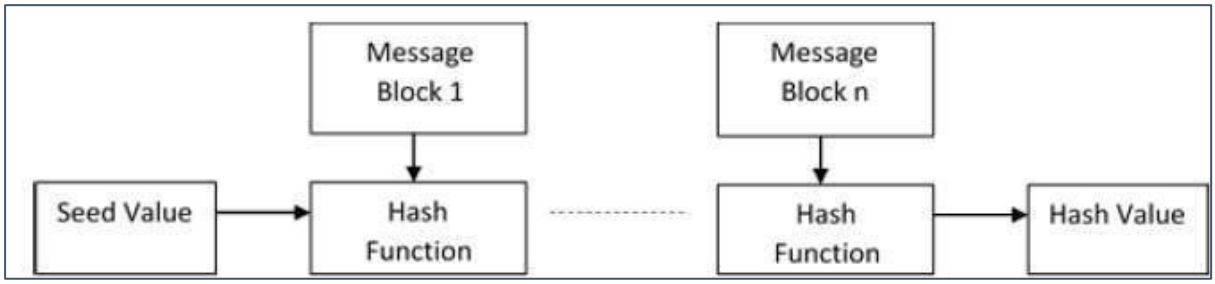
At the heart of a hashing is a mathematical function that operates on two fixed-size blocks of data to create a hash code. This hash function forms the part of the hashing algorithm.

The size of each data block varies depending on the algorithm. Typically, the block sizes are from 128 bits to 512 bits. The following illustration demonstrates hash function −



Hashing algorithm involves rounds of above hash function like a block cipher. Each round takes an input of a fixed size, typically a combination of the most recent message block and the output of the last round.

This process is repeated for as many rounds as are required to hash the entire message. Schematic of hashing algorithm is depicted in the following illustration:



Since, the hash value of first message block becomes an input to the second hash operation, output of which alters the result of the third operation, and so on. This effect, known as an **avalanche** effect of hashing.

Avalanche effect results in substantially different hash values for two messages that differ by even a single bit of data.

Understand the difference between hash function and algorithm correctly. The hash function generates a hash code by operating on two blocks of fixed-length binary data.

Hashing algorithm is a process for using the hash function, specifying how the message will be broken up and how the results from previous message blocks are chained together.

Popular Hash Functions:

Let us briefly see some popular hash functions:

1. **Message Digest (MD):**

   MD5 was most popular and widely used hash function for quite some years.

   - The MD family comprises of hash functions MD2, MD4, MD5 and MD6. It was adopted as Internet Standard RFC 1321. It is a 128-bit hash function.
   - MD5 digests have been widely used in the software world to provide assurance about integrity of transferred file. For example, file servers often provide a pre-computed MD5 checksum for the files, so that a user can compare the checksum of the downloaded file to it.
   - In 2004, collisions were found in MD5. An analytical attack was reported to be successful only in an hour by using computer cluster. This collision attack resulted in compromised MD5 and hence it is no longer recommended for use.

2. **Secure Hash Function (SHA):**

   Family of SHA comprise of four SHA algorithms; SHA-0, SHA-1, SHA-2, and SHA-3. Though from same family, there are structurally different.

   - The original version is SHA-0, a 160-bit hash function, was published by the National Institute of Standards and Technology (NIST) in 1993. It had few weaknesses and did not become very popular. Later in 1995, SHA-1 was designed to correct alleged weaknesses of SHA-0.
   - SHA-1 is the most widely used of the existing SHA hash functions. It is employed in several widely used applications and protocols including Secure Socket Layer (SSL) security.
   - In 2005, a method was found for uncovering collisions for SHA-1 within practical time frame making long-term employability of SHA-1 doubtful.
   - SHA-2 family has four further SHA variants, SHA-224, SHA-256, SHA-384, and SHA-512 depending up on number of bits in their hash value. No successful attacks have yet been reported on SHA-2 hash function.
   - Though SHA-2 is a strong hash function. Though significantly different, its basic design is still following design of SHA-1. Hence, NIST called for new competitive hash function designs.

- In October 2012, the NIST chose the Keccak algorithm as the new SHA-3 standard. Keccak offers many benefits, such as efficient performance and good resistance for attacks.

3. **RIPEMD:**

   The RIPEMD is an acronym for RACE Integrity Primitives Evaluation Message Digest. This set of hash functions was designed by open research community and generally known as a family of European hash functions.

   - The set includes RIPEMD, RIPEMD-128, and RIPEMD-160. There also exist 256, and 320-bit versions of this algorithm.
   - Original RIPEMD (128 bit) is based upon the design principles used in MD4 and found to provide questionable security. RIPEMD 128-bit version came as a quick fix replacement to overcome vulnerabilities on the original RIPEMD.
   - RIPEMD-160 is an improved version and the most widely used version in the family. The 256 and 320-bit versions reduce the chance of accidental collision, but do not have higher levels of security as compared to RIPEMD-128 and RIPEMD-160 respectively.

4. **Whirlpool:**

   This is a 512-bit hash function.

   - It is derived from the modified version of Advanced Encryption Standard (AES). One of the designer was Vincent Rijmen, a co-creator of the AES.
   - Three versions of Whirlpool have been released; namely WHIRLPOOL-0, WHIRLPOOL-T, and WHIRLPOOL.

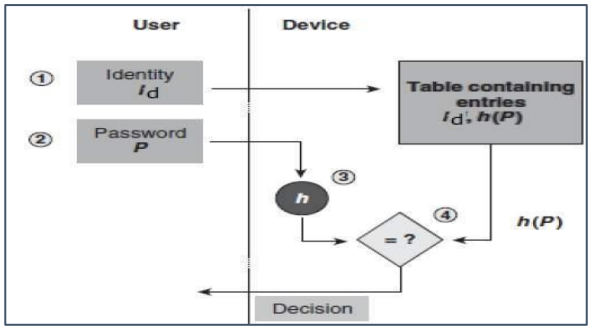
Applications of Hash Functions:

There are two direct applications of hash function based on its cryptographic properties.

1. **Password Storage:**

   Hash functions provide protection to password storage.

   - Instead of storing password in clear, mostly all logon processes store the hash values of passwords in the file.
   - The Password file consists of a table of pairs which are in the form (user id, h(P)).
   - The process of logon is depicted in the following illustration

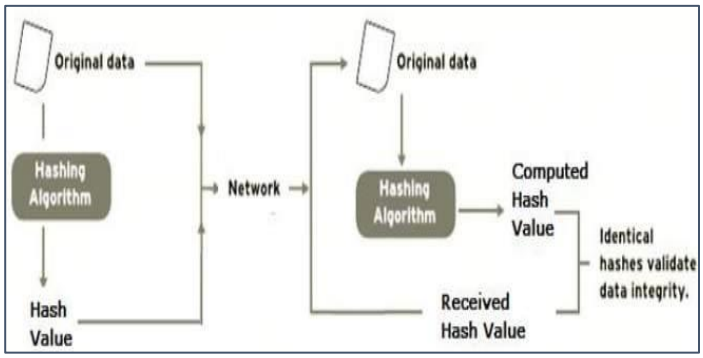


- An intruder can only see the hashes of passwords, even if he accessed the password. He can neither logon using hash nor can he derive the password from hash value since hash function possesses the property of pre-image resistance.

2. Data Integrity Check:

Data integrity check is a most common application of the hash functions. It is used to generate the checksums on data files. This application provides assurance to the user about correctness of the data.

The process is depicted in the following illustration −



The integrity check helps the user to detect any changes made to original file. It however, does not provide any assurance about originality. The attacker, instead of modifying file data, can change the entire file and compute all together new hash and send to the receiver. This integrity check application is useful only if the user is sure about the originality of file.

Message Authentication:

Message authentication is a procedure to verify that received messages come from the alleged source and have not been altered. Message authentication may also verify sequencing and timeliness. It is intended against the attacks like content modification, sequence modification, timing modification and repudiation. For repudiation, concept of digital signatures is used to counter it.

There are three classes by which different types of functions that may be used to produce an authenticator. They are:

1. Message encryption: The ciphertext serves as authenticator
2. Message authentication code (MAC): A public function of the message and a secret key producing a fixed-length value to serve as authenticator. This does not provide a digital signature because A and B share the same key.
3. Hash function: A public function mapping an arbitrary length message into a fixed- length hash value to serve as authenticator. This does not provide a digital signature because there is no key.

1. Message Encryption:

Message encryption by itself can provide a measure of authentication. The analysis differs for conventional and public-key encryption schemes. The message must have come from the sender itself, because the ciphertext can be decrypted using his (secret or public) key. Also, none of the bits in the message have been altered because an opponent does not know how to manipulate the bits of the ciphertext to induce meaningful changes to the plaintext. Often one needs alternative authentication schemes than just encrypting the message.

Sometimes one needs to avoid encryption of full messages due to legal requirements.

Encryption and authentication may be separated in the system architecture.

The different ways in which message encryption can provide authentication, confidentiality in both symmetric and asymmetric encryption techniques is explained with the table below:

Confidentiality and Authentication Implications of Message Encryption

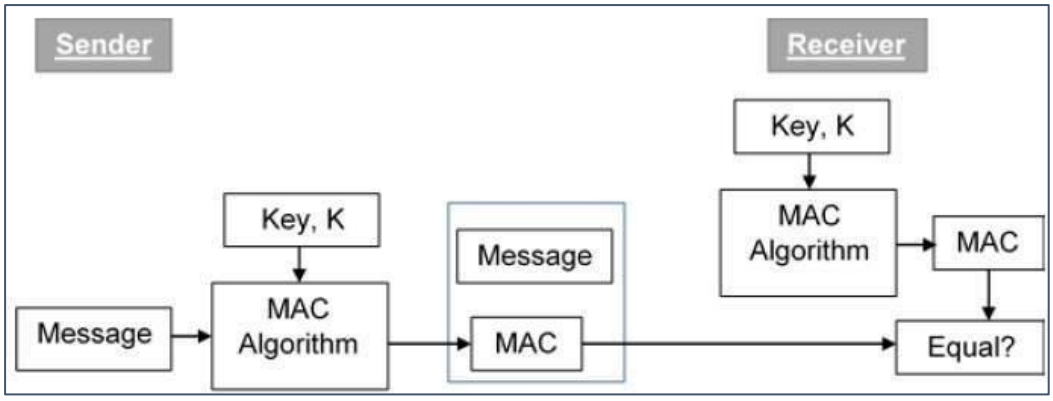
A → B: $E_K[M]$ •Provides confidentiality —Only A and B share K •Provides a degree of authentication —Could come only from A —Has not been altered in transit —Requires some formatting/redundancy •Does not provide signature —Receiver could forge message —Sender could deny message (a) Symmetric encryption
A → B: $E_{KU_b}[M]$ •Provides confidentiality —Only B has KR_b to decrypt •Provides no authentication —Any party could use KU_b to encrypt message and claim to be A (b) Public-key encryption: confidentiality
A → B: $E_{KR_a}[M]$ •Provides authentication and signature —Only A has KR_a to encrypt —Has not been altered in transit —Requires some formatting/redundancy —Any party can use KU_a to verify signature (c) Public-key encryption: authentication and signature
A → B: $E_{KU_b}[E_{KR_a}(M)]$ •Provides confidentiality because of KU_b •Provides authentication and signature because of Kr_a (d) Public-key encryption: confidentiality, authentication, and signature

2. Message Authentication Code (MAC)

MAC algorithm is a symmetric key cryptographic technique to provide message authentication. For establishing MAC process, the sender and receiver share a symmetric key K.

Essentially, a MAC is an encrypted checksum generated on the underlying message that is sent along with a message to ensure message authentication.

The process of using MAC for authentication is depicted in the following illustration −



Let us now try to understand the entire process in detail:

- The sender uses some publicly known MAC algorithm, inputs the message and the secret key K and produces a MAC value.
- Similar to hash, MAC function also compresses an arbitrary long input into a fixed length output. The major difference between hash and MAC is that MAC uses secret key during the compression.
- The sender forwards the message along with the MAC. Here, we assume that the message is sent in the clear, as we are concerned of providing message origin authentication, not confidentiality. If confidentiality is required then the message needs encryption.
- On receipt of the message and the MAC, the receiver feeds the received message and the shared secret key K into the MAC algorithm and re-computes the MAC value.
- The receiver now checks equality of freshly computed MAC with the MAC received from the sender. If they match, then the receiver accepts the message and assures himself that the message has been sent by the intended sender.
- If the computed MAC does not match the MAC sent by the sender, the receiver cannot determine whether it is the message that has been altered or it is the origin that has been falsified. As a bottom-line, a receiver safely assumes that the message is not the genuine.

Secure Hash Algorithm (SHA-512):

SHA-512 is a hashing algorithm that performs a hashing function on some data given to it.

Hashing algorithms are used in many things such as internet security, digital certificates and even blockchains. Since hashing algorithms play such a vital role in digital security and cryptography, this is an easy-to-understand walkthrough, with some basic and simple maths along with some diagrams, for a hashing algorithm called SHA-512. It's part of a group of hashing algorithms called SHA-2 which includes SHA-256 as well which is used in the bitcoin blockchain for hashing.

Hashing Functions

Hashing functions take some data as input and produce an output (called hash digest) of fixed length for that input data. This output should, however, satisfy some conditions to be useful.

1. Uniform Distribution: Since the length of the output hash digest is of a fixed length and the input size may vary, it is apparent that there are going to be some output values that can be obtained for different input values. Even though this is the case, the hash function should be such that for any input value, each possible output value should be equally likely. That is to say that every possible output has the same likelihood to be produced for any given input value.

2. Fixed Length: This is should be quite self-explanatory. The output values should all be of a fixed length. So, for example, a hashing function could have an output size of 20 characters or 12 characters, etc. SHA-512 has an output size of 512 bits.

3. Collision Resistance: Simply speaking, this means that there aren't any or rather it is not feasible to find two distinct inputs to the hash function that result in the same output (hash digest).

Hashing Algorithm:

SHA-512 does its work in a few stages. These stages go as follows:

1.Input formatting

2.Hash buffer initialization

3.Message Processing

4.Output

1. Input Formatting:

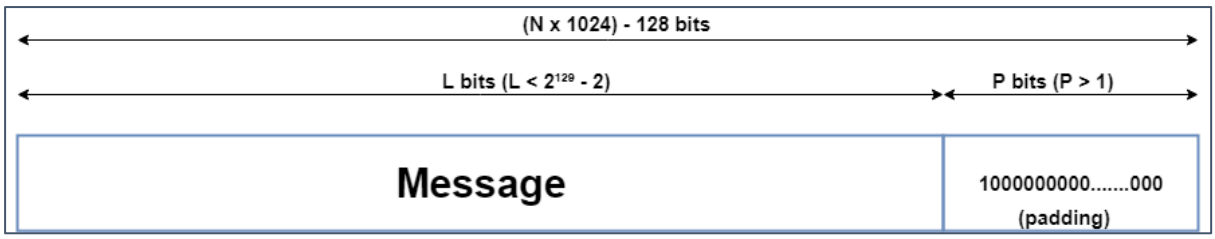
SHA-512 can't actually hash a message input of any size, i.e., it has an input size limit. This limit is imposed by its very structure as you may see further on. The entire formatted message has basically three parts: the original message, padding bits, size of original message. And this should all have a combined size of a whole multiple of 1024 bits. This is because the formatted message will be processed as blocks of 1024 bits each, so each bock should have 1024 bits to work with.

i. Original Message

ii. Padding bits

The input message is taken and some padding bits are appended to it in order to get it to the desired length. The bits that are used for padding are simply '0' bits with a leading '1' (100000…000). Also, according to the algorithm, padding needs to be done, even if it is by one bit. So, a single padding bit would only be a '1'.

The total size should be equal to 128 bits short of a multiple of 1024 since the goal is to have the formatted message size as a multiple of 1024 bits (N x 1024).

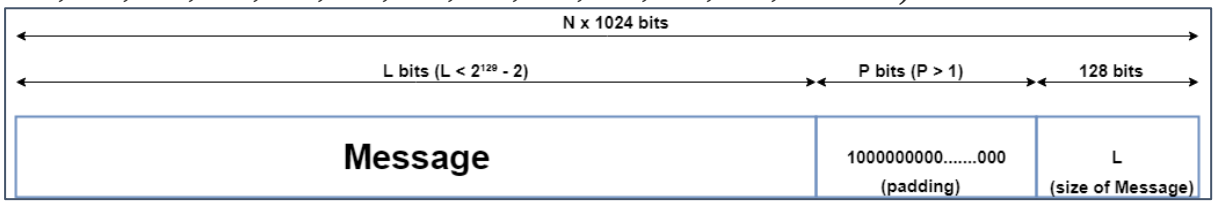


Message with padding

iii. **Padding size**

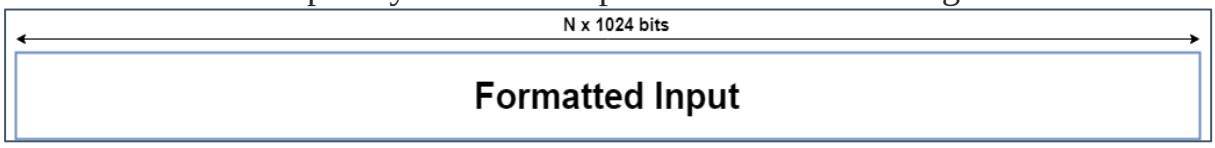
After this, the size of the original message given to the algorithm is appended. This size value needs to be represented in 128 bits and is the only reason that the SHA-512 has a limitation for its input message.

Since the size of the original message needs to be represented in 128 bits and the largest number that can be represented using 128 bits is (2^{128}-1), the message size can be at most (2^{128}-1) bits; and also taking into consideration the necessary single padding bit, the maximum size for the original message would then be (2^{128}-2). Even though this limit exists, it doesn't actually cause a problem since the actual limit is so high (2^{128}-2 = 340,282,366,920,938,463,463,374,607,431,768,211,454 bits).



Message with padding and size

Now that the padding bits and the size of the message have been appended, we are left with the completely formatted input for the SHA-512 algorithm.



Formatted Message

2. Hash Buffer Initialization:

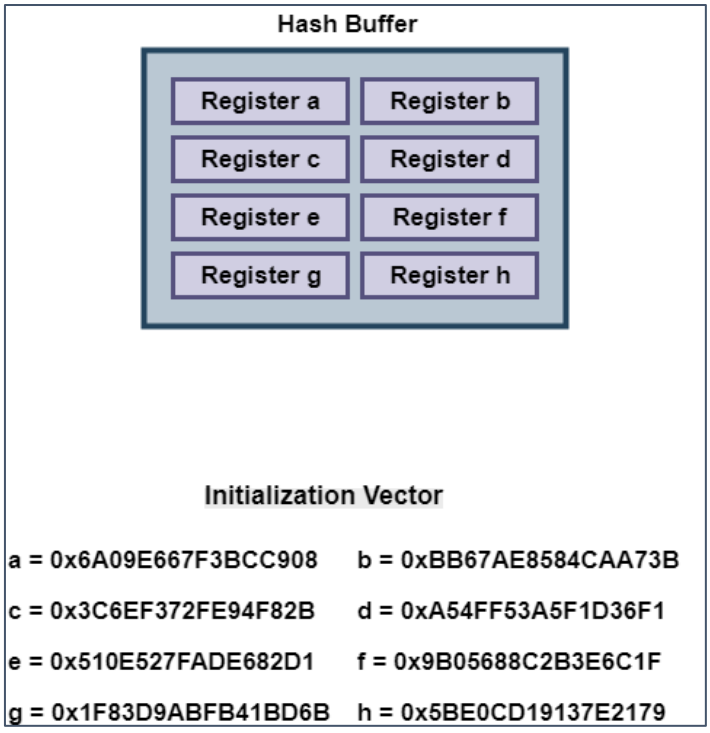
The algorithm works in a way where it processes each block of 1024 bits from the message using the result from the previous block. Now, this poses a problem for the first 1024-bit block which can't use the result from any previous processing.

This problem can be solved by using a default value to be used for the first block in order to start off the process. (Have a look at the second-last diagram).

Since each intermediate result needs to be used in processing the next block, it needs to be stored somewhere for later use. This would be done by the hash buffer; this would also then hold the final hash digest of the entire processing phase of SHA-512 as the last of these 'intermediate' results.

So, the default values used for starting off the chain processing of each 1024-bit block are also stored into the hash buffer at the start of processing. The actual value used is of little consequence, but for those interested, the values used are obtained by taking the first 64 bits of the fractional parts of the square roots of the first 8 prime numbers (2,3,5,7,11,13,17,19). These values are called the Initial Vectors (IV).

Why 8 prime numbers instead of 9? Because the hash buffer actually consists of 8 subparts (registers) for storing them.

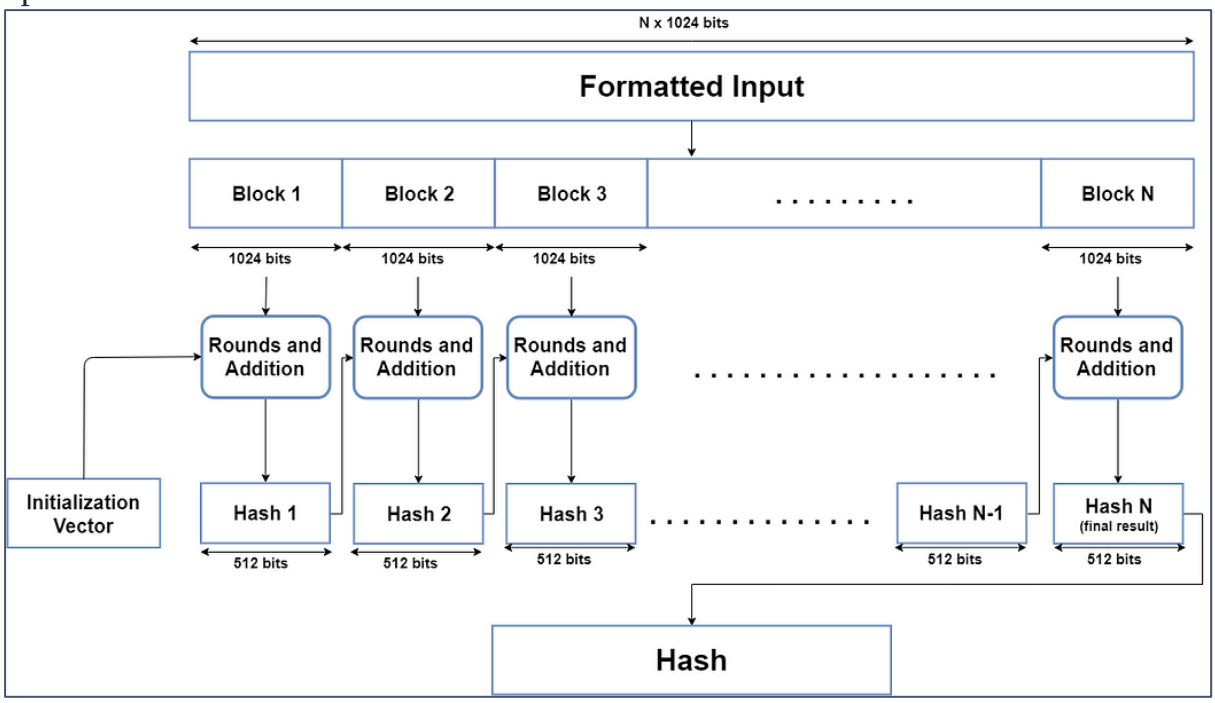


Hash buffer and Initialization Vector values

3. Message Processing:

Message processing is done upon the formatted input by taking one block of 1024 bits at a time. The actual processing takes place by using two things: The 1024-bit block, and the result from the previous processing.

This part of the SHA-512 algorithm consists of several ‘Rounds’ and an addition operation.



So, the Message block (1024 bit) is expanded out into ‘Words’ using a ‘message sequencer’. Eighty Words to be precise, each of them having a size of 64 bits.

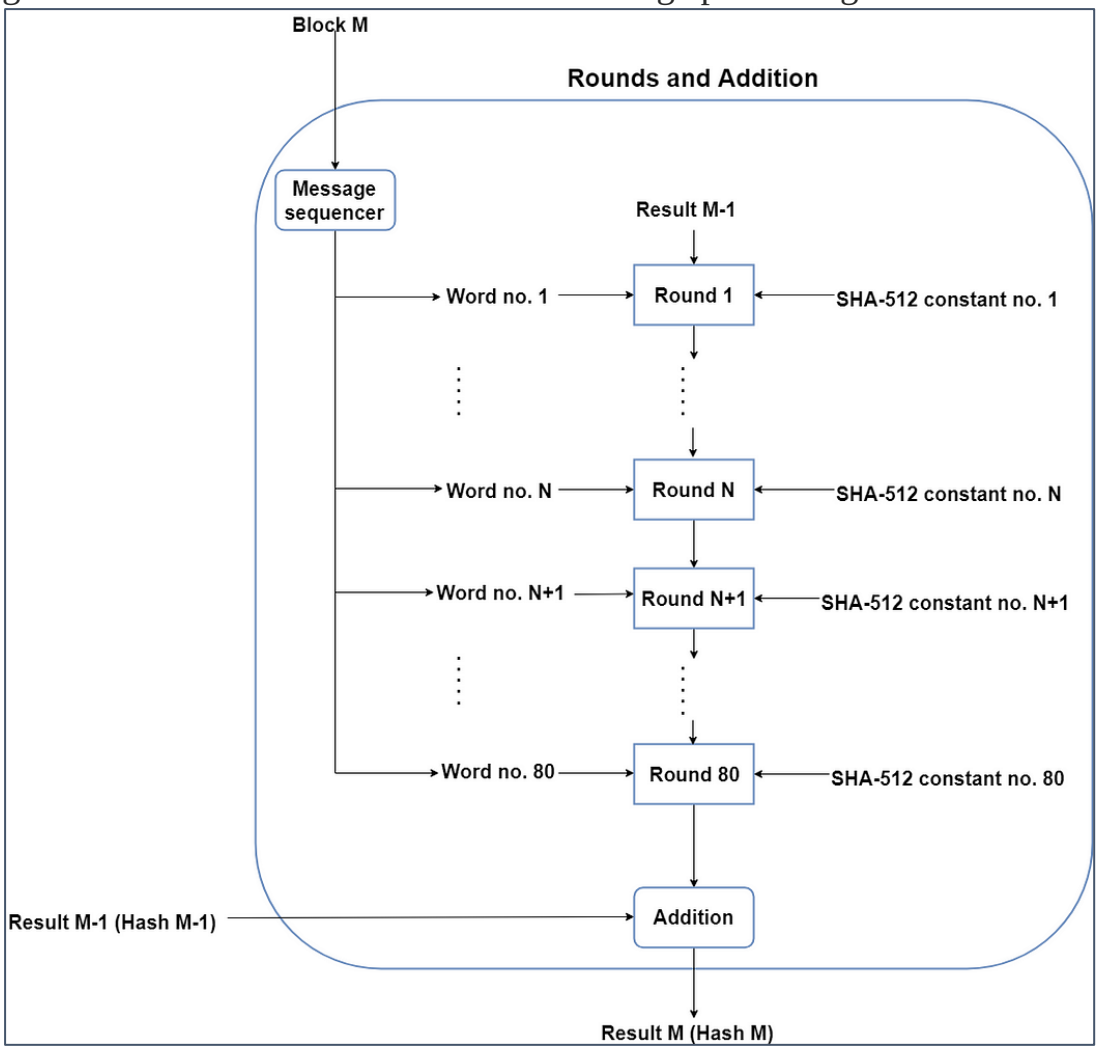
Rounds

The main part of the message processing phase may be considered to be the Rounds. Each round takes 3 things: one Word, the output of the previous Round, and a SHA-512 constant. The first Round doesn’t have a previous Round whose output it can use, so it uses the final output from the previous message processing phase for the previous block of 1024 bits. For the first Round of the first block (1024 bits) of the formatted input, the Initial Vector (IV) is used.

SHA-512 constants are predetermined values, each of whom is used for each Round in the message processing phase. Again, these aren’t very important, but for those interested, they are the first 64 bits from the fractional part of the cube

roots of the first 80 prime numbers. Why 80? Because there are 80 Rounds and each of them needs one of these constants.

Once the Round function takes these 3 things, it processes them and gives an output of 512 bits. This is repeated for 80 Rounds. After the 80th Round, its output is simply added to the result of the previous message processing phase to get the final result for this iteration of message processing.



4. Output:

After every block of 1024 bits goes through the message processing phase, i.e., the last iteration of the phase, we get the final 512 bit Hash value of our original message. So, the intermediate results are all used from each block for processing the next block. And when the final 1024 bit block has finished being processed, we have with us the final result of the SHA-512 algorithm for our original message.

Thus, we obtain the final hash value from our original message. The SHA-512 is part of a group of hashing algorithms that are very similar in how they work, called SHA-2. Algorithms such as SHA-256 and SHA-384 are a part of this group

alongside SHA-512. SHA-256 is also used in the Bitcoin blockchain as the designated hash function. That is a brief overview of how the SHA-512 hashing algorithm works.

Message Authentication Codes:

1. Authentication requirements:

- **Revelation:** It means releasing the content of the message to someone who does not have an appropriate cryptographic key.
- **Analysis of Traffic:** Determination of the pattern of traffic through the duration of connection and frequency of connections between different parties.
- **Deception:** Adding out of context messages from a fraudulent source into a communication network. This will lead to mistrust between the parties communicating and may also cause loss of critical data.
- **Modification in the Content:** Changing the content of a message. This includes inserting new information or deleting/changing the existing one.
- **Modification in the sequence:** Changing the order of messages between parties. This includes insertion, deletion, and reordering of messages.
- **Modification in the Timings:** This includes replay and delay of messages sent between different parties. This way session tracking is also disrupted.
- **Source Refusal:** When the source denies being the originator of a message.
- **Destination refusal:** When the receiver of the message denies the reception.

Message Authentication Functions:

All message authentication and digital signature mechanisms are based on two functionality levels:

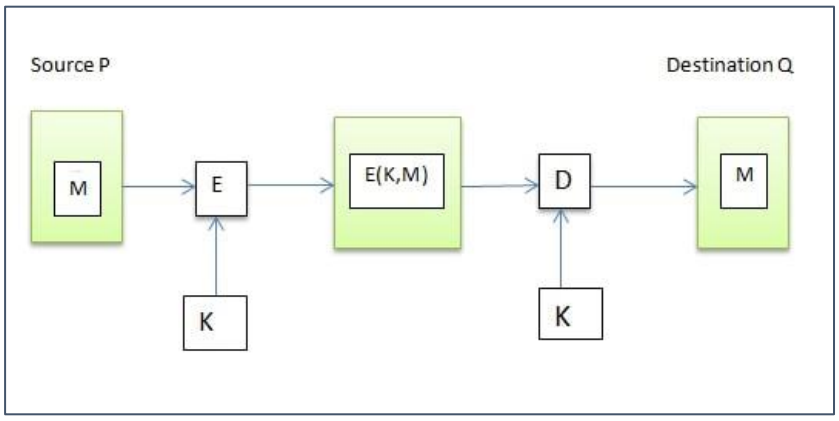
- **Lower level:** At this level, there is a need for a function that produces an authenticator, which is the value that will further help in the authentication of a message.
- **Higher-level:** The lower-level function is used here in order to help receivers verify the authenticity of messages.

These message authentication functions are divided into three classes:

- **Message encryption:** While sending data over the internet, there is always a risk of a Man in the middle(MITM) attack. A possible solution for this is to use message encryption. In message encryption, the data is first converted to a ciphertext and then sent any further.

  Message encryption can be done in two ways:

- **Symmetric Encryption:** Say we have to send the message M from a source P to destination Q. This message M can be encrypted using a secret key K that both P and Q share. Without this key K, no other person can get the plain text from the ciphertext. This maintains confidentiality. Further, Q can be sure that P has sent the message. This is because other than Q, P is the only party who possesses the key K and thus the ciphertext can be decrypted only by Q and no one else. This maintains authenticity. At a very basic level, symmetric encryption looks like this:

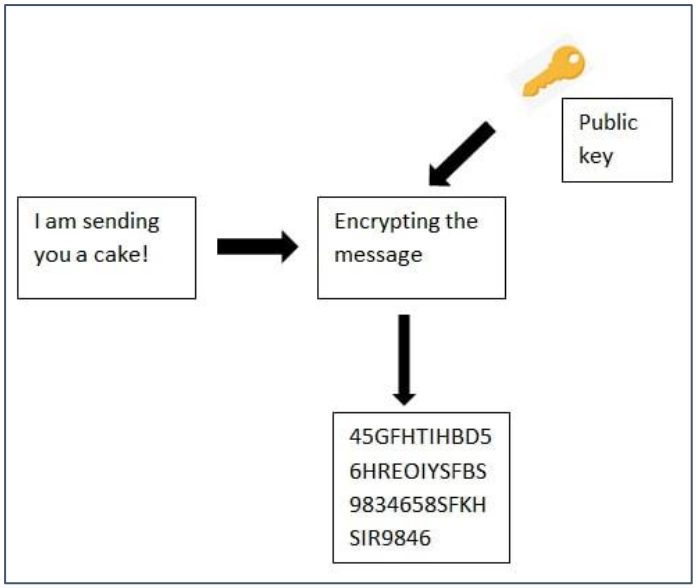


- **Public key Encryption:** Public key encryption is not as advanced as symmetric encryption as it provides confidentiality but not authentication. To provide both authentication and confidentiality, the private key is used.
- **Message authentication code (MAC):** A message authentication code is a security code that the user of a computer has to type in order to access any account or portal. These codes are recognized by the system so that it can grant access to the right user. These codes help in maintaining information integrity. It also confirms the authenticity of the message.
- **Hash function:** A hash function is nothing but a mathematical function that can convert a numeric value into another numeric value that is compressed. The input to this hash function can be of any length but the output is always of fixed length. The values that a hash function returns are called the message digest or hash values.

Measures to deal with these attacks:

Each of the above attacks has to be dealt with differently.

- **Message Confidentiality:** To prevent the messages from being revealed, care must be taken during the transmission of messages. For this, the message should be encrypted before it is sent over the network.



- **Message Authentication:** To deal with the analysis of traffic and deception issues, message authentication is helpful. Here, the receiver can be sure of the real sender and his identity. To do this, these methods can be incorporated:

    - Parties should share secret codes that can be used at the time of identity authentication.
    - Digital signatures are helpful in the authentication.
    - A third party can be relied upon for verifying the authenticity of parties.

- **Digital Signatures:** Digital signatures provide help against a majority of these issues. With the help of digital signatures, content, sequence, and timing of the messages can be easily monitored. Moreover, it also prevents denial of message transmission by the source.
- **Combination of protocols with Digital Signatures:** This is needed to deal with the denial of messages received. Here, the use of digital signature is not sufficient and it additionally needs protocols to support its monitoring.

2. HMAC(Hash Based Message Authentication Code):

HMAC (Hash-based Message Authentication Code) is a type of a message authentication code (MAC) that is acquired by executing a cryptographic hash function on the data (that is) to be authenticated and a secret shared key. Like any of the MAC, it is used for both data integrity and authentication. Checking data integrity is necessary for the parties involved in communication. HTTPS, SFTP, FTPS, and other transfer protocols use HMAC. The cryptographic hash function may be MD-5, SHA-1, or SHA-256. Digital signatures are nearly similar to HMACs i.e., they both employ a hash function and a shared key. The difference lies in the keys i.e., HMACs use symmetric key(same copy) while Signatures use asymmetric (two different keys).

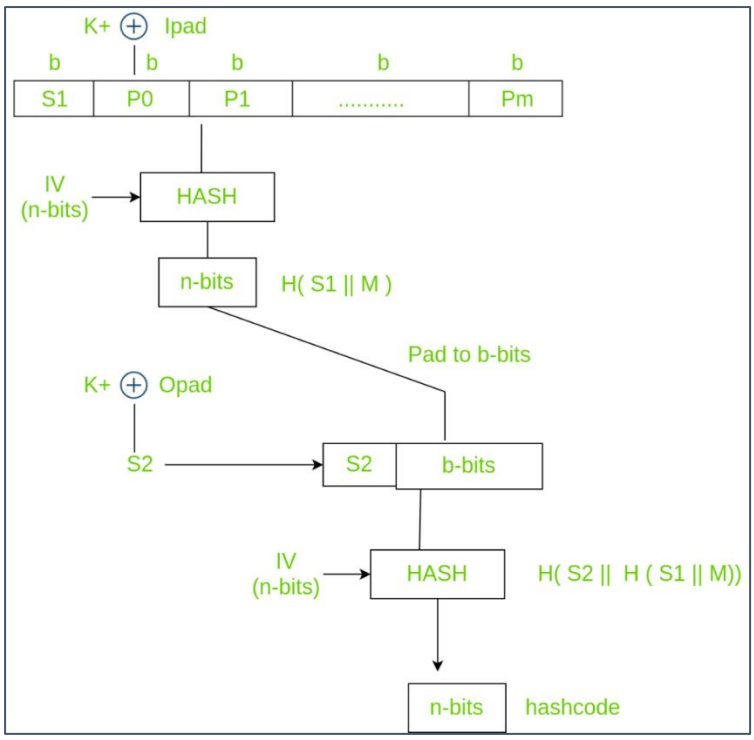
Working of HMAC:

HMACs provides client and server with a shared private key that is known only to them. The client makes a unique hash (HMAC) for every request. When the client requests the server, it hashes the requested data with a private key and sends it as a part of the request. Both the message and key are hashed in separate steps making it secure. When the server receives the request, it makes its own HMAC. Both the HMACS are compared and if both are equal, the client is considered legitimate.

The formula **for HMAC:**

HMAC = hashFunc(secret key + message)

There are three types of authentication functions. They are message encryption, message authentication code, and hash functions. The major difference between MAC and hash (HMAC here) is the dependence of a key. In HMAC we have to apply the hash function along with a key on the plain text. The hash function will be applied to the plain text message. But before applying, we have to compute S bits and then append it to plain text and after that apply the hash function. For generating those S bits, we make use of a key that is shared between the sender and receiver.



Using key K (0 < K < b), K+ is generated by padding O's on left side of key K until length becomes b bits. The reason why it's not padded on right is change(increase) in the length of key. b bits because it is the block size of plain text. There are two predefined padding bits called ipad and opad. All this is done before applying hash function to the plain text message.

ipad - 00110110

opad – 01011100

Now we have to calculate S bits

K+ is EXORed with ipad and the result is S1 bits which is equivalent to b bits since both K+ and ipad are b bits. We have to append S1 with plain text messages. Let P be the plain text message.
S1, p0, p1 upto Pm each is b bits. m is the number of plain text blocks. P0 is plain text block and b is plain text block size. After appending S1 to Plain text we have to apply HASH algorithm (any variant). Simultaneously we have to apply initialization vector (IV) which is a buffer of size n-bits. The result produced is therefore n-bit hash code i.e., H(S1 || M).
Similarly, n-bits are padded to b-bits And K+ is EXORed with opad producing output S2 bits. S2 is appended to the b-bits and once again hash function is applied with IV to the block. This further results into n-bit hash code which is H(S2 || H(S1 || M)).

Summary:

1. Select K.
   If K < b, pad 0's on left until k=b. K is between 0 and b (0 < K < b)
2. EXOR K+ with ipad equivalent to b bits producing S1 bits.
3. Append S1 with plain text M
4. Apply SHA-512 on (S1 || M)
5. Pad n-bits until length is equal to b-bits
6. EXOR K+ with opad equivalent to b bits producing S2 bits.
7. Append S2 with output of step 5.
8. Apply SHA-512 on step 7 to output n-bit hash code.

Advantages:

- HMACs are ideal for high-performance systems like routers due to the use of hash functions which are calculated and verified quickly unlike the public key systems.
- Digital signatures are larger than HMACs, yet the HMACs provide comparably higher security.
- HMACs are used in administrations where public key systems are prohibited.

Disadvantages:

- HMACs uses shared key which may lead to non-repudiation. If either sender or receiver's key is compromised then it will be easy for attackers to create unauthorized messages.

3. Digital Signatures:

Digital signatures are the public-key primitives of message authentication. In the physical world, it is common to use handwritten signatures on handwritten or typed messages. They are used to bind signatory to the message.

Similarly, a digital signature is a technique that binds a person/entity to the digital data. This binding can be independently verified by receiver as well as any third party.

Digital signature is a cryptographic value that is calculated from the data and a secret key known only by the signer.

In real world, the receiver of message needs assurance that the message belongs to the sender and he should not be able to repudiate the origination of that message. This requirement is very crucial in business applications, since likelihood of a dispute over exchanged data is very high.

Kerberos:

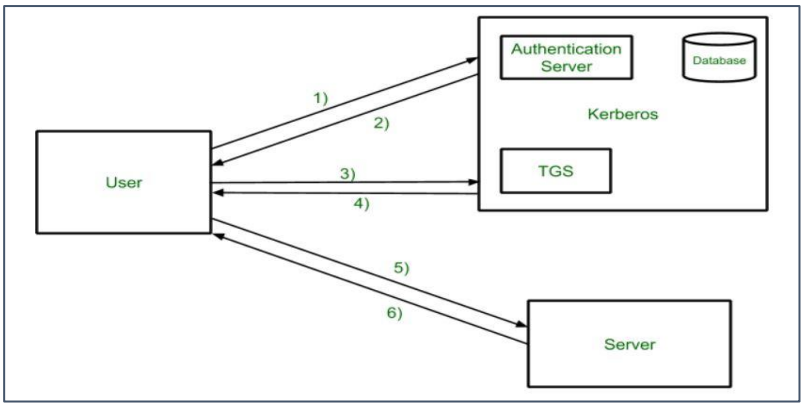
Kerberos provides a centralized authentication server whose function is to authenticate users to servers and servers to users. In Kerberos Authentication server and database is used for client authentication. Kerberos runs as a third-party trusted server known as the Key Distribution Centre (KDC). Each user and service on the network is a principal.
The main components of Kerberos are:

- **Authentication Server (AS):**
  The Authentication Server performs the initial authentication and ticket for Ticket Granting Service.

- **Database:**
  The Authentication Server verifies the access rights of users in the database.

- **Ticket Granting Server (TGS):**
  The Ticket Granting Server issues the ticket for the Server

Kerberos Overview:



- **Step-1:**
  User login and request services on the host. Thus, user requests for ticket-granting service.

- **Step-2:**
  Authentication Server verifies user's access right using database and then gives ticket-granting-ticket and session key. Results are encrypted using the Password of the user.

- **Step-3:**
  The decryption of the message is done using the password then send the ticket to Ticket Granting Server. The Ticket contains authenticators like user names and network addresses.

- **Step-4:**
  Ticket Granting Server decrypts the ticket sent by User and authenticator verifies the request then creates the ticket for requesting services from the Server.

- **Step-5:**
  The user sends the Ticket and Authenticator to the Server.

- **Step-6:**
  The server verifies the Ticket and authenticators then generate access to the service. After this User can access the services.

Kerberos Limitations:

- Each network service must be modified individually for use with Kerberos
- It doesn't work well in a timeshare environment
- Secured Kerberos Server
- Requires an always-on Kerberos server
- Stores all passwords are encrypted with a single key
- Assumes workstations are secure
- May result in cascading loss of trust.
- Scalability

Is Kerberos Infallible?

No security measure is 100% impregnable, and Kerberos is no exception. Because it's been around for so long, hackers have had the ability over the years to find ways around it, typically through forging tickets, repeated attempts at password guessing (brute force/credential stuffing), and the use of malware, to downgrade the encryption.

Despite this, Kerberos remains the best access security protocol available today. The protocol is flexible enough to employ stronger encryption algorithms to combat new threats, and if users employ good password-choice guidelines, you shouldn't have a problem!

What is Kerberos Used For?

Although Kerberos can be found everywhere in the digital world, it is commonly used in secure systems that rely on robust authentication and auditing capabilities. Kerberos is used for Posix, Active Directory, NFS, and Samba authentication. It is also an alternative authentication system to SSH, POP, and SMTP.

Applications:

- **User Authentication**: User Authentication is one of the main applications of Kerberos. Users only have to input their username and password once with Kerberos to gain access to the network. The Kerberos server subsequently receives the encrypted authentication data and issues a ticket granting ticket (TGT).
- **Single Sign-On (SSO)**: Kerberos offers a Single Sign-On (SSO) solution that enables users to log in once to access a variety of network resources. A user can access any network resource they have been authorized to use after being authenticated by the Kerberos server without having to provide their credentials again.
- **Mutual Authentication**: Before any data is transferred, Kerberos uses a mutual authentication technique to make sure that both the client and server are authenticated. Using a shared secret key that is securely kept on both the client and server, this is accomplished. A client asks the Kerberos server for a service ticket whenever it tries to access a network resource. The client must use its shared secret key to decrypt the challenge that the Kerberos server sends via encryption. If the decryption is successful, the client responds to the server with evidence of its identity.
- **Authorization**: Kerberos also offers a system for authorization in addition to authentication. After being authenticated, a user can submit service tickets for certain network resources. Users can access just the resources they have been given permission to use thanks to information about their privileges and permissions contained in the service tickets.
- **Network Security**: Kerberos offers a central authentication server that can regulate user credentials and access restrictions, which helps to ensure network security. In order to prevent unwanted access to

sensitive data and resources, this server may authenticate users before granting them access to network resources.

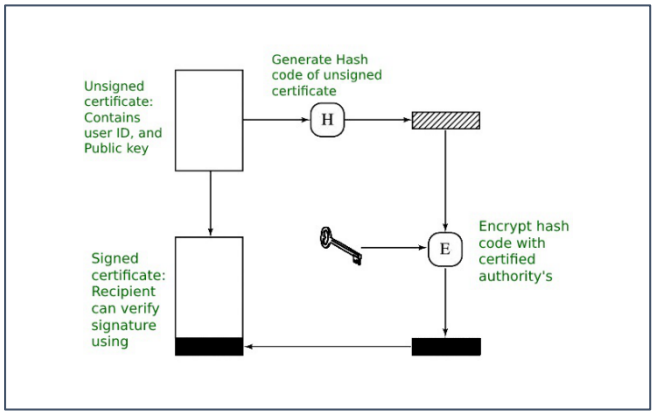
X.509 Authentication Service:

X.509 is a digital certificate that is built on top of a widely trusted standard known as ITU or International Telecommunication Union X.509 standard, in which the format of PKI certificates is defined. X.509 digital certificate is a certificate-based authentication security framework that can be used for providing secure transaction processing and private information. These are primarily used for handling the security and identity in computer networking and internet-based communications.

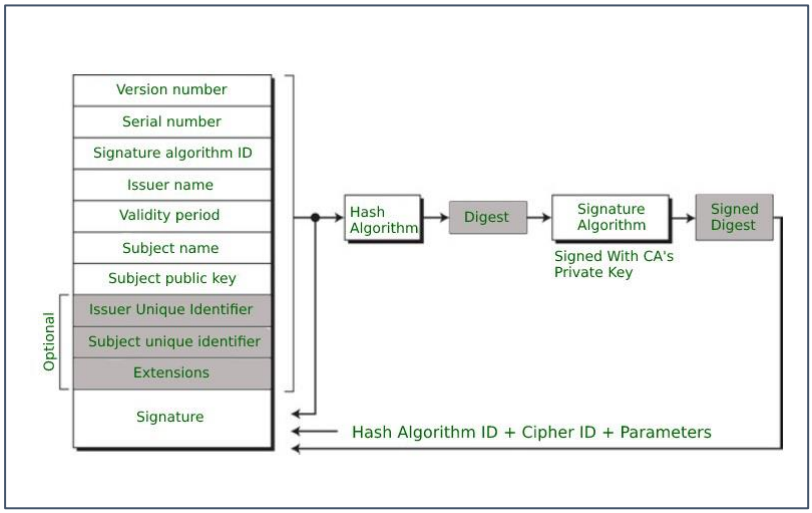
Working of X.509 Authentication Service Certificate:

The core of the X.509 authentication service is the public key certificate connected to each user. These user certificates are assumed to be produced by some trusted certification authority and positioned in the directory by the user or the certified authority. These directory servers are only used for providing an effortless reachable location for all users so that they can acquire certificates. X.509 standard is built on an IDL known as ASN.1. With the help of Abstract Syntax Notation, the X.509 certificate format uses an associated public and private key pair for encrypting and decrypting a message.

Once an X.509 certificate is provided to a user by the certified authority, that certificate is attached to it like an identity card. The chances of someone stealing it or losing it are less, unlike other unsecured passwords. With the help of this analogy, it is easier to imagine how this authentication works: the certificate is basically presented like an identity at the resource that requires authentication.



Format of X.509 Authentication Service Certificate:



Generally, the certificate includes the elements given below:

- **Version number:** It defines the X.509 version that concerns the certificate.
- **Serial number:** It is the unique number that the certified authority issues.
- **Signature Algorithm Identifier:** This is the algorithm that is used for signing the certificate.
- **Issuer name:** Tells about the X.500 name of the certified authority which signed and created the certificate.
- **Period of Validity:** It defines the period for which the certificate is valid.
- **Subject Name:** Tells about the name of the user to whom this certificate has been issued.
- **Subject's public key information:** It defines the subject's public key along with an identifier of the algorithm for which this key is supposed to be used.
- **Extension block:** This field contains additional standard information.
- **Signature:** This field contains the hash code of all other fields which is encrypted by the certified authority private key.

Applications of X.509 Authentication Service Certificate:

Many protocols depend on X.509 and it has many applications, some of them are given below:

- Document signing and Digital signature
- Web server security with the help of Transport Layer Security (TLS)/Secure Sockets Layer (SSL) certificates

- Email certificates
- Code signing
- Secure Shell Protocol (SSH) keys
- Digital Identities

Unit-4

Web Security Considerations:

The World Wide Web is fundamentally a client/server application running over the Internet and TCP/IP intranets. As such, the security tools and approaches discussed so far in this book are relevant to the issue of Web security. But, as pointed out in [GARF02], the Web presents new challenges not generally appreciated in the context of computer and network security.

The Internet is two-way. Unlike traditional publishing environments—even electronic publishing systems involving teletext, voice response, or fax-back—the Web is vulnerable to attacks on the Web servers over the Internet.

- The Web is increasingly serving as a highly visible outlet for corporate and product information and as the platform for business transactions. Reputations can be damaged and money can be lost if the Web servers are subverted.
- Although Web browsers are very easy to use, Web servers are relatively easy to configure and manage, and Web content is increasingly easy to develop, the underlying software is extraordinarily complex. This complex software may hide many potential security flaws. The short history of the Web is filled with examples of new and upgraded systems, properly installed, that are vulnerable to a variety of security attacks.
- A Web server can be exploited as a launching pad into the corporation's or agency's entire computer complex. Once the Web server is subverted, an attacker may be able to gain access to data and systems not part of the Web itself but connected to the server at the local site.
- Casual and untrained (in security matters) users are common clients for Web based services. Such users are not necessarily aware of the security risks that exist and do not have the tools or knowledge to take effective countermeasures.

Web Security Threats:

Table provides a summary of the types of security threats faced when using the Web. One way to group these threats is in terms of passive and active attacks. Passive attacks include eavesdropping on network traffic between browser and server and gaining access to information on a Web site that is supposed to be restricted. Active attacks include impersonating another user, altering messages in transit between client and server, and altering information on a Web site.

Another way to classify Web security threats is in terms of the location of the threat: Web server, Web browser, and network traffic between browser and

server. Issues of server and browser security fall into the category of computer system security; Part Four of this book addresses the issue of system security in general but is also applicable to Web system security. Issues of traffic security fall into the category of network security and are addressed in this chapter.

Web Traffic Security Approaches:

A number of approaches to providing Web security are possible. The various approaches that have been considered are similar in the services they provide and, to some extent, in the mechanisms that they use, but they differ with respect to their scope of applicability and their relative location within the TCP/IP protocol stack.

Figure 16.1 illustrates this difference. One way to provide Web security is to use IP security (IPsec) (Figure 16.1a). The advantage of using IPsec is that it is trans- parent to end users and applications and provides a general-purpose solution. Furthermore, IPsec includes a filtering capability so that only selected traffic need incur the overhead of IPsec processing.

Another relatively general-purpose solution is to implement security just above TCP (Figure 16.1b). The foremost example of this approach is the secure.

Table 16.1 A Comparison of Threats on the Web

	Threats	Consequences	Countermeasures
Integrity	• Modification of user data • Trojan horse browser • Modification of memory • Modification of message traffic in transit	• Loss of information • Compromise of machine • Vulnerabilty to all other threats	Cryptographic checksums
Confidentiality	• Eavesdropping on the net • Theft of info from server • Theft of data from client • Info about network configuration • Info about which client talks to server	• Loss of information • Loss of privacy	Encryption, Web proxies
Denial of Service	• Killing of user threads • Flooding machine with bogus requests • Filling up disk or memory • Isolating machine by DNS attacks	• Disruptive • Annoying • Prevent user from getting work done	Difficult to prevent
Authentication	• Impersonation of legitimate users • Data forgery	• Misrepresentation of user • Belief that false information is valid	Cryptographic techniques

Sockets Layer (SSL) and the follow-on Internet standard known as Transport Layer Security (TLS). At this level, there are two implementation choices. For full generality, SSL (or TLS) could be provided as part of the underlying protocol suite and therefore be transparent to applications. Alternatively, SSL can be embedded in specific packages. For example, Netscape and Microsoft Explorer browsers come equipped with SSL, and most Web servers have implemented the protocol.

Application-specific security services are embedded within the particular application. Figure 16.1c shows examples of this architecture. The advantage of this approach is that the service can be tailored to the specific needs of a given application.

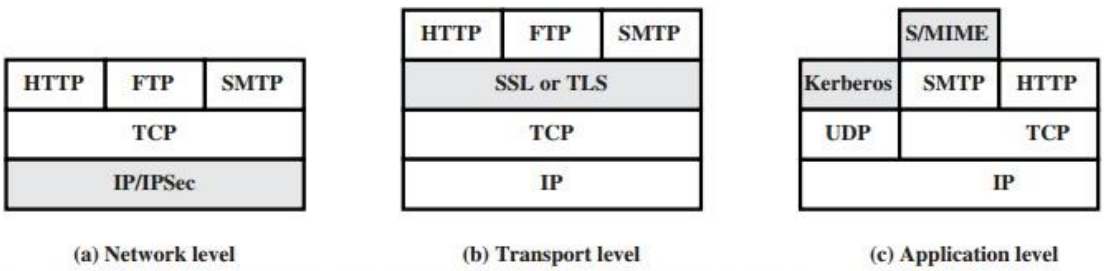


Figure 16.1 Relative Location of Security Facilities in the TCP/IP Protocol Stack

Secure Socket Layer (SSL) and Transport Layer Security:

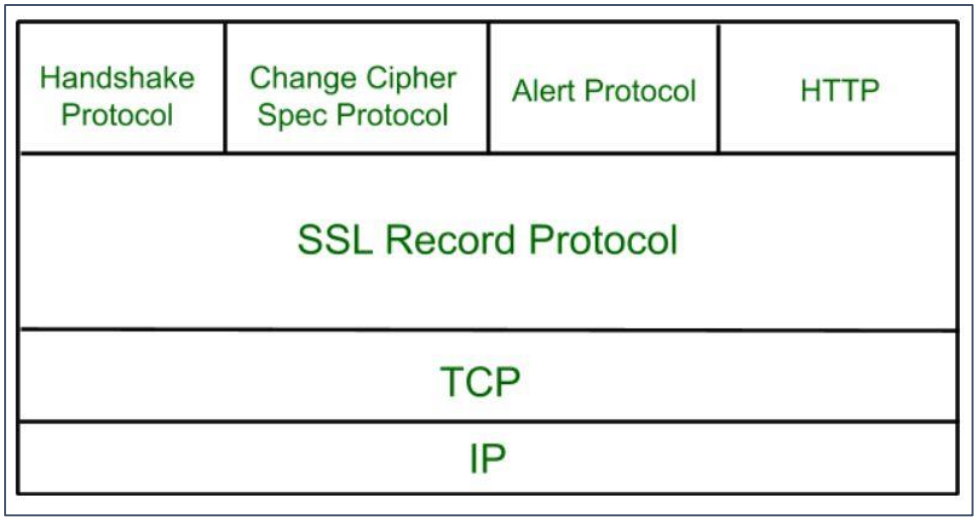
1. SSL Architecture:

Secure Socket Layer (SSL) provides security to the data that is transferred between web browser and server. SSL encrypts the link between a web server and a browser which ensures that all data passed between them remain private and free from attack.

Secure Socket Layer Protocols:

- SSL record protocol
- Handshake protocol
- Change-cipher spec protocol
- Alert protocol

SSL Protocol Stack:

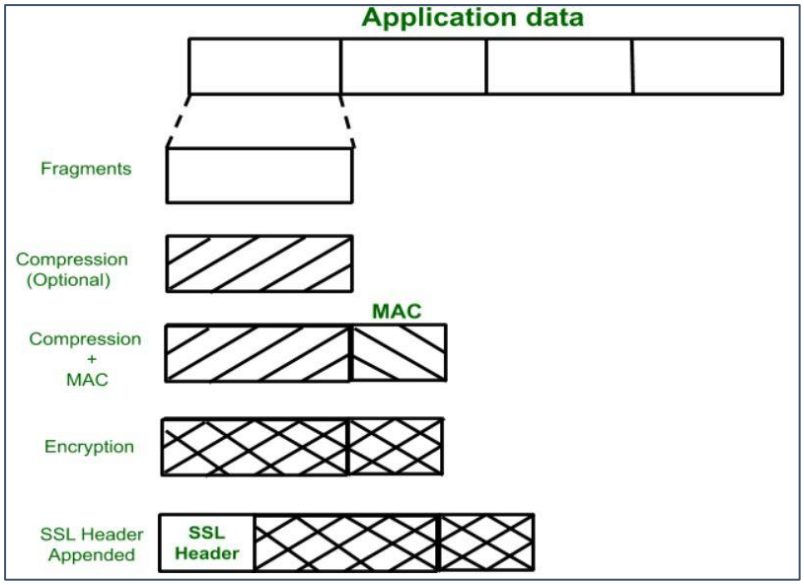


SSL Record Protocol:

SSL Record provides two services to SSL connection.

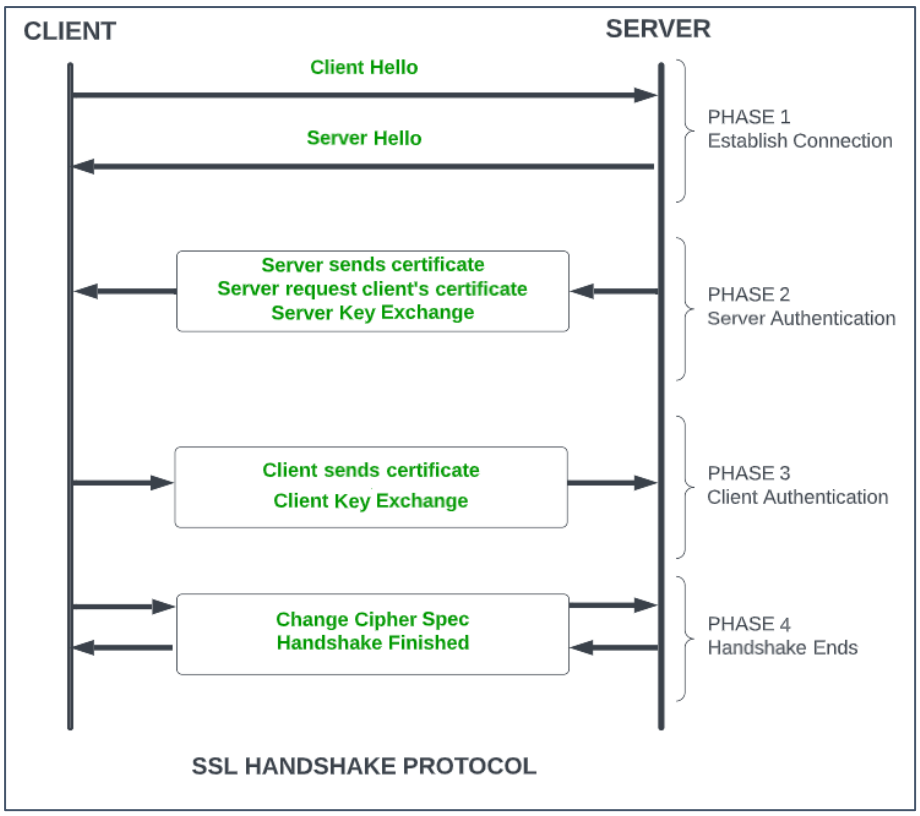
- Confidentiality
- Message Integrity

In the SSL Record Protocol application data is divided into fragments. The fragment is compressed and then encrypted MAC (Message Authentication Code) generated by algorithms like SHA (Secure Hash Protocol) and MD5 (Message Digest) is appended. After that encryption of the data is done and in last SSL header is appended to the data.



Handshake Protocol:
Handshake Protocol is used to establish sessions. This protocol allows the client and server to authenticate each other by sending a series of messages to each other. Handshake protocol uses four phases to complete its cycle.

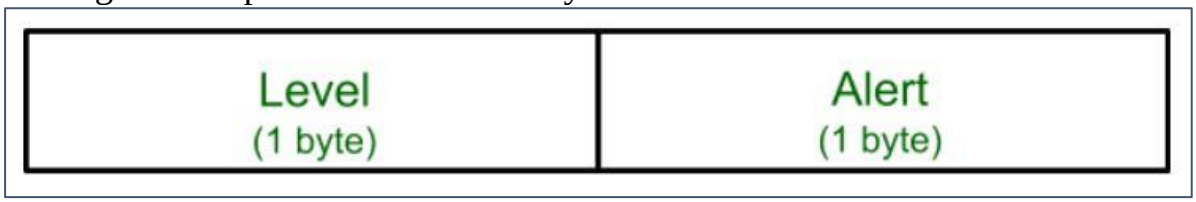
- **Phase-1:** In Phase-1 both Client and Server send hello-packets to each other. In this IP session, cipher suite and protocol version are exchanged for security purposes.
- **Phase-2:** Server sends his certificate and Server-key-exchange. The server end phase-2 by sending the Server-hello-end packet.
- **Phase-3:** In this phase, Client replies to the server by sending his certificate and Client-exchange-key.
- **Phase-4:** In Phase-4 Change-cipher suite occurs and after this the Handshake Protocol ends.



SSL Handshake Protocol Phases diagrammatic representation

Change-cipher Protocol:
This protocol uses the SSL record protocol. Unless Handshake Protocol is completed, the SSL record Output will be in a pending state. After the handshake protocol, the Pending state is converted into the current state. Change-cipher protocol consists of a single message which is 1 byte in length and can have only one value. This protocol's purpose is to cause the pending state to be copied into the current state.

Alert Protocol:
This protocol is used to convey SSL-related alerts to the peer entity. Each message in this protocol contains 2 bytes.



The level is further classified into two parts:

Warning (level = 1):
This Alert has no impact on the connection between sender and receiver. Some of them are:
Bad certificate: When the received certificate is corrupt.
No certificate: When an appropriate certificate is not available.
Certificate expired: When a certificate has expired.
Certificate unknown: When some other unspecified issue arose in processing the certificate, rendering it unacceptable.
Close notify: It notifies that the sender will no longer send any messages in the connection.
Unsupported certificate: The type of certificate received is not supported.
Certificate revoked: The certificate received is in revocation list.

Fatal Error (level = 2):
This Alert breaks the connection between sender and receiver. The connection will be stopped, cannot be resumed but can be restarted. Some of them are :
Handshake failure: When the sender is unable to negotiate an acceptable set of security parameters given the options available.
Decompression failure: When the decompression function receives improper input.
Illegal parameters: When a field is out of range or inconsistent with other fields.
Bad record MAC: When an incorrect MAC was received.
Unexpected message: When an inappropriate message is received.
The second byte in the Alert protocol describes the error.
Salient Features of Secure Socket Layer:

- The advantage of this approach is that the service can be tailored to the specific needs of the given application.
- Secure Socket Layer was originated by Netscape.
- SSL is designed to make use of TCP to provide reliable end-to-end secure service.
- This is a two-layered protocol.

Versions of SSL:
SSL 1 – Never released due to high insecurity.
SSL 2 – Released in 1995.
SSL 3 – Released in 1996.
TLS 1.0 – Released in 1999.
TLS 1.1 – Released in 2006.
TLS 1.2 – Released in 2008.
TLS 1.3 – Released in 2018.

SSL (Secure Sockets Layer) certificate is a digital certificate used to secure and verify the identity of a website or an online service. The certificate is issued by a trusted third-party called a Certificate Authority (CA), who verifies the identity of the website or service before issuing the certificate. The SSL certificate has several important characteristics that make it a reliable solution for securing online transactions:

1. **Encryption**: The SSL certificate uses encryption algorithms to secure the communication between the website or service and its users. This ensures that the sensitive information, such as login credentials and credit card information, is protected from being intercepted and read by unauthorized parties.
2. **Authentication**: The SSL certificate verifies the identity of the website or service, ensuring that users are communicating with the intended party and not with an impostor. This provides assurance to users that their information is being transmitted to a trusted entity.
3. **Integrity**: The SSL certificate uses message authentication codes (MACs) to detect any tampering with the data during transmission. This ensures that the data being transmitted is not modified in any way, preserving its integrity.
4. **Non-repudiation**: SSL certificates provide non-repudiation of data, meaning that the recipient of the data cannot deny having received it. This is important in situations where the authenticity of the information needs to be established, such as in e-commerce transactions.
5. **Public-key cryptography:** SSL certificates use public-key cryptography for secure key exchange between the client and server. This allows the client and server to securely exchange encryption keys, ensuring that the encrypted information can only be decrypted by the intended recipient.
6. **Session management**: SSL certificates allow for the management of secure sessions, allowing for the resumption of secure sessions after interruption. This helps to reduce the overhead of establishing a new secure connection each time a user accesses a website or service.
7. **Certificates issued by trusted CAs**: SSL certificates are issued by trusted CAs, who are responsible for verifying the identity of the website or service before issuing the certificate. This provides a high level of trust and assurance to users that the website or service they are communicating with is authentic and trustworthy.

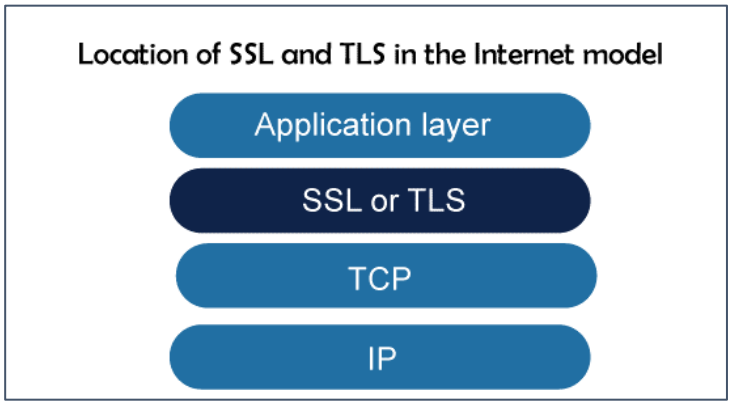
In addition to these key characteristics, SSL certificates also come in various levels of validation, including Domain Validation (DV), Organization Validation (OV), and Extended Validation (EV). The level of validation

determines the amount of information that is verified by the CA before issuing the certificate, with EV certificates providing the highest level of assurance and trust to users.

2. Transport Layer Security:

The application layer, which makes use of TCP (or SCTP) as a connection-oriented protocol, is actually secured by the transport layer, which also offers security for that layer. These apps' messages are first enclosed in security protocol packets before being contained in TCP. Due to the nature of security, which necessitates connection formation between the two entities, apps that employ UDP cannot take advantage of these security services. The transport-layer security does not apply to electronic mail (e-mail), which is another application. We need a particular security provision for this application because it only allows one-way communication between the sender and the receiver. This is covered in the below section.

The Secure Sockets Layer (SSL) and the Transport Layer Security (TLS) protocols are currently the two most used ones for delivering security at the transport layer. Actually, the latter is a modified version of the former by the IETF. TLS is extremely similar to SSL, which is covered in this section. The location of TLS and SSL in the Internet paradigm is depicted in the image below:



These protocols aim to guarantee data secrecy, data integrity, and server and client authentication, among other things. Applications-layer client/server programs that employ TCP services, like HTTP, can encapsulate their data in SSL packets (HTTPS). In order to allow HTTP messages to be encased in SSL (or TLS) packets, the client can use the URL https://... rather than http://... if the server and client are both capable of executing SSL (or TLS) programs. For example, banking information can be safely exchanged over the Internet for online buyers.

HTTPS

What is HTTPS?

HTTPS stands for **Hyper Text Transfer Protocol Secure**. It is the most common protocol for sending data between a web browser and a website. It is the secure variant of HTTP used for communication between the browser and the webserver. In order to make the data transfer more secure, it is encrypted. Encryption is required to ensure security while transmitting sensitive information like passwords, contact information, etc.

How does HTTPS work?

HTTPS establishes the communication between the browser and the webserver. It uses the **Secure Socket Layer** (SSL) and **Transport Layer Security** (TLS) protocol for establishing communication. The new version of SSL is TSL.

HTTPS uses the conventional HTTP protocol and adds a layer of SSL/TSL over it. The workflow of HTTP and HTTPS remains the same, the browsers and servers still communicate with each other using the HTTP protocol. However, this is done over a secure SSL connection. The SSL connection is responsible for the encryption and decryption of the data that is being exchanged in order to ensure data safety.

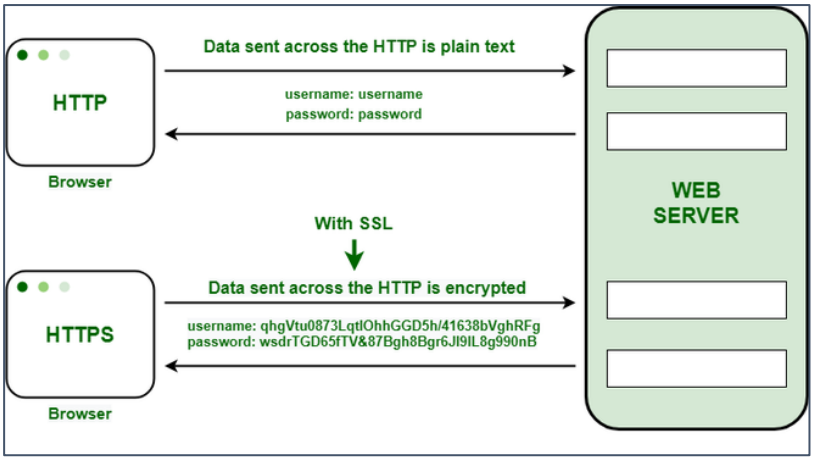
Secure Socket Layer (SSL)

The main responsibility of SSL is to ensure that the data transfer between the communicating systems is **secure and reliable**. It is the standard security technology that is used for encryption and decryption of data during the transmission of requests.

As discussed earlier, HTTPS is basically the same old HTTP but with SSL. For establishing a secure communication link between the communicating devices, SSL uses a digital certificate called **SSL certificate**.

There are two major roles of the SSL layer –

- Ensuring that the browser communicates with the required server directly.
- Ensuring that only the communicating systems have access to the messages they exchange.



HTTP transfers data in a hypertext format between the browser and the web server, whereas HTTPS transfers data in an encrypted format.

As a result, HTTPS protects websites from having their information broadcast in a way that anyone eavesdropping on the network can easily see.

During the transit between the browser and the web server, HTTPS protects the data from being accessed and altered by hackers.

Even if the transmission is intercepted, hackers will be unable to use it because the message is encrypted.

It uses an asymmetric public key infrastructure for securing a communication link. There are two different kinds of keys used for encryption –

1. **Private Key**: It is used for the decryption of the data that has been encrypted by the public key. It resides on the server-side and is controlled by the owner of the website. It is private in nature.
2. **Public Key**: It is public in nature and is accessible to all the users who communicate with the server. The private key is used for the decryption of the data that has been encrypted by the public key.

Advantage of HTTPS

1. **Secure Communication:** HTTPS establishes a secure communication link between the communicating system by providing encryption during transmission.

2. **Data Integrity:** By encrypting the data, HTTPS ensures data integrity. This implies that even if the data is compromised at any point, the hackers won't be able to read or modify the data being exchanged.

3. **Privacy and Security:** HTTPS prevents attackers from accessing the data being exchanged passively, thereby protecting the privacy and security of the users.

4. **Faster Performance:** TTPS encrypts the data and reduces its size. Smaller size accounts for faster data transmission in the case of HTTPS.

Secure Shell:

SSH stands for Secure Shell or Secure Socket Shell. It is a cryptographic network protocol that allows two computers to communicate and share the data over an insecure network such as the internet. It is used to login to a remote server to execute commands and data transfer from one machine to another machine.

The SSH protocol was developed by SSH communication security Ltd to safely communicate with the remote machine.

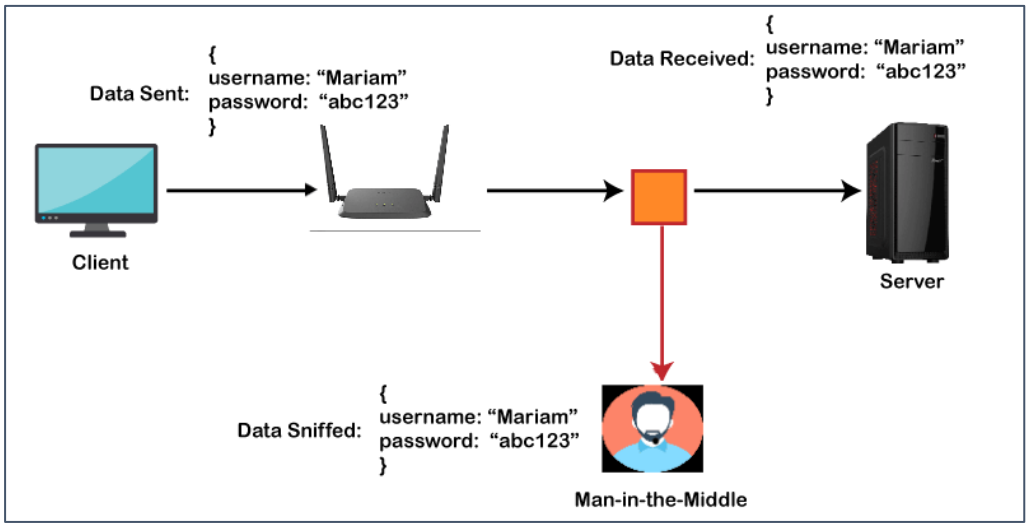
Secure communication provides a strong password authentication and encrypted communication with a public key over an insecure channel. It is used to replace unprotected remote login protocols such as Telnet, rlogin, rsh, etc., and insecure file transfer protocol FTP.

Its security features are widely used by network administrators for managing systems and applications remotely.

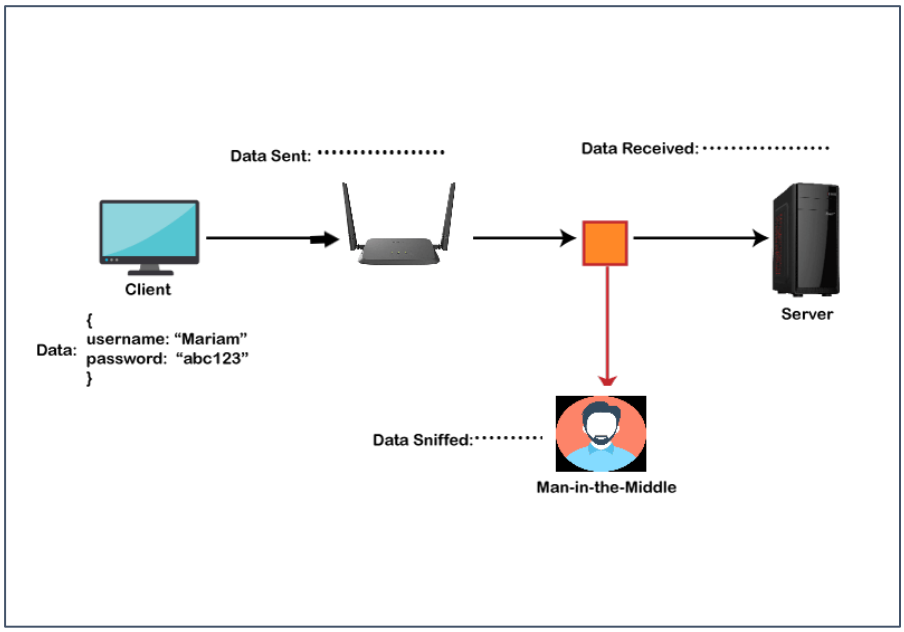
The SSH protocol protects the network from various attacks such as DNS spoofing, IP source routing, and IP spoofing.

A simple example can be understood, such as suppose you want to transfer a package to one of your friends. Without SSH protocol, it can be opened and read by anyone. But if you will send it using SSH protocol, it will be encrypted and secured with the public keys, and only the receiver can open it.

Before SSH:



After SSH:



Usages of SSH protocol:

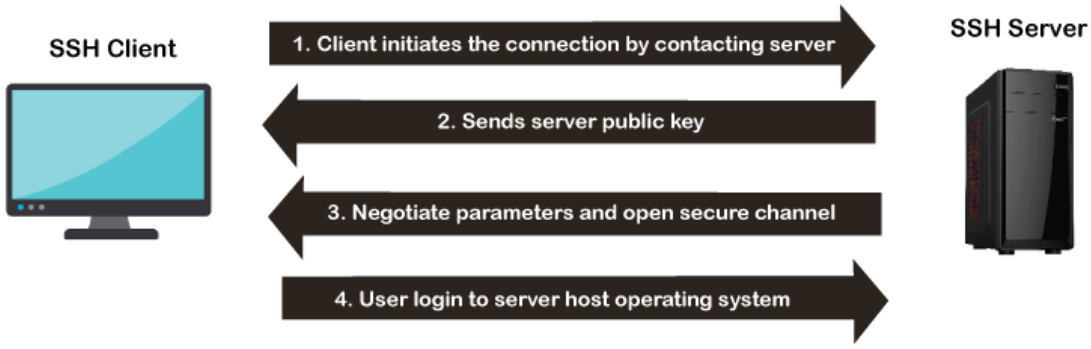
The popular usages of SSH protocol are given below:

- It provides secure access to users and automated processes.
- It is an easy and secure way to transfer files from one system to another over an insecure network.
- It also issues remote commands to the users.
- It helps the users to manage the network infrastructure and other critical system components.
- It is used to log in to shell on a remote system (Host), which replaces Telnet and rlogin and is used to execute a single command on the host, which replaces rsh.
- It combines with rsync utility to backup, copy, and mirror files with complete security and efficiency.
- It can be used for forwarding a port.
- By using SSH, we can set up the automatic login to a remote server such as OpenSSH.
- We can securely browse the web through the encrypted proxy connection with the SSH client, supporting the SOCKS protocol.

How does SSH Works?

The SSH protocol works in a client-server model, which means it connects a secure shell client application (End where the session is displayed) with the SSH server (End where session executes).

As discussed above, it was initially developed to replace insecure login protocols such as Telnet, rlogin, and hence it performs the same function.



The basic use of SSH is to connect a remote system for a terminal session and to do this, following command is used:

1. ssh UserName@SSHserver.test.com

The above command enables the client to connect to the server, named server.test.com, using the ID UserName.

If we are connecting for the first time, it will prompt the remote host's public key fingerprint and ask to connect. The below message will be prompt:

1. The authenticity of host 'sample.ssh.com' cannot be established.
2. DSA key fingerprint is 01:23:45:67:89:ab:cd:ef:ff:fe:dc:ba:98:76:54:32:10.
3. Are you sure you want to continue connecting (yes/no)?

To continue the session, we need to click yes, else no. If we click yes, then the host key will be stored in the known_hosts file of the local system. The key is contained within the hidden file by default, which is /.ssh/known_hosts in the home directory. Once the host key is stored in this hidden file, there is no need for further approval as the host key will automatically authenticate the connection.

The architecture of SSH Protocol:

The SSH architecture is made-up of three well-separated layers. These layers are:

1. Transport Layer
2. User-authentication layer
3. Connection Layer

The SSH protocol architecture is an open architecture; hence it provides great flexibility and enables SSH use for many other purposes instead of only a secure shell. In the architecture, the transport layer is similar to the transport layer security (TLS). The User-authentication layer can be used with the custom authentication methods, and the connection layer allows multiplexing different secondary sessions into a single SSH connection.

Transport Layer:

The transport layer is the top layer of the TCP/IP protocol suite. For SSH-2, this layer is responsible for handling initial key exchange, server authentication, set up encryption, compression, and integrity verification. It works as an interface for sending and receiving plaintext packets with sizes up to 32, 768bytes.

User authentication Layer:

As its name suggests, the user authentication layer is responsible for handling client authentication and provides various authentication methods. The authentication is done at the client-side; hence when a prompt occurs for a password, it usually for an SSH client rather than a server, and the server responds to these authentications.

This layer includes various methods of authentication; these methods are:

- **Password:** Password authentication is a straightforward way of authentication. It includes the feature to change the password for easy access. But it is not used by all the applications.
- **Public-key:** The public-key is a public key-based authentication method, which supports DSA, ECDSA, or RSA keypairs.
- **Keyboard-interactive:** It is one of the versatile authentication methods. In this, the server sends a prompt to enter information & the client sends it back with keyed-in responses by the user. It is used to provide a one-time password or OTP authentication.

- **GSSAPI:** In this method, the authentication is performed by external methods such as Kerberos 5 or NTLM, which provide the single sign-on capability to SSH sessions.

Connection Layer:

The connection layer defines various channels through which SSH services are provided. It defines the concept of channels, channel requests, and global requests. One SSH connection can host different channels simultaneously and can also transfer data in both directions simultaneously. Channel requests are used in the connection layer to relay out-of-band channel-specific data, for example, the altered size of a terminal window or the exit code of a server-side process. The standard channel types of connection layer are:

- **shell:** It is used for terminal shells, SFTP, and exec requests.
- **direct-tcpip:** It is used for the client-to-server forwarded connections.
- **forwarded-tcpip:** It is used for the server-to-client forwarded connections.

What can be transferred with SSH protocol?

The SSH protocol can transfer the following:

- Data
- Text
- Commands
- Files

The files are transferred using the SFTP(Secure file transfer protocol), the encrypted version of FTP that provides security to prevent any threat.

Difference between SSH and Telnet:

- Telnet was the first internet application protocol used to create and maintain a terminal session on a remote host.
- Both SSH and Telnet have the same functionality. Still, the main difference is that SSH protocol is secured with public-key cryptography that authenticates endpoint while setting up a terminal session. On the other hand, no authentication is provided in Telnet for the user's authentication, making it less secure.
- SSH sends the encrypted data, while Telnet sends data in plain text.
- Due to high security, SSH is the preferred protocol for public networks, while due to less security, Telnet is suitable for private networks.

- SSH runs on port no 22 by default, but it can be changed, while Telnet uses port number 23, specifically designed for the Local area network.

SSH Encryption Techniques:

To make a secure transmission, SSH uses three different encryption techniques at various points during a transmission. These techniques are:

1. Symmetrical Encryption
2. Asymmetrical Encryption
3. Hashing

Symmetrical Encryption:

Only one key can be used in symmetric encryption techniques to encrypt & decrypt messages sent and received from the destination. This technique is also known as shared key encryption because both devices use the same key to encrypt the data they send and decrypt the received data.

This technique encrypts the entire SSH connection to prevent man-in-middle attacks. In this technique, one issue arises at the time of initial key exchange. As per this problem, if a third party is present during the key exchange, they could know the key and read the entire message.

The Key exchange algorithm is used to prevent this problem. With this algorithm, the secret keys can be securely exchanged without an interception.

Asymmetrical encryption is required to implement the key exchange algorithm.

Asymmetrical Encryption:

In asymmetrical encryption, two different keys are used for encryption and decryption, private and public keys. The private key is private to the user only and cannot be shared with any other user, whereas the public key is shared publicly. The public key is saved on the SSH server, whereas the private key is saved locally on the SSH client; these two keys form a key pair. The message encrypted with the public key can only decrypt with the corresponding private key.

It is a much secure technique as if a third party gets the public key, and they cannot decrypt the message because they don't know the private key.

The asymmetrical encryption does not encrypt the complete SSH session. Instead, it is mainly used for the key exchange algorithm of symmetric encryption. In this,

before establishing a connection, both systems (client and server) generate public-private key pairs temporarily and then share their private keys to generate the shared secret key.

After establishing a secure symmetric connection, the server uses the public key to transmit it to the client for authentication. The client can only decrypt the data if it has the private key, and hence the SSH session establishes.

Hashing:

In SSH, one-way hashing is used as the encryption technique, which is another form of cryptography. The hashing technique is different from the above two methods, as it is not meant by decryption. It generates the signature or summary of information. SSH uses HMAC(Hash-based Message authentication) to ensure that messages are reached in complete and unmodified form.

In this technique, each transmitted message must have a MAC, which uses three components: symmetric key, packet sequence number, and message content. These three components form the hash function that generates a string that doesn't have any meaning, and this string is sent to the host. The host also has the same information, so they also generate a hash function, and if the generated hash matches with the received hash, it means the message is not tempered.

Wireless Security:

Wireless Network provides various comfort to end users but actually they are very complex in their working. There are many protocols and technologies working behind to provide a stable connection to users. Data packets traveling through wire provide a sense of security to users as data traveling through wire probably not heard by eavesdroppers.

To secure the wireless connection, we should **focus on** the following areas:

- Identify endpoint of wireless network and end-users i.e., Authentication.
- Protecting wireless data packets from middleman i.e., Privacy.
- Keeping the wireless data packets intact i.e., Integrity.

We know that wireless clients form an association with Access Points (AP) and transmit data back and forth over the air. As long as all wireless devices follow 802.11 standards, they all coexist. But all wireless devices are not friendly and trustworthy, some rogue devices may be a threat to wireless security. Rogue devices can steal our important data or can cause the unavailability of the network.

Wireless security is **ensured by** following methods:

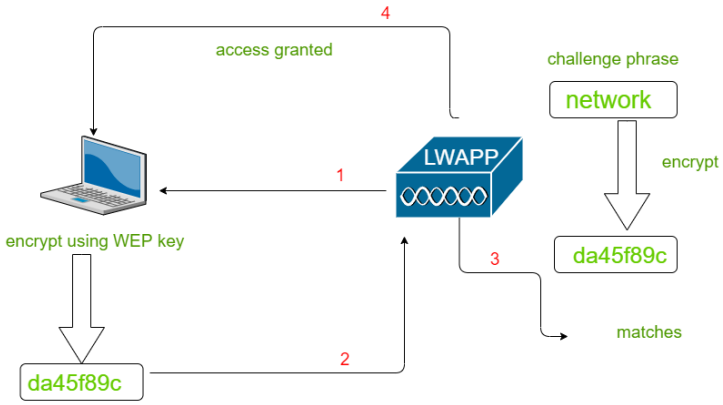
- Authentication
- Privacy and Integrity

In this article, we talk about Authentication. There are broadly two types of Authentication process: Wired Equivalent Privacy (WEP), and Extensible Authentication Protocol (802.1x/EAP).

These are explained as following below.

1. Wired Equivalent Privacy (WEP) :
For wireless data transmitting over the air, open authentication provides no security uses the RC4 cipher algorithm for making every frame encrypted. The RC4 cipher also encrypts data at the sender side and decrypt data at the receiving site, using a string of bits as key called WEP key.
WEP key can be used as an authentication method or encryption tool. A client can associate with AP only if it has the correct WEP key. AP tests the knowledge of the WEP key by using a challenge phrase. The client encrypts the phrase with his own key and send back to AP. AP compares the received

encrypted frame with his own encrypted phrase. If both matches, access to the association is granted.



Working of WEP Authentication

2. Extensible Authentication Protocol (802.1x/EAP) :
In WEP authentication, authentication of the wireless clients takes place locally at AP. But Scenario gets changed with 802.1x. A dedicated authentication server is added to the infrastructure. There is the participation of three devices –

1. **Supplicant –**
   Device requesting access.
2. **Authenticator –**
   Device that provides access to network usually a Wlan controller (WLC).
3. **Authentication Server –**
   Device that takes client credentials and deny or grant access.

Supplicant WLC Authentication sever

EAP is further of four types with some amendments over each other –

- LEAP
- EAP-FAST
- PEAP
- EAP-TLS

Unit-5

Email Security:

What is Email Security?

Email security is the process of ensuring the availability, integrity and authenticity of email communications by protecting against the risk of email threats.

Email enables billions of connected people and organizations to communicate with one another to send messages. Email is at the foundation of how the internet is used, and it has long been a target for attacks.

Since the earliest days of email, it has been abused and misused in different ways with no shortage of email threats. Abuse of email includes the following:

- Phishing Attempts
- Spoofing
- Spam Phishing
- Malware Delivery
- Business Email Compromise (Bec)
- Denial Of Service (Dos) Attacks

Email security aims to help prevent attacks and abuse of email communication systems. Within the domain of email security, there are various email security protocols that technology standards organizations have proposed and recommended for implementation to help limit email risks. Protocols can be implemented by email clients and email servers, such as Microsoft Exchange and Microsoft 365, to help ensure the secure transit of email. Looking beyond just protocols, secure email gateways can help organizations and individuals to protect email from various threats.

How secure is Email?

By default, email is not secure for a variety of different reasons.

The original implementation of email protocols, including Simple Mail Transfer Protocol, Internet Message Access Protocol and Post Office Protocol 3, did not mandate the use of secure transport mechanisms, such as Secure Sockets Layer (SSL) and Transport Layer Security (TLS). As such, connections to and from an email server were not done over an encrypted tunnel, which means that an intercepted message could have potentially been read by anyone.

Adding further complications, email messages are often stored on email servers in an unencrypted format. System administrators with access to an unencrypted email server could potentially gain access to read any email. A user email account can only be as secure as the server on which the email is stored.

The user side of email is another area that demonstrates its inherent insecurity. Access to user email accounts is commonly secured only by a username and password, which is often insufficient to deal with modern email threats. With the volume of data breaches continuing to grow year after year, an increasing number of email credentials have been leaked to public sites. Attackers can sometimes simply find user credentials for email services from public data breaches. Brute-force and password-guessing attacks are also a risk of the username/password approach to email access.

Email insecurity also comes from a lack of guaranteed authenticity. It is possible and common for attackers to spoof an email address and make it appear as though a fake email has come from a legitimate address. The lack of email authenticity is a common tactic used in phishing attempts, as well as spam phishing attacks that cast a wider net to attract unsuspecting victims to click.

Why is Email Security Important?

Email is used for business communications and is often a foundational element of an organization's IT operations and ability to communicate both inside and outside of the company.

A risk to email, such as a lack of access due to a DoS attack, can potentially restrict the ability of a business to conduct business. Spam, which is another key email threat, can have negative impacts on a business, including filling up inboxes with useless information and potentially leading to phishing attacks.

Email can also often include sensitive data that is intended only for the recipient of an email message. Without email security, the sensitive information could be leaked to an unauthorized entity.

Authenticity of corporate email also highlights the importance of email security. If an unauthorized individual is able to send email that seemingly comes from a corporate email account, it could lead to fraud as part of a BEC attack.

What are the benefits of Email Security for businesses?

As most organizations continue to rely on email for business operations, email security technologies and best practices provide several critical benefits for business of all sizes, including the following:

- **Availability.** At the most basic level, email security can help to ensure the continued availability of email services so a business can continue to communicate with its employees and customers.
- **Authenticity.** Having email authenticity measures in place can help to build trust for an organization and its users that email coming from its domain is authentic.
- **Fraud prevention.** The ability to identify potential email security risks, such as spoofing, can potentially help an organization to reduce the opportunity for fraud.
- **Malware prevention.** An appropriate set of security capabilities in place on an email platform can limit risks of malware transmitted by email.
- **Phishing protection.** Phishing attacks can trick employees of a business to click on links or download things that could be harmful and lead to information disclosure and credential theft.

Email Security Best Practices:

While email is not secure by default, there are proactive best practices that individuals and organizations can take to significantly improve email security, including the following:

- **Enforce encrypted connections.** All connections to and from an email platform should occur over an SSL/TLS connection that encrypts the data as it transits the public internet.
- **Encrypt email.** While perhaps not an ideal option for every user at every organization, encrypting email messages provides an additional layer of privacy that can help to protect against unauthorized information disclosure.
- **Create strong passwords.** For users, it is important that any passwords are complex and not easy to guess. It's often recommended that users have passwords with a combination of letter, numbers and symbols.
- **Implement 2FA or MFA.** While strong passwords are helpful, they often aren't enough. Implementing two-factor authentication (2FA) or multifactor authentication (MFA) provides an additional layer of access control that can help to improve email security.
- **Train on anti-phishing.** Phishing is a common email threat. It's important to train users to avoid risky behaviours and spot phishing attacks that get through to their inbox.

- **Use domain authentication.** The use of domain authentication protocols and techniques, including domain-based message authentication, reporting and conformance, can help to reduce the risk of domain spoofing.

Email Security Tools:

Best practices alone are not typically enough to help guarantee email security and reduce the risk from threats. Email security tools and services can help organizations with managing and improving security posture. Examples of these tools include the following:

- Integrated online email service provider platforms. Microsoft Exchange Online is part of the Microsoft 365 Business Standard suite and provides an integrated set of email security capabilities for users. Similarly, Google Workspace offers an enterprise supported version of Gmail that integrates email security as part of the online service. Both Microsoft and Google services provide integrated antimalware and antispam capabilities, as well options for encrypted data transit.
- Email security gateways. For organizations that have on-premises email systems and cloud-hosted email, an email security gateway can provide an inspection point for malware, spam and phishing attempts. Email security gateways are available from multiple vendors, including Barracuda, Cisco, Forcepoint, Fortinet, Mimecast, Proofpoint and Sophos.

Pretty Good Privacy (PGP):

What is Pretty Good Privacy (PGP)?

Pretty Good Privacy or PGP was a popular program used to encrypt and decrypt email over the internet, as well as authenticate messages with digital signatures and encrypted stored files. PGP now commonly refers to any encryption program or application that implements the OpenPGP public key cryptography standard.

PGP was initially brought out as freeware and later as a low-cost commercial product. First published by Philip R. Zimmermann in 1991, it was once the most used privacy program and a de facto email encryption standard.

The original freeware and commercial versions of PGP are no longer available. Ownership of the program shifted several times before its eventual demise. Zimmerman originally owned PGP, and later PGP Inc., the company he founded to market PGP, took over ownership. Network Associates Inc. (NAI) acquired PGP Inc. in 1997. Other companies that have marketed some or all of the PGP technologies include the following:

- Broadcom
- Intel
- McAfee Associates
- PGP Corp.
- Symantec
- Townsend Security

While it may no longer be simple to acquire a new copy of the original PGP program, the Internet Engineering Task Force (IETF) has published PGP protocols as internet standards since 1996. Both open source and commercial implementations of the OpenPGP protocol are widely available. The GNU Privacy Guard (GPG) implementation is published under the GNU Public License (GPL).

In 2015, it was reported that Zimmermann no longer used PGP because working versions were not available for any of his devices.

The Pretty Good Privacy trademark was abandoned as of April 2020. Implementations of the OpenPGP specification now often refer to their implementations of the protocol as Pretty Good Privacy or simply *PGP*.

How does PGP Work?

Pretty Good Privacy uses a variation of the public key system. In this system, each user has an encryption key that is publicly known and a secret or private key that is known only to that user. Users encrypt a message they send to someone else using that person's public PGP key. When the recipient receives the message, they decrypt it using their private key.

Encrypting an entire message using public key encryption can consume excessive amounts of resources. As a result, PGP uses a symmetric key encryption algorithm to encrypt the message and then uses the public key to encrypt that symmetric encryption key. Both the encrypted message and the encrypted symmetric encryption key are sent to the recipient, who first uses their private key to decrypt the short key and then uses that key to decrypt the message.

The original PGP program was offered in two versions, one using the Rivest-Shamir-Adleman (RSA) algorithm for key exchange, and one using the Diffie-Hellman algorithm for key exchange. PGP was required to pay a license fee to RSA for the RSA version. That version used the International Data Encryption Algorithm to generate a short key for the entire message and RSA to encrypt the short key. The Diffie-Hellman version used the CAST algorithm for the short key to encrypt the message and the Diffie-Hellman algorithm to encrypt the short key.

When sending digital signatures, PGP uses an efficient algorithm that generates a hash (a mathematical summary) from the user's name and other signature information. This hash code is then encrypted with the sender's private key. The receiver uses the sender's public key to decrypt the hash code. If it matches the hash code sent as the digital signature for the message, the receiver is sure that the message has arrived securely from the stated sender. PGP's RSA version used the MD5 algorithm to generate the hash code. PGP's Diffie-Hellman version used the SHA-1 algorithm to generate the hash code; neither of those hashing algorithms is considered secure today.

How to get PGP?

To get a Pretty Good Privacy program, users must download or buy it and install it on their computer system. It typically contains a user interface that works with the user's email program. The public key that the PGP program provides must be registered with a PGP public-key server so that people exchanging messages with the user will be able to find it.

PGP software that supports the OpenPGP protocol can be a standalone application like GPG, or it may be a front-end
interface, applet or plugin implementing the protocol. In most cases, PGP software is packaged as part of an email client or web browser.

PGP Concepts:

PGP depends on some concepts that enable users to easily access and share public keys, and to transmit cryptographic information across networks and systems. Important terms include the following:

- Alice and Bob are names assigned to generic actors in cryptographic processes. Alice, Bob and other generic actor names are often used when illustrating cryptographic exchanges like those used by PGP.
- Web of trust is a concept used to describe the way trust is established in public keys. A PGP user can try to establish trust directly with every key holder they interact with. In those cases where trust is established, they may also be willing to sign those keys to signify that they
have authenticated the key pair and its holder. The PGP user can also accept trust in key holders that certain other PGP users have already signed to indicate they are trustworthy. If Alice accepts that Bob is sufficiently trustworthy in how carefully he vets the public keys he accepts as authenticated, then Alice can also trust those other public keys that Bob trusts.
- Implicit trust is one of the two different types of trust that can be established through the web of trust. Implicit trust is used when Alice signs Bob's public key pair. This indicates that Alice has vetted Bob -- and his private and public keys and his email address -- and is willing to assert (through her own signature) that she found Bob to be who he says he is and that the email and key pair are under Bob's control.
- Explicit trust is the other type of trust established through the web of trust. It occurs when Carlos, a third generic user, is willing to trust Alice's judgement about other individuals whose keys she has signed. Carlos can use explicit trust in Alice to accept that Bob's public key pair is also valid.
- Key signing is the PGP function that enables one person to announce that they have verified the person who claims to own the public key pair. PGP creator Zimmermann stresses verifying the following:
- The key you are signing should be verified as controlled by the person who claims it.
- The identity of the key holder should be verified with at least one form of photo ID. Even friends or coworkers should be formally identified if you have never previously seen that person's ID.

- Email and private key ownership should be verified. The email address in the signed key should be verified as the correct one for the person claiming the key pair.
- ASCII armor, also known as Radix-64 encoding, is a way of formatting encrypted data in a printable format. PGP uses ASCII armor to format data in a way that resists the introduction of errors through different computer formats as the data transits the internet. ASCII armor uses only ASCII characters and header and footer blocks to identify the start and finish of the armored data.
- Session key is a symmetric encryption key used for just one encryption session.

What is PGP used for?

There are two main reasons to use PGP.

1. **Encryption**: PGP enables encryption of sensitive information or data whether it is a file, email or message. A PGP user can secure data through encryption, in a format that is easily transmitted but that can only be decrypted with the recipient's secret key.
2. **Authentication**: PGP enables digital signing of a message, file or email whether encrypted or not. The recipient uses the signer's public key to authenticate the digital signature.

More specifically, PGP software enables users to do all basic PGP transactions, including the following functions:

- Creating a PGP public key pair;
- Revoking a PGP public key pair, so that others will no longer use it;
- Key server functions, like specifying a default key server and registering key pairs;
- Encrypting a message or file;
- Decrypting a message or file;
- Digitally signing a message or file;
- Authenticating a digital signature;
- Signing a public key; and
- Key management.

Different OpenPGP implementations have different but similar processes for each of these functions.

PGP is used mostly to encrypt or digitally sign emails, though it can also be used to do the following:

- Encrypt and digitally sign transmissions in messaging applications. PGP has been implemented as an applet or an add-on to messaging applications. The basic GPG implementation operates at the command line, but numerous projects and some products act as a graphical user interface (GUI) front end for GPG.
- Encrypt and digitally sign disk drives. Depending on the OS, PGP-based applications are available for encrypting disk volumes.
- Scripts and application programming interfaces (API)s for programming with PGP. Developers can use scripts of cryptographic processes. Many of these common but complicated scripts are available online. Users can also develop their own scripts or use APIs to integrate PGP support into their customized applications.

PGP's Challenges:

PGP's success was largely a result of offering early users access to strong cryptography with little or no investment in software licenses. However, implementing and using PGP can be challenging for the following reasons:

- Usability. PGP implementations tend to be difficult to use, whether at the command line or in a GUI.
- Conceptual complexity. New users often have difficulty understanding key PGP concepts and processes.
- Decentralized infrastructure. Using a web of trust can pose a problem when there are not enough participants in the larger general population.

While most users do not use PGP, there is still enough of a user base to fuel continued development of OpenPGP-compliant implementations and related applications.

Secure/Multipurpose Internet Mail Extensions (S/MIME):

The S/MIME certificate's nitty-gritty will assist you in strengthening your critical security concerns in the mail while also advancing your commercial goals. Continue reading to learn more.

Over the last two decades, business and official interactions have shifted from phone conversations to emails. Because email is the most used mode of communication, according to Statista, 4.03 billion people will use email in 2021, and that number is expected to climb to 4.48 billion by 2024.

Every day, emails are sent and received across devices, necessitating the need to secure these interactions. Because of the amount and type of sensitive data in a commercial firm, this criticality is increased. Assume you work in a field where sensitive data is handled.

- Intellectual property is something that belongs to you.
- Personal information about employees
- Customer information and contact information
- Card information (credit and debit)

If this is the case, consider safeguarding your emails and safeguarding sensitive information. Apart from preventing anyone from reading your emails, you must also protect your data from fraudsters. These individuals are well-known for utilizing your email and concocting phishing schemes to dupe people into handing over personal information.

What Exactly is S/MIME?

Secure/Multipurpose Internet Mail Extension (S/MIME) is an industry-standard for email encryption and signature that is commonly used by businesses to improve email security. S/MIME is supported by the majority of corporate email clients.

S/MIME encrypts and digitally signs emails to verify that they are verified and that their contents have not been tampered with.

How Does S/MIME Address Email Security Problems?

An S/MIME certificate is an end-to-end encryption solution for MIME data, a.k.a. email communications, as shown in the preceding sections. The use of asymmetric cryptography by S/MIME certificates prevents the message's integrity from being compromised by a third party. In basic English, a digital signature is used to hash the message. The mail is then encrypted to protect the message's secrecy.

S/MIME employs public encryption to protect communications that can only be decoded with the corresponding private key obtained by the authorized mail

receiver, according to GlobalSign, a company that provides specialized Public Key Infrastructure (PKI) solutions to businesses.

Stepping back in time allows us to visualize the situation. Wax seals on letters served as a unique identifying proof of the sender while also assisting the recipient in determining whether the letters had been tampered with. S/MIME certificates work on a similar principle.

The sender can use a private key to digitally sign the letter he is sending. The email is then accompanied by a public key while in transit. The recipient will use it to verify the sender's digital signature and decode the message using his own private key. Using 'asymmetric cryptography,' this system uses two separate but mathematically comparable cryptographic keys to provide end-to-end encryption. The completely encrypted contents of the email will be nearly hard to crack without both keys.

S/MIME Certificate Characteristics

You receive a slew of cryptographic security features when you use an S/MIME certificate for email apps.

- **Authentication** − It refers to the verification of a computer user's or a website's identity.
- **Message consistency** − This is a guarantee that the message's contents and data have not been tampered with. The message's secrecy is crucial. The decryption procedure entails checking the message's original contents and guaranteeing that they have not been altered.
- **Use of digital signatures that invoke non-repudiation** − This is a circumstance in which the original sender's identity and digital signatures are validated so that there is no doubt about it.
- **Protection of personal information** − A data breach cannot be caused by an unintentional third party.
- **Encryption is used to protect data** − It relates to the procedures described above, in which data security is ensured by a mix of public and private keys representing asymmetric cryptography.

The MIME type is designated by a S/MIME certificate. The enclosed data is referred to by the MIME type. The MIME entity is completely prepared, encrypted, and packaged inside a digital envelope.

Support for S/MIME

Some of the most popular email programs that support S/MIME are listed below.

- iPhone iOS Mail
- Apple Mail
- Gmail IBM Notes

- Mozilla Thunderbird Mail Mate Microsoft Outlook or Outlook on the Web
- Cipher Mail

Although an S/MIME certificate has been around for a long time and is supported by most email clients, the disadvantages of using it include complicated implementation owing to the public and private keys of the sender and receiver. As a result, it was restricted to highly classified government communications and those started by techies.

The adoption trend has improved, thanks to the advent of automated solutions for deploying and managing S/MIME certificates. The benefits of using S/MIME certificates to safeguard data in transit and, at rest, have surpassed the disadvantages.

What is the Best Way to Send Encrypted Emails?

Secure email service providers are used by certain companies and individuals to send secure emails. These services, such as Proton Mail, may allow you to send and receive private messages for free, but the disadvantage is that both the sender and the recipient must have the same account. This is a common disadvantage of end-to-end encryption services.

Aside from this issue, there is a far more serious one that limits the usability of email services for businesses. These ostensibly safe email service companies are nonetheless vulnerable to cyber-attacks. VFEMail is a classic example of a secure email service provider that, after 20 years of operation, fell to a cyber-attack.

A method is to use a S/MIME certificate to digitally sign and send encrypted emails. This technology is classified as secure public-key encryption by the Internet Engineering Task Force (IETF), and it is also suggested by the National Institute of Standards and Technology (NIST) as a "protocol for email end-to-end authentication and secrecy".

Features	PGP	S/MIME
Full form	PGP is an abbreviation for Pretty Good Privacy.	S/MIME is an abbreviation for Secure/Multipurpose Internet Mail Extension.
Effectively process	It is made to process emails in plain text.	It permits emails that also contain multimedia assets.
Cost	It is less costly than S/MIME.	It is more expensive than PGP.
Dependency	It relies on the user key exchange.	It relies on a hierarchically valid certificate for key exchange.
Usage	It is useful for both personal and organizational purposes.	It is suitable for usage in the industry.
Efficient	It is less efficient.	It is more efficient.
Convenient	It is less convenient.	It is more convenient because all applications are securely transformed.
Public Keys	It has 4096 public keys.	It has only 1024 public keys.
Encryption	It is the standard for secure encryption.	It is a robust encryption standard with some limitations.
Digital Signature	It utilizes Diffie hellman's digital signature.	It utilizes Elgamal's digital signature.
VPN	It may be utilized in VPNs.	It is utilized with email services, not VPNs.

IP Security:

IP Sec (Internet Protocol Security) is an Internet Engineering Task Force (IETF) standard suite of protocols between two communication points across the IP network that provide data authentication, integrity, and confidentiality. It also defines the encrypted, decrypted, and authenticated packets. The protocols needed for secure key exchange and key management are defined in it.

Uses of IP Security:
IPsec can be used to do the following things:

- To encrypt application layer data.
- To provide security for routers sending routing data across the public internet.
- To provide authentication without encryption, like to authenticate that the data originates from a known sender.
- To protect network data by setting up circuits using IPsec tunnelling in which all data being sent between the two endpoints is encrypted, as with a Virtual Private Network(VPN) connection.

Components of IP Security:
It has the following components:

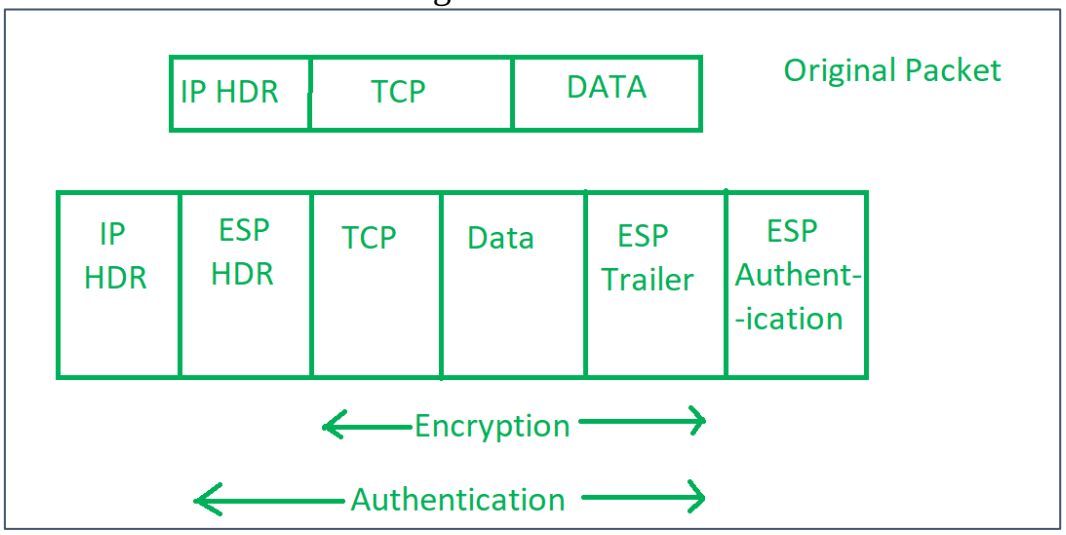
1. Encapsulating Security Payload (ESP)
2. Authentication Header (AH)
3. Internet Key Exchange (IKE)

1. Encapsulating Security Payload (ESP): It provides data integrity, encryption, authentication, and anti-replay. It also provides authentication for payload.
2. Authentication Header (AH): It also provides data integrity, authentication, and anti-replay and it does not provide encryption. The anti-replay protection protects against the unauthorized transmission of packets. It does not protect data confidentiality.

IP Header

3. Internet Key Exchange (IKE): It is a network security protocol designed to dynamically exchange encryption keys and find a way over Security Association

(SA) between 2 devices. The Security Association (SA) establishes shared security attributes between 2 network entities to support secure communication. The Key Management Protocol (ISAKMP) and Internet Security Association provides a framework for authentication and key exchange. ISAKMP tells how the setup of the Security Associations (SAs) and how direct connections between two hosts are using IPsec. Internet Key Exchange (IKE) provides message content protection and also an open frame for implementing standard algorithms such as SHA and MD5. The algorithm's IP sec users produce a unique identifier for each packet. This identifier then allows a device to determine whether a packet has been correct or not. Packets that are not authorized are discarded and not given to the receiver.

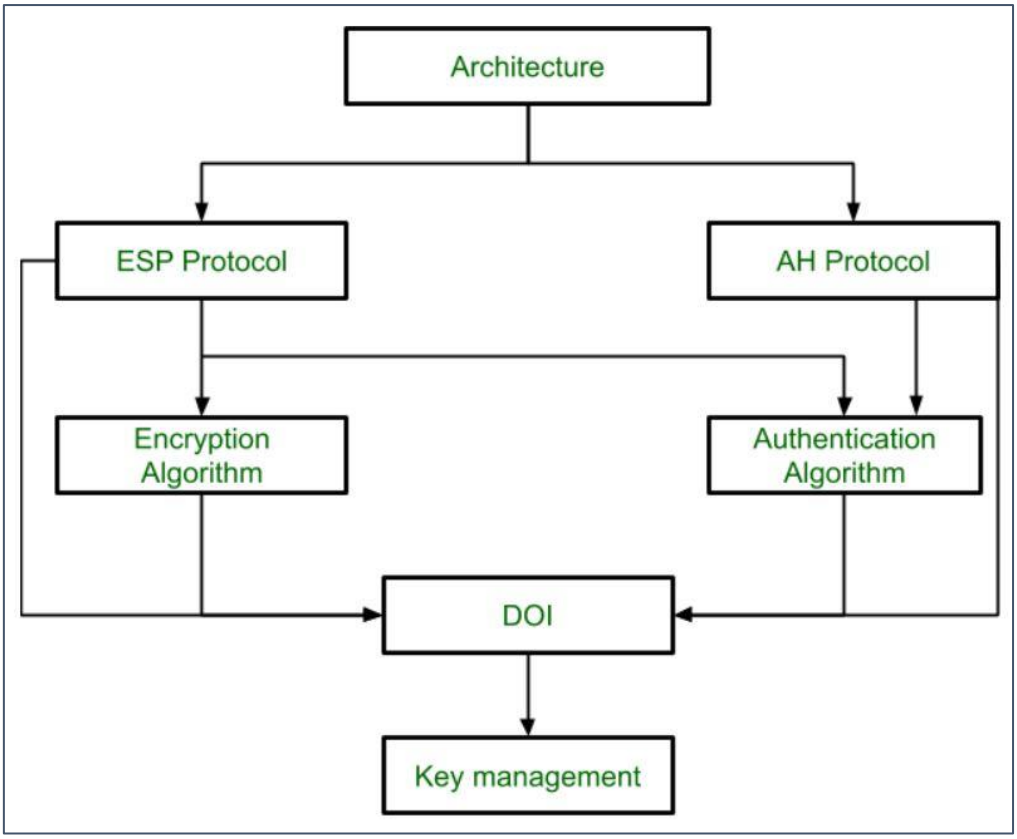


Packets in Internet Protocol

IP Security Architecture

IPSec (IP Security) architecture uses two protocols to secure the traffic or data flow. These protocols are ESP (Encapsulation Security Payload) and AH (Authentication Header). IPSec Architecture includes protocols, algorithms, DOI, and Key Management. All these components are very important in order to provide the three main services:

- Confidentiality
- Authenticity
- Integrity



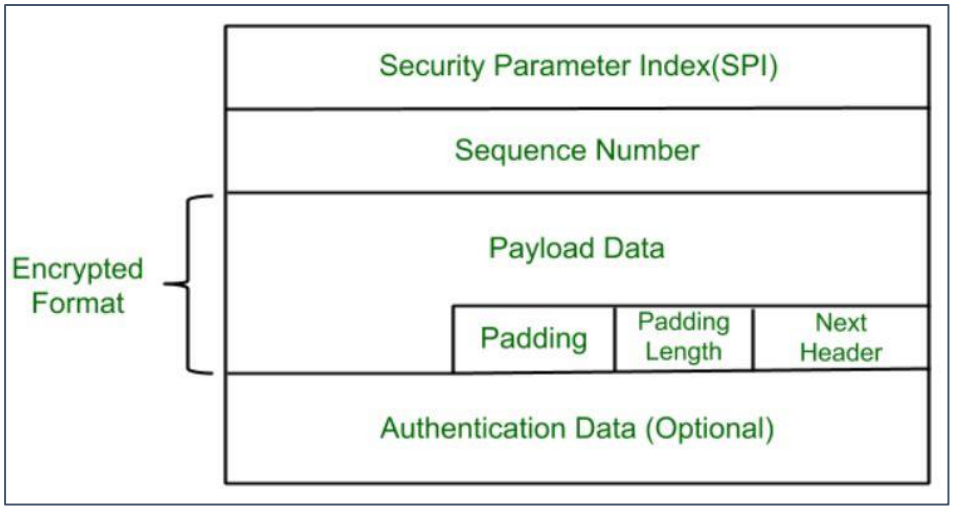
IP Security Architecture

1. Architecture: Architecture or IP Security Architecture covers the general concepts, definitions, protocols, algorithms, and security requirements of IP Security technology.

2. ESP Protocol: ESP(Encapsulation Security Payload) provides a confidentiality service. Encapsulation Security Payload is implemented in either two ways:

- ESP with optional Authentication.
- ESP with Authentication.

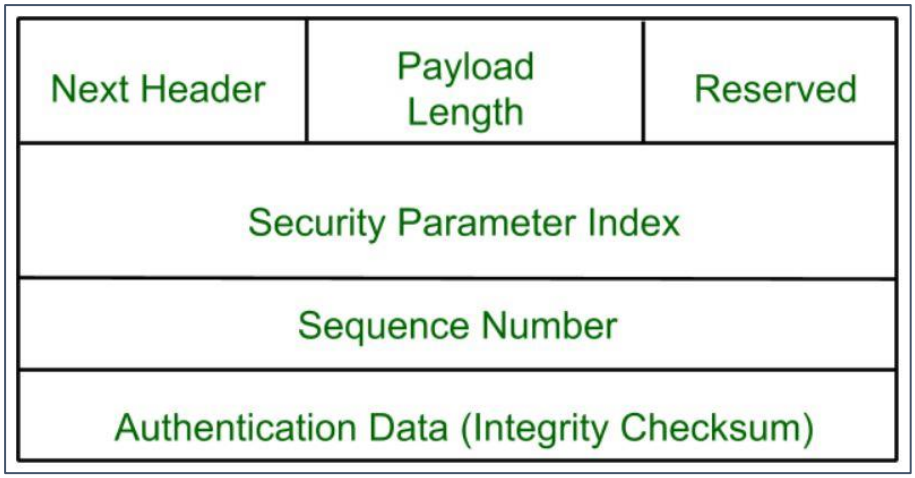
Packet Format:



- **Security Parameter Index(SPI):** This parameter is used by Security Association. It is used to give a unique number to the connection built between the Client and Server.
- **Sequence Number:** Unique Sequence numbers are allotted to every packet so that on the receiver side packets can be arranged properly.
- **Payload Data:** Payload data means the actual data or the actual message. The Payload data is in an encrypted format to achieve confidentiality.
- **Padding:** Extra bits of space are added to the original message in order to ensure confidentiality. Padding length is the size of the added bits of space in the original message.
- **Next Header:** Next header means the next payload or next actual data.
- **Authentication Data** This field is optional in ESP protocol packet format.

3. Encryption algorithm: The encryption algorithm is the document that describes various encryption algorithms used for Encapsulation Security Payload.

4. AH Protocol: AH (Authentication Header) Protocol provides both Authentication and Integrity service. Authentication Header is implemented in one way only: Authentication along with Integrity.



Authentication Header covers the packet format and general issues related to the use of AH for packet authentication and integrity.

5. Authentication Algorithm: The authentication Algorithm contains the set of documents that describe the authentication algorithm used for AH and for the authentication option of ESP.

6. DOI (Domain of Interpretation): DOI is the identifier that supports both AH and ESP protocols. It contains values needed for documentation related to each other.

7. Key Management: Key Management contains the document that describes how the keys are exchanged between sender and receiver.

Features of IPSec

1. **Authentication:** IPSec provides authentication of IP packets using digital signatures or shared secrets. This helps ensure that the packets are not tampered with or forged.
2. **Confidentiality:** IPSec provides confidentiality by encrypting IP packets, preventing eavesdropping on the network traffic.
3. **Integrity:** IPSec provides integrity by ensuring that IP packets have not been modified or corrupted during transmission.
4. **Key management:** IPSec provides key management services, including key exchange and key revocation, to ensure that cryptographic keys are securely managed.
5. **Tunnelling:** IPSec supports tunnelling, allowing IP packets to be encapsulated within another protocol, such as GRE (Generic Routing Encapsulation) or L2TP (Layer 2 Tunnelling Protocol).
6. **Flexibility:** IPSec can be configured to provide security for a wide range of network topologies, including point-to-point, site-to-site, and remote access connections.
7. **Interoperability:** IPSec is an open standard protocol, which means that it is supported by a wide range of vendors and can be used in heterogeneous environments.

Advantages of IPSec:

1. **Strong security:** IPSec provides strong cryptographic security services that help protect sensitive data and ensure network privacy and integrity.
2. **Wide compatibility:** IPSec is an open standard protocol that is widely supported by vendors and can be used in heterogeneous environments.
3. **Flexibility:** IPSec can be configured to provide security for a wide range of network topologies, including point-to-point, site-to-site, and remote access connections.
4. **Scalability:** IPSec can be used to secure large-scale networks and can be scaled up or down as needed.
5. **Improved network performance:** IPSec can help improve network performance by reducing network congestion and improving network efficiency.

Disadvantages of IPSec:

1. **Configuration complexity:** IPSec can be complex to configure and requires specialized knowledge and skills.
2. **Compatibility issues:** IPSec can have compatibility issues with some network devices and applications, which can lead to interoperability problems.
3. **Performance impact:** IPSec can impact network performance due to the overhead of encryption and decryption of IP packets.
4. **Key management:** IPSec requires effective key management to ensure the security of the cryptographic keys used for encryption and authentication.
5. **Limited protection:** IPSec only provides protection for IP traffic, and other protocols such as ICMP, DNS, and routing protocols may still be vulnerable to attacks.

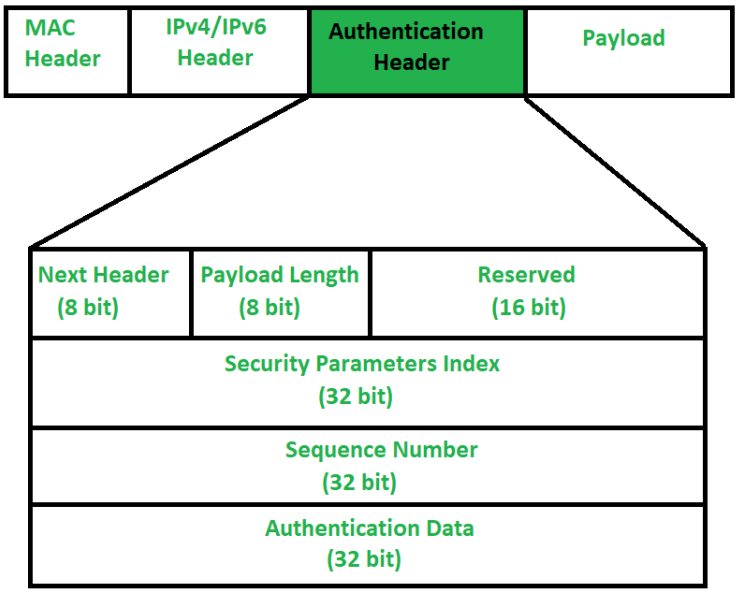
Authentication Header:

There are two main advantages that Authentication Header provides,

- **Message Integrity:** It means, message is not modified while coming from the source.
- **Source Authentication:** It means, the source is exactly the source from whom we were expecting data.

When packet is sent from source A to Destination B, it consists of data that we need to send and header which consist of information regarding packet. Authentication Header verifies origin of data and also payload to confirm if there has been modification done in between, during transmission between source and destination. However, in transit, values of some IP header fields might change (like- Hop count, options, extension headers). So, values of such fields cannot be protected from Authentication header. Authentication header cannot protect every field of IP header. It provides protection to fields which are essential to be protected.

Authentication Header : The question may arise, that how IP header will know that adjacent Extension header is Authentication Header. Well, there is protocol field in IP Header which tells type of header that is present in packet. So, protocol field in IP Header should have value of "51" in order to detect Authentication Header.



1. **Next Header –** Next Header is 8-bit field that identifies type of header present after Authentication Header. In case of TCP, UDP or destination header or some other extension header it will store correspondence IP protocol number . Like, number 4 in this field will

indicate IPv4, number 41 will indicate IPv6 and number 6 will indicate TCP.

2. **Payload Length –** Payload length is length of Authentication header and here we use scaling factor of 4. Whatever be size of header, divide it by 4 and then subtract by 2. We are subtracting by 2 because we're not counting first 8 bytes of Authentication header, which is first two row of picture given above. It means we are not including Next Header, Payload length, Reserved and Security Parameter index in calculating payload length. Like, say if payload length is given to be X. Then (X+2)*4 will be original Authentication header length.
3. **Reserved –** This is 16-bit field which is set to "zero" by sender as this field is reserved for future use.

4. **Security Parameter Index (SPI) –** It is arbitrary 32-bit field. It is very important field which identifies all packets which belongs to present connection. If we're sending data from Source A to Destination B. Both A and B will already know algorithm and key they are going to use. So, for Authentication, hashing function and key will be required which only source and destination will know about. Secret key between A and B is exchanged by method of Diffie Hellman algorithm. So Hashing algorithm and secret key for Security parameter index of connection will be fixed. Before data transfer starts security association needs to be established. In **Security Association**, both parties needs to communicate prior to data exchange. Security association tells what is security parameter index, hashing algorithm and secret key that are being used.

5. **Sequence Number –** This unsigned 32-bit field contains counter value that increases by one for each packet sent. Every packet will need sequence number. It will start from 0 and will go till – 1 and there will be no wrap around. Say, if all sequence numbers are over and none of it is left but we cannot wrap around as it is not allowed. So, we will end connection and re-establish connection again to resume transfer of remaining data from sequence number 0. Basically, sequence numbers are used to stop replay attack. In Replay attack, if same message is sent twice or more, receiver won't be able to know if both messages are sent from a single source or not. Say, I am requesting 100$ from receiver and Intruder in between asked for another 100$. Receiver won't be able to know that there is intruder in between.

6. **Authentication Data (Integrity Check Value) –** Authentication data is variable length field that contains Integrity Check Value

(ICV) for packet. Using hashing algorithm and secret key, sender will create message digest which will be sent to receiver. Receiver on other hand will use same hashing algorithm and secret key. If both message digest matches then receiver will accept data. Otherwise, receiver will discard it by saying that message has been modified in between. So basically, authentication data is used to verify integrity of transmission. Also, length of Authentication data depends upon hashing algorithm you choose.

Modes of operations in Authentication Header:
There are two modes in the authentication header

1. **Authentication Header Transport Mode:** In the authentication header transport mode, it is lies between the original IP Header and IP Packets original TCP header.
2. **Authentication Header Tunnel Mode:** In this authentication header tunnel mode, the original IP packet is authenticated entire and the authentication header is inserted between the **original IP header and new outer IP header**. Here, the inner IP header contains the ultimate source IP address and destination IP address. whereas the outer IP header contains different IP address that is IP address of the firewalls or other security gateways.

How does the header deals with Replay attack?

- In a replay attack, the attacker a copy of an authenticated packet and then send to the intended destination. As the same packet received twice, the destination user can face some problems. To reduce this problem, **the authentication header use a sequence number field.**
- At this **initial stage**, the value of this field is set to **0**. whenever the sender sends the packets to the same receiver over the same SA, it **increments the fields value by 1.** If the number of packets over the same increase this number, then communication with the receiver sender must establishing a new SA with the receiver.
- At the receiver side, the receiver maintains a sliding window size to W. **The default value of W is 64**. This window right edge represents the highest sequence number N received so far for a valid packet. When the receiver gets a packet from the sender, it perform some action. **The appropriate action depends on the sequence number of the packet.**

Encapsulating Security Payload:

Cyber Security is the branch of computer technology that deals with the security of the virtual cloud and internet. Any information that is stored or transmitted through the cloud needs to be secure and safe. Cyber Networking plays a very important role in maintaining that the connection established is secured and content goes through a secured/ safe channel for transmission.

Security in the network is very important and can't be compromised in any situation. Security in Networking particularly in IP Sec or IP Network Security is significant and has some characteristics associated with it.

Characteristics Associated with IPSec:

1. The standardized algorithms present in IP Sec are SHA and MD5.
2. IPSec uniquely identifies every packet, and then authentication is carried out based on verifying the same uniqueness of the packet.
3. IP network or IPSec has an ESP present in it for security purposes.

Here, we will discuss ESP, the structure of ESP, and its importance in security.

Encapsulation security payload, also abbreviated as ESP plays a very important role in network security. ESP or Encapsulation security payload is an individual protocol in IPSec. ESP is responsible for the CIA triad of security (Confidentiality, Integrity, Availability), which is considered significant only when encryption is carried along with them. Securing all payload/ packets/ content in IPv4 and IPv6 is the responsibility of ESP.
As the name suggests, it involves encapsulation of the content/ payload encrypts it to suitable form and then there a security check or authentication takes place for payload in IP Network. Encryption/ encapsulation and security/ authentication make the payload extremely secure and safe from any kind of harm or threat to content/ data/ payload being stolen by any third party. The encryption process is performed by authenticated user, similarly, the decryption process is carried out only when the receiver is verified, thus making the entire process very smooth and secure. The entire encryption that is performed by ESP is carried on the principle of the integrity of payload and not on the typical IP header.

Working of ESP:

1. Encapsulating Security Payload supports both main Transport layer protocols: IPv4 and IPv6 protocols.
2. It performs the functioning of encryption in headers of Internet Protocol or in general say, it resides and performs functions in IP Header.
3. One important thing to note here is that the insertion of ESP is between Internet Protocol and other protocols such as UDP/ TCP/ ICMP.

Modes in ESP:

Encapsulating Security Payload supports two modes, i.e., Transport mode, and tunnel mode.

Tunnel mode:

1. Mandatory in Gateway, tunnel mode holds utmost importance.
2. Here, a new IP Header is created which is used as the outer IP Header followed by ESP.

Transport mode:

1. Here, IP Header is not protected via encryption or authentication, making it vulnerable to threats.
2. Less processing is seen in this mode, so the inclusion of ESP is preferred.

Advantages:

Below listed are the advantages of Encapsulating Security Payload:

1. Encrypting data to provide security.
2. Maintaining a secure gateway for data/ message transmission.
3. Properly authenticating the origin of data.
4. Providing needed data integrity.
5. Maintaining data confidentiality.
6. Helping with antireplay service using authentication header.

Disadvantages:

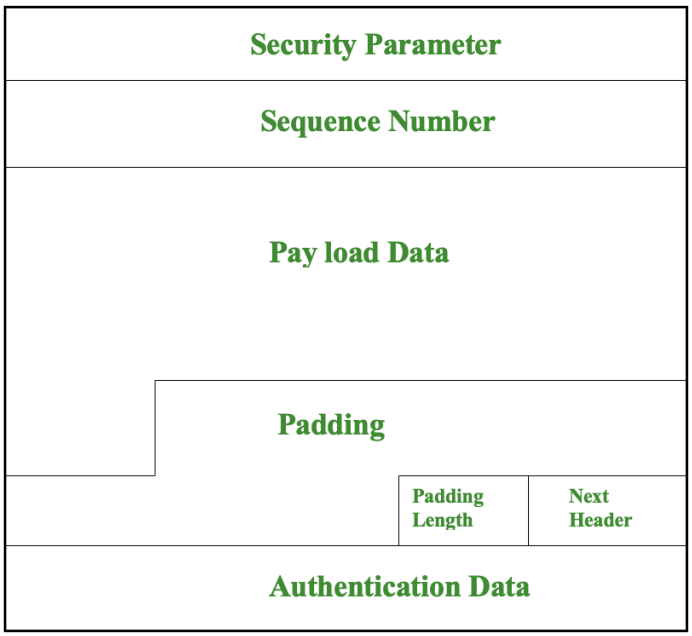
Below listed are the disadvantages of Encapsulating Security Payload:

1. There is a restriction on the encryption method to be used.
2. For global use and implementation, weaker encryptions are mandatory to use.

Components of ESP:

An important point to note is that authentication and security are not provided for the entire IP packet in transport mode. On the other hand, for the tunnel mode, the entire IP packet along with the new packet header is encapsulated.

ESP structure is composed of the following parts as shown below :



ESP Structure

The diagrammatic representation of ESP has the below-mentioned components:

1. Security Parameter :

- Security parameters are assigned a size of 32 bits for use.
- Security Parameter is mandatory to security parameter in ESP for security links and associations.

2. Sequence Number:

- The sequence number is 32 bits in size and works as an incremental counter.
- The first packet has a sequence number 1 assigned to it whenever sent through SA.

3. Payload Data:

- Payload data don't have fixed size and are variable in size to use.
- It refers to the data/ content that is provided security by the method of encryption.

4. **Padding:**
   - Padding has an assigned size of 0-255 bytes assigned to it.
   - Padding is done to ensure that the payload data which needs to be sent securely fits into the cipher block correctly, so for this padding payloads come to the rescue.
5. **Pad Length:**
   - Pad Length is assigned the size of 8 bits to use.
   - It is a measure of pad bytes that are preceding.
6. **Next Header:**
   - The next header is associated with a size of 8 bits to use.
   - It is responsible for determining the data type of payload by studying the first header of the payload.
7. **Authentication Data:**
   - The size associated with authentication data is variable and never fixed for use-case.
   - Authentication data is an optional field that is applicable only when SA is selected. It serves the purpose of providing integrity.